

Using Deep Learning Methods to Classify Wound Types

Members: 黃妤涵、吳佩婷、溫苡均、李佩臻

Advisors : 鄧有光、林先彥

Contents

1. Introduction

2. Dataset

3. Backend Model

4. Analysis

5. Conclusion & Future Work

Motivation

- Wounds of the same category exhibit higher similarity than those of different categories.
- This project primarily focuses on pressure ulcer wound types (Stages 1 to 4).
- The objective is to identify the optimal deep learning model for classifying wounds of the same type.

Dataset



	Image Size	Stage 1	Stage 2	Stage 3	Stage 4	SUM
<u>Roboflow1</u>	640 * 640	468	419	503	464	1854
<u>Roboflow2</u>	640 * 640	219	138	392	421	1170
SUM		687	557	895	885	3024

- Train, val, test = 7:2:1
- Balanced number of images per class
- Approximately 500 images per stage (350:100:50)
- Image Resize to 224*224

Backend Model



Classification model: Efficientnet 、 Resnet

img_width, img_height = 224, 224

batch_size = 64

optimizer: Adam

Epoch: 80

Train, val, test = 7:2:1

Backend Model - Efficientnet

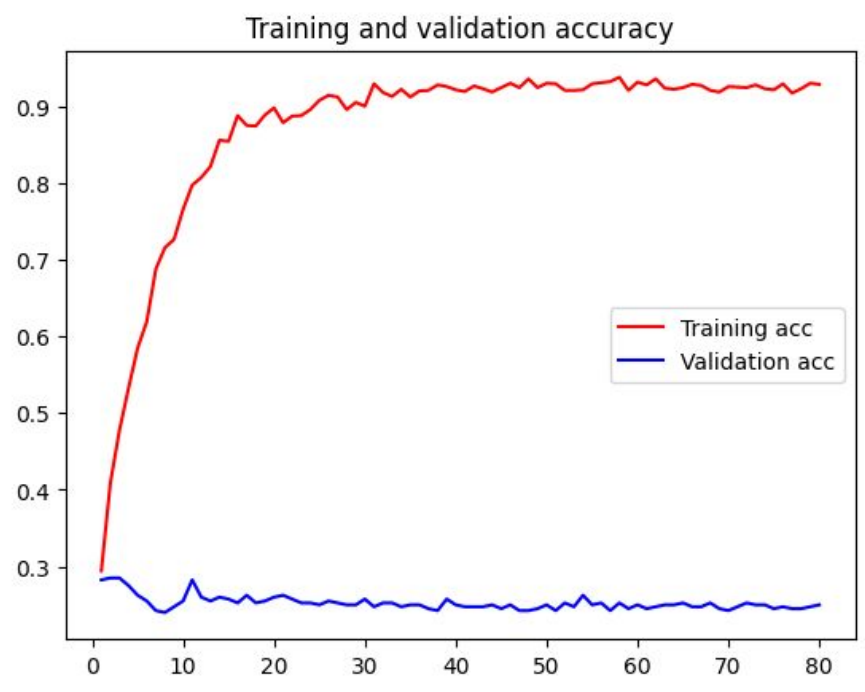
	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy	F1-Score
Roboflow1	0.23	0.35	0.34	0.29	0.3100	0.3040
Roboflow2	0.91	0.83	0.80	0.91	0.8611	0.8643

- ❑ Batch_Size: 16 ~ 128

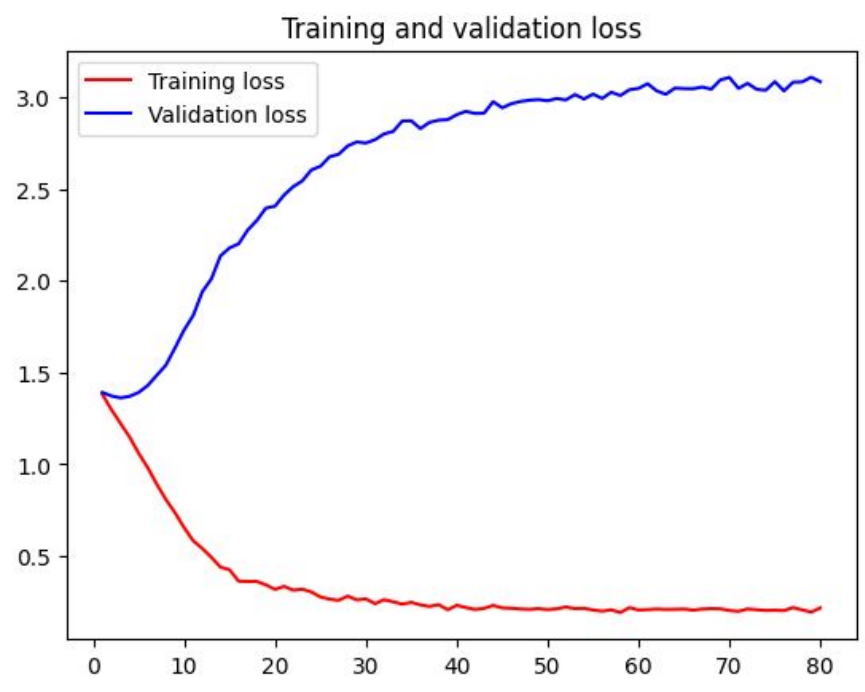
❑ Epoch: 50 ~ 100
- ❑ Learning_rate: 1e-4 ~ 1e-5

❑ Weight_Decay: 2e-3
- ❑ Optimizer: Adam, SGD

Roboflow 1

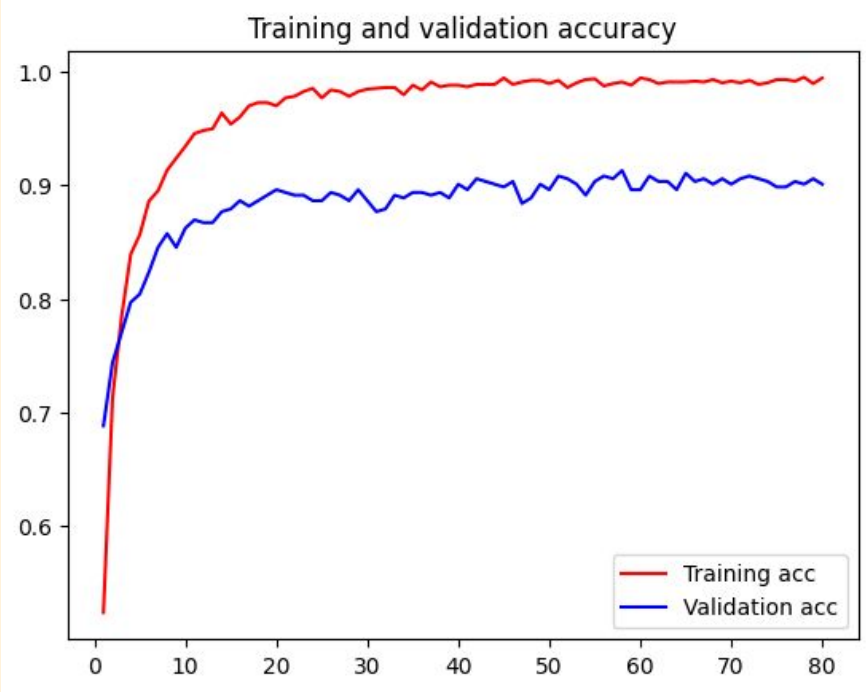


Train

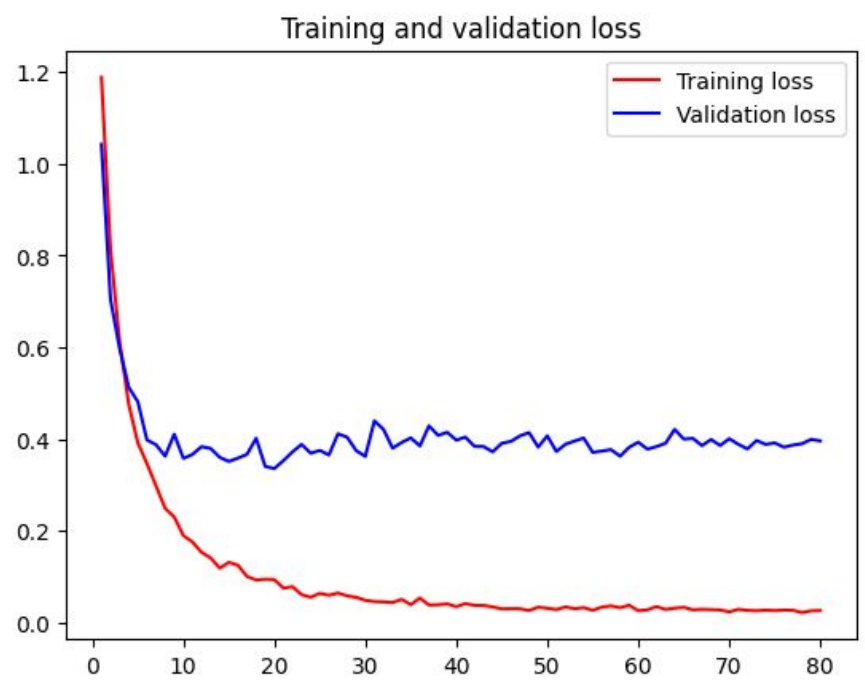


Val

Roboflow 2



Train



Val

Backend Model - Resnet

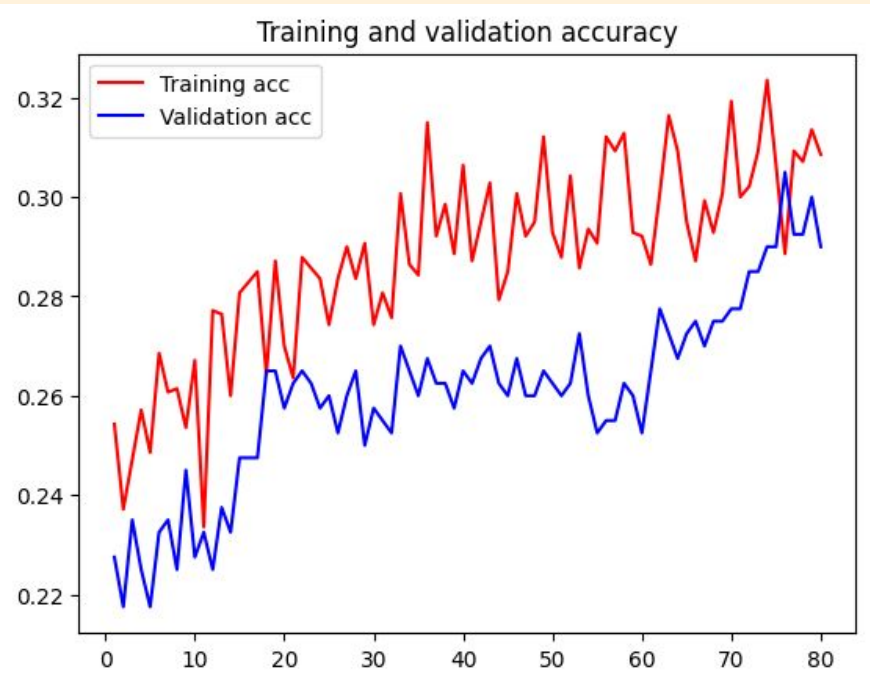
	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy	F1-Score
Roboflow1	0.20	0.38	0.14	0.24	0.2500	0.2381
Roboflow2	0.86	0.67	0.55	0.75	0.7130	0.7072

- ❑ Batch_Size: 16 ~ 128

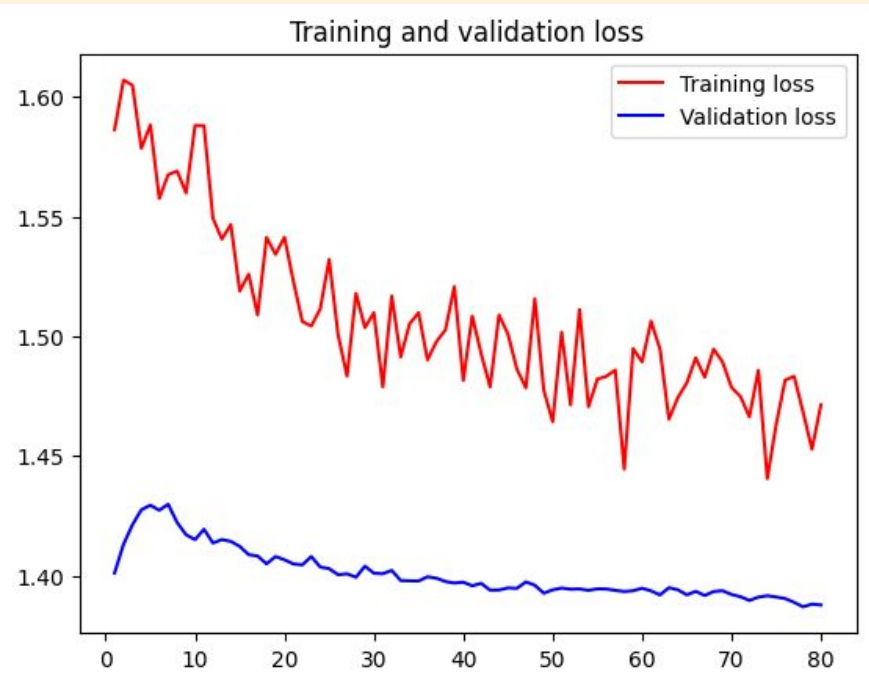
❑ Epoch: 50 ~ 100
- ❑ Learning_rate: 1e-4 ~ 1e-5

❑ Weight_Decay: 2e-3
- ❑ Optimizer: Adam, SGD

Roboflow 1

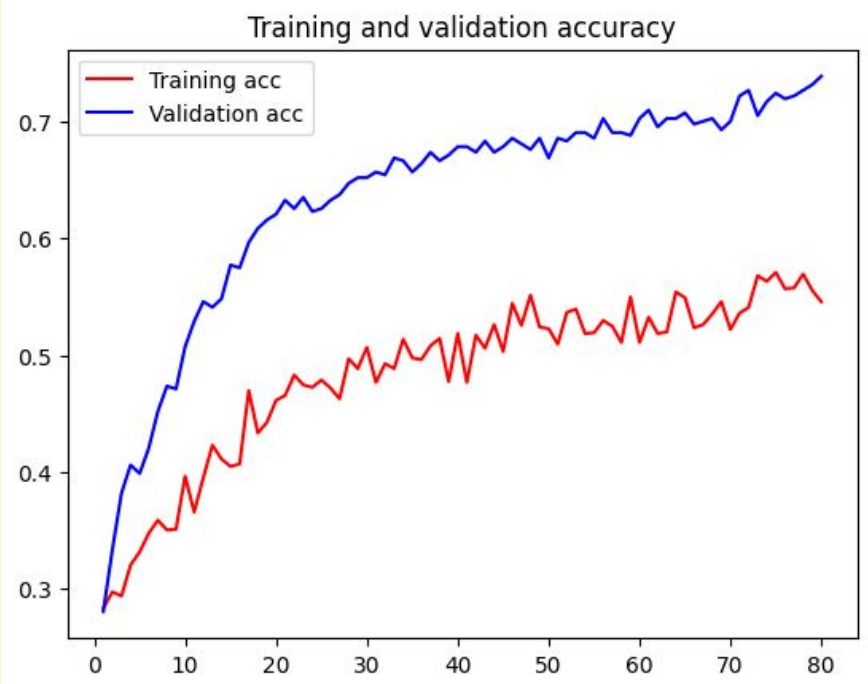


Train

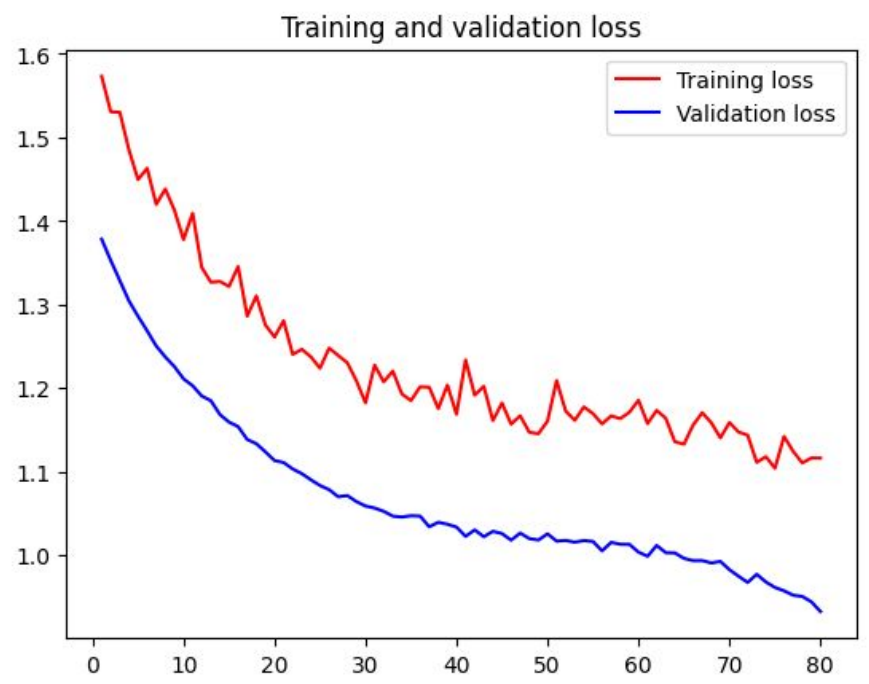


Val

Roboflow 2

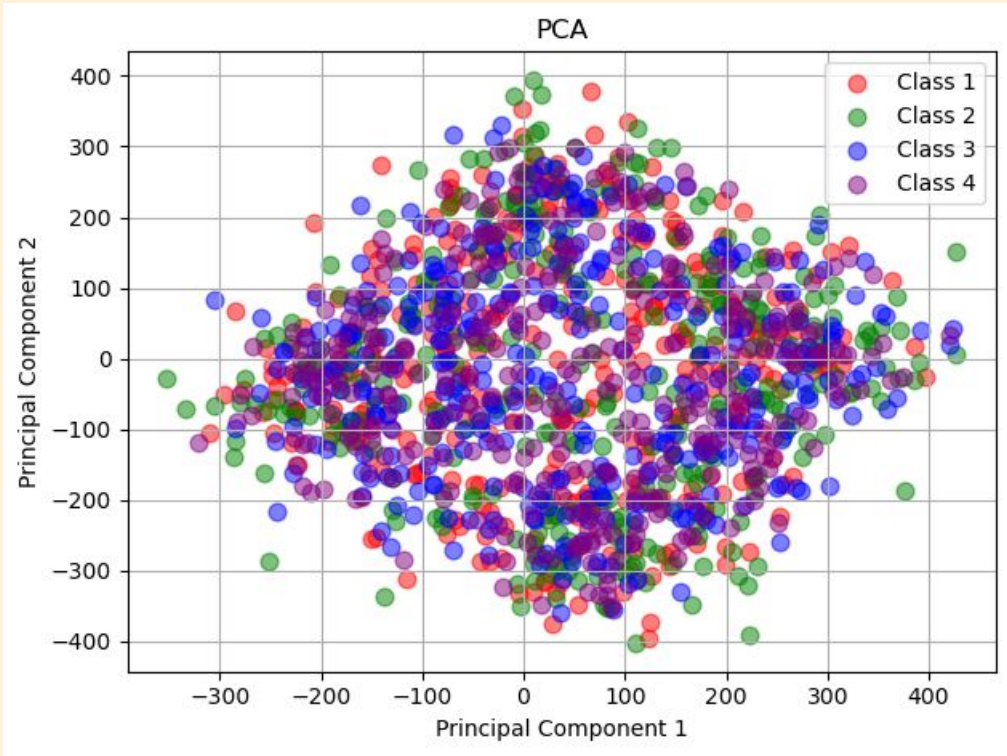


Train

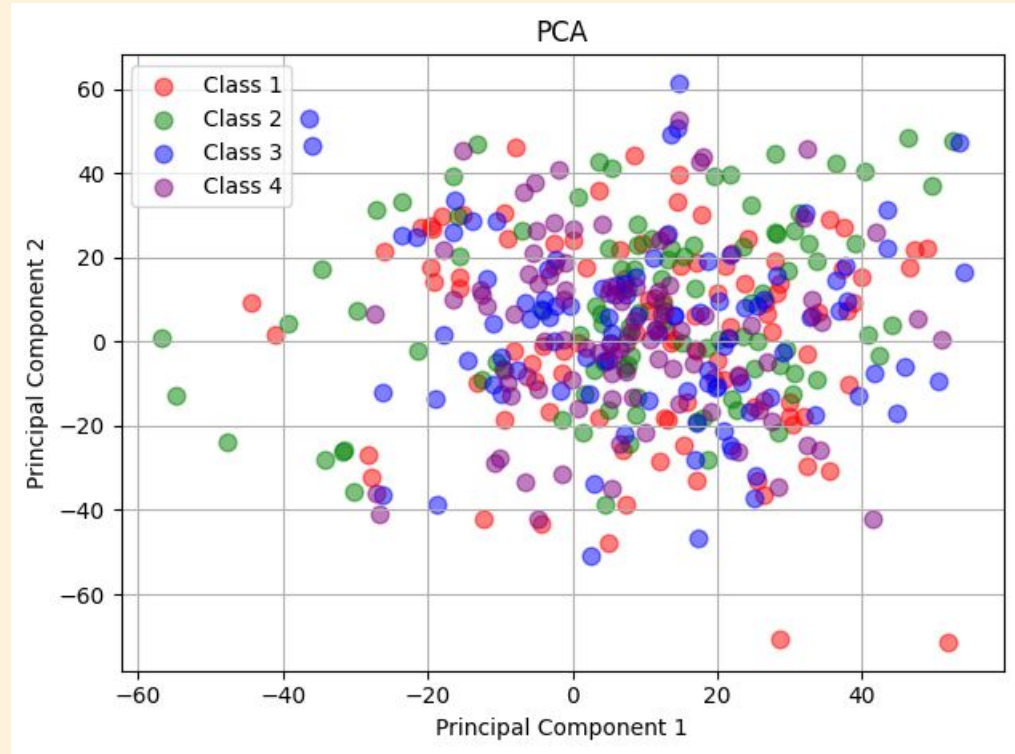


Val

Roboflow 1

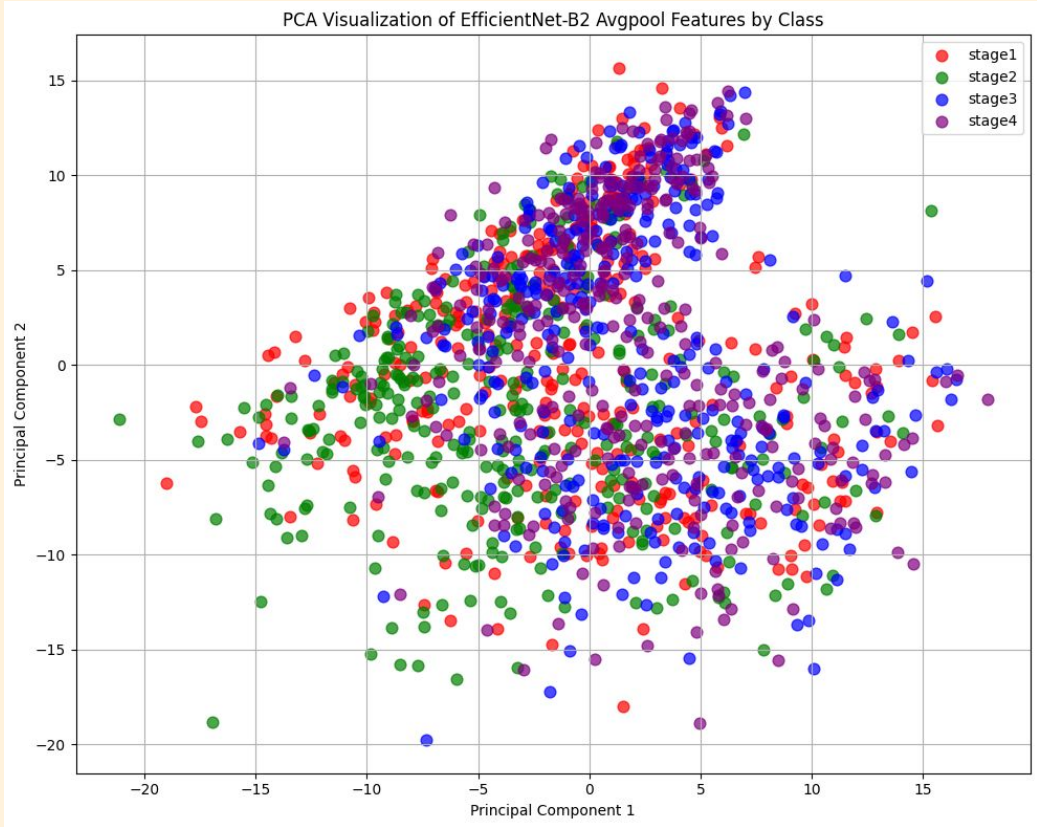


Train

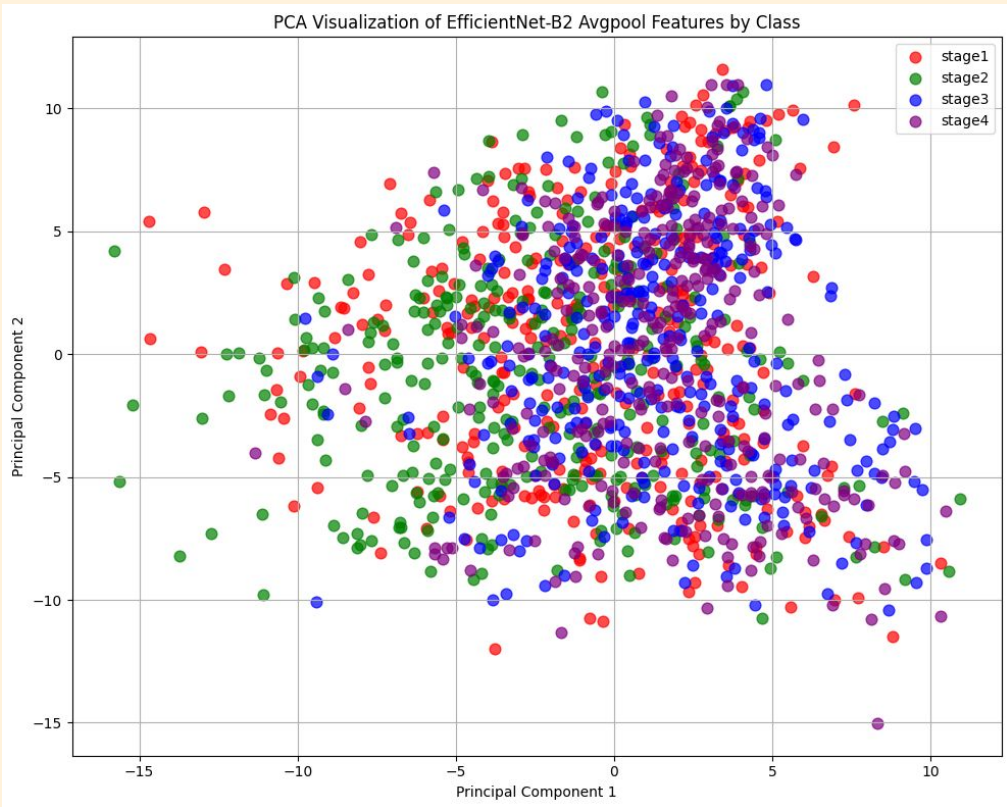


Val

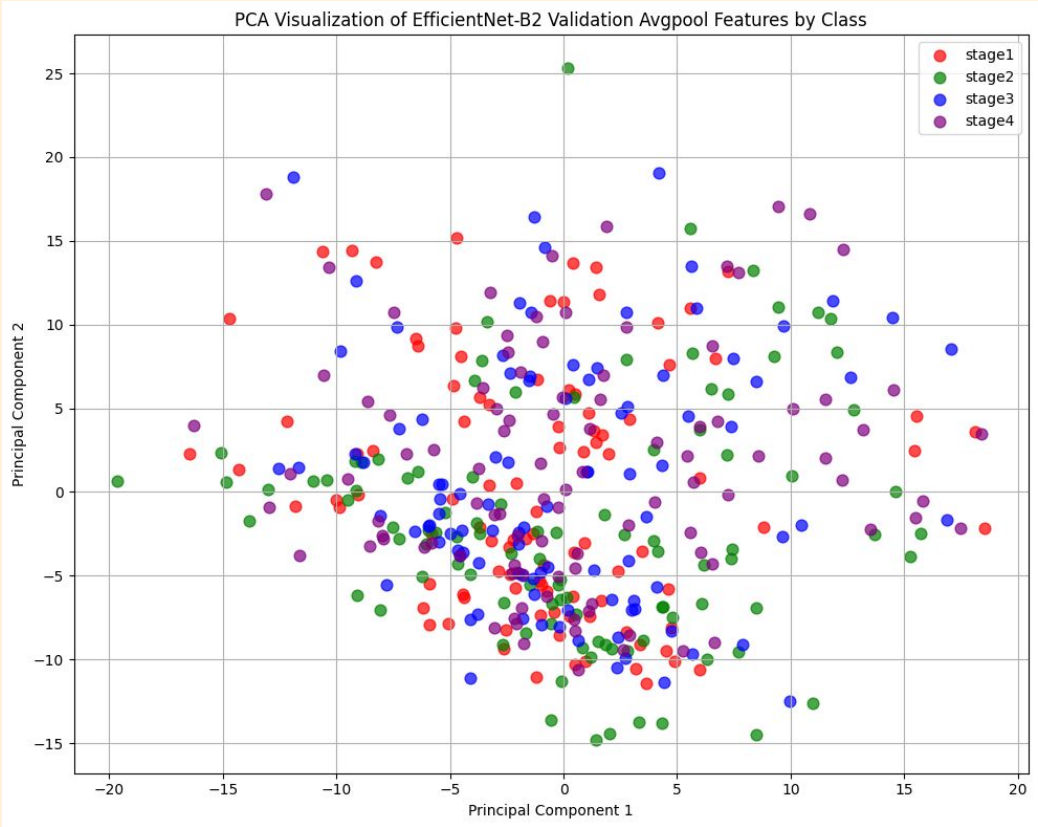
EfficientNet



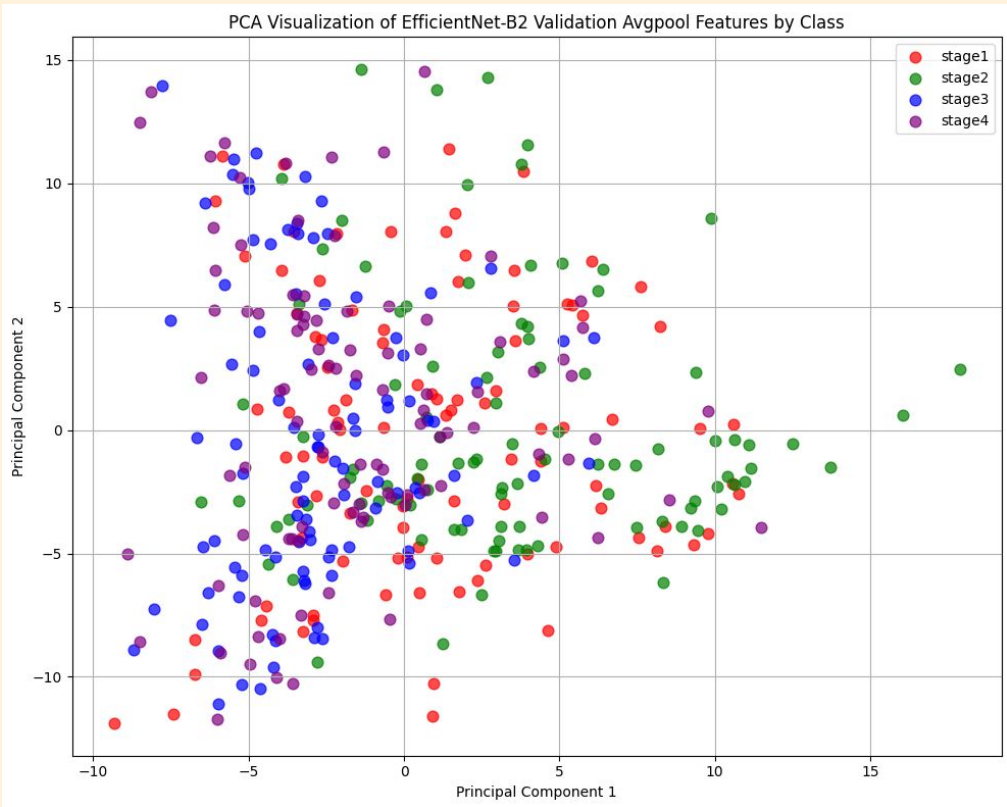
Train



Train

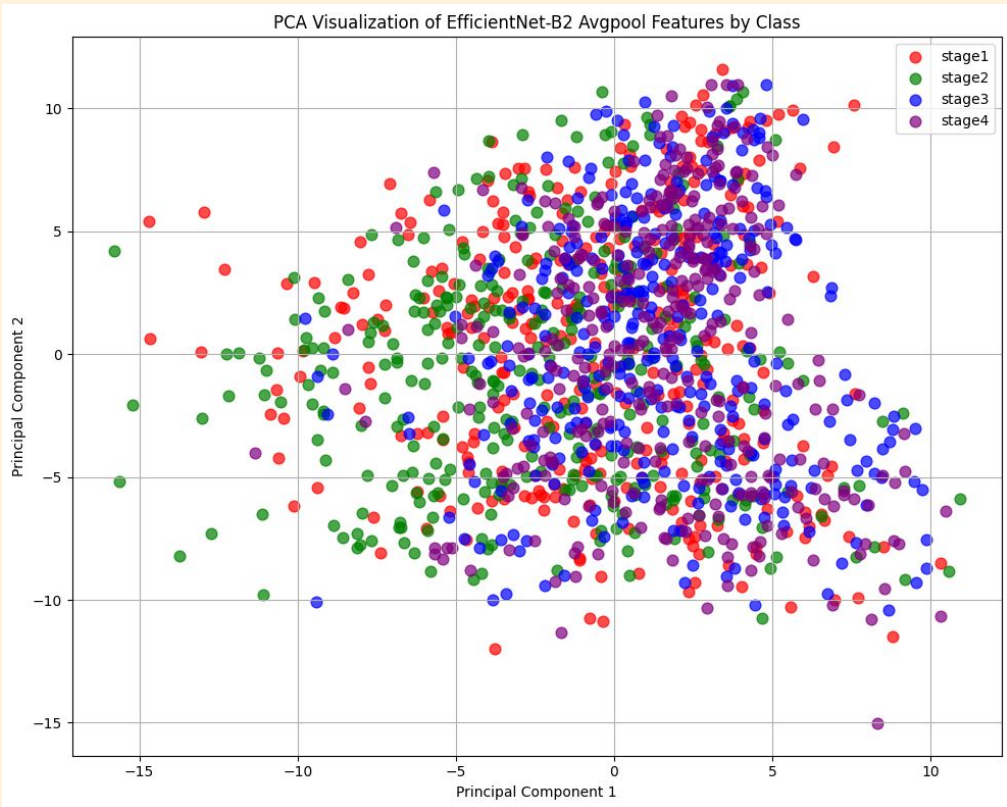


Val

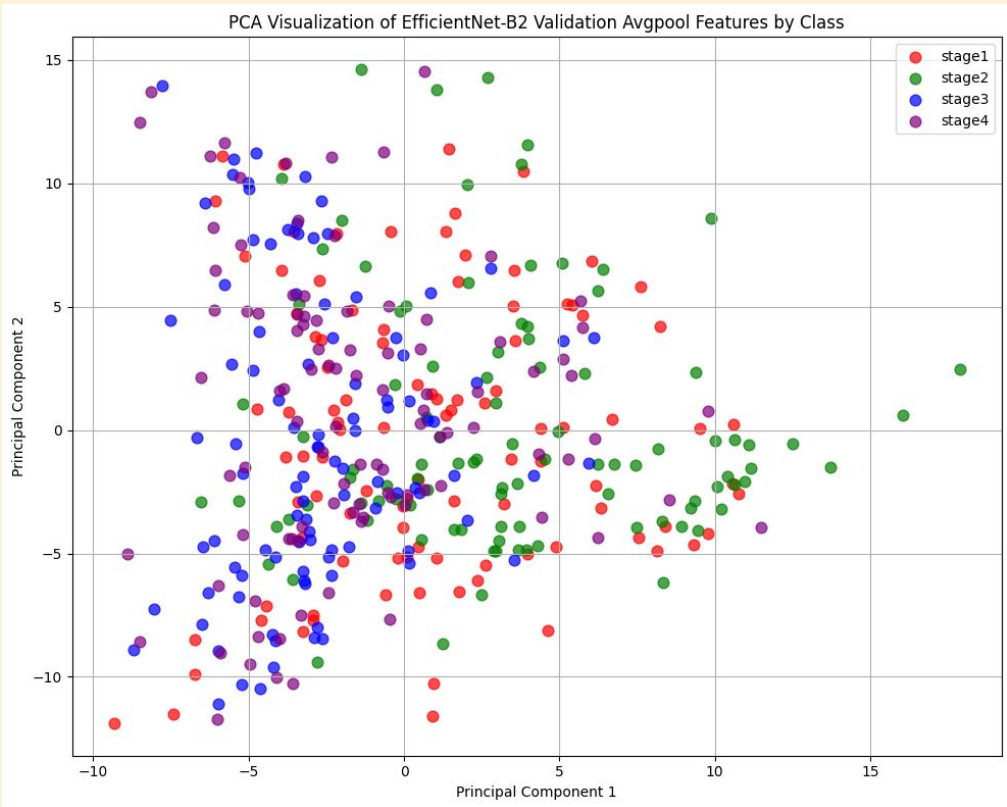


Val

ResNet

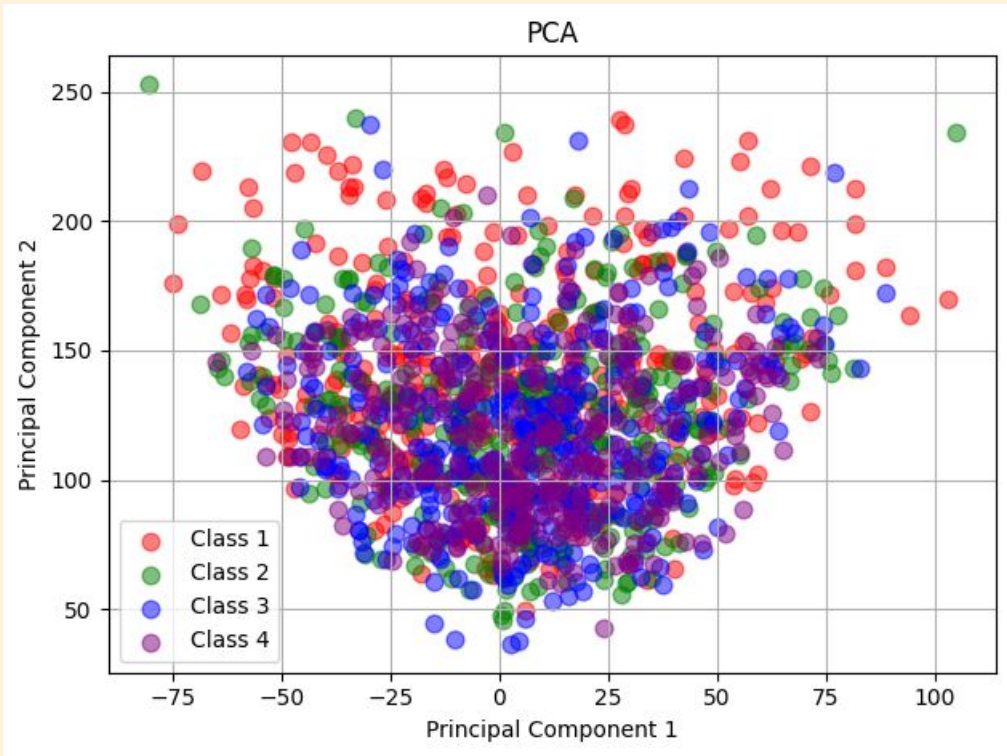


Train

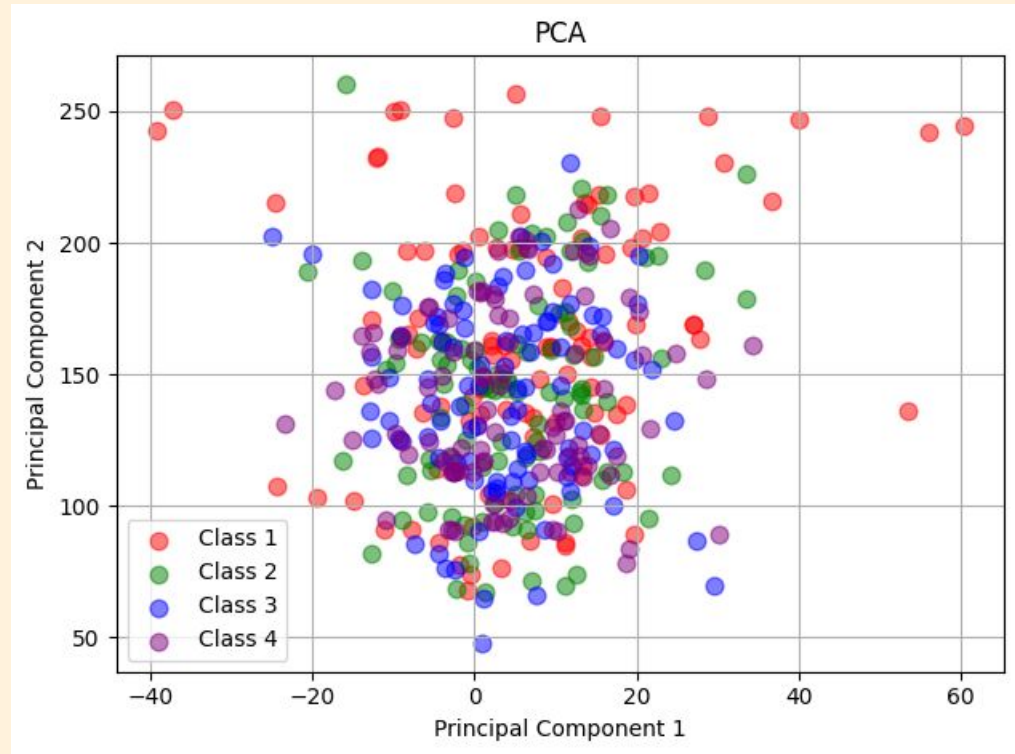


Val

Roboflow 2

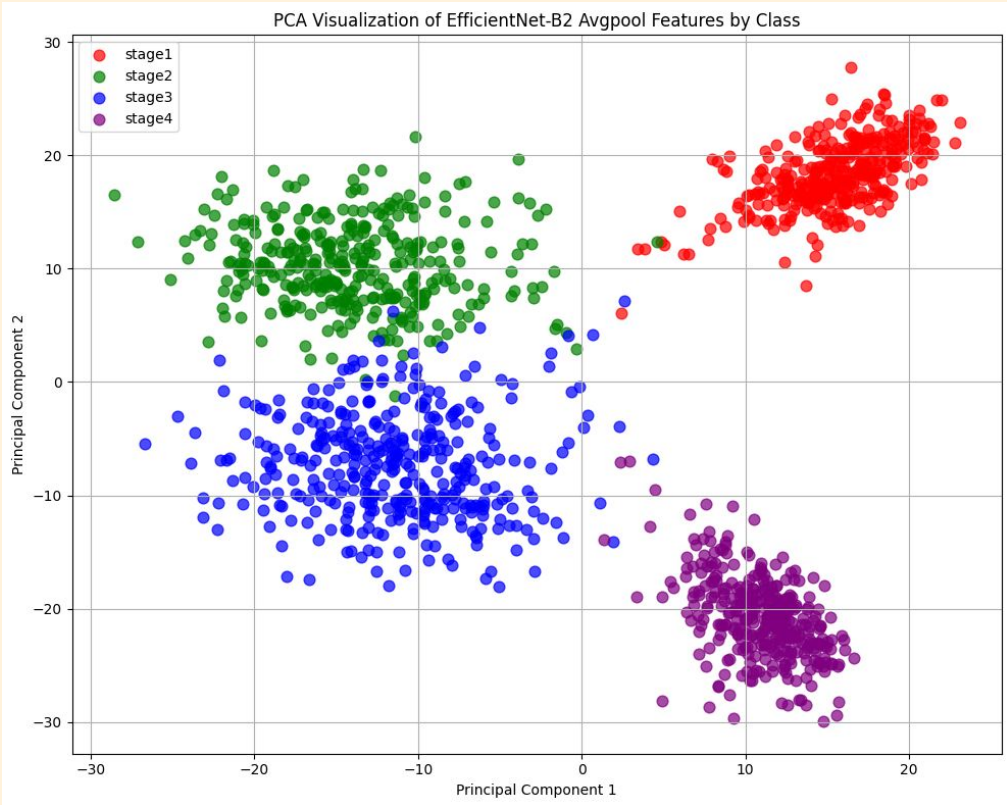


Train

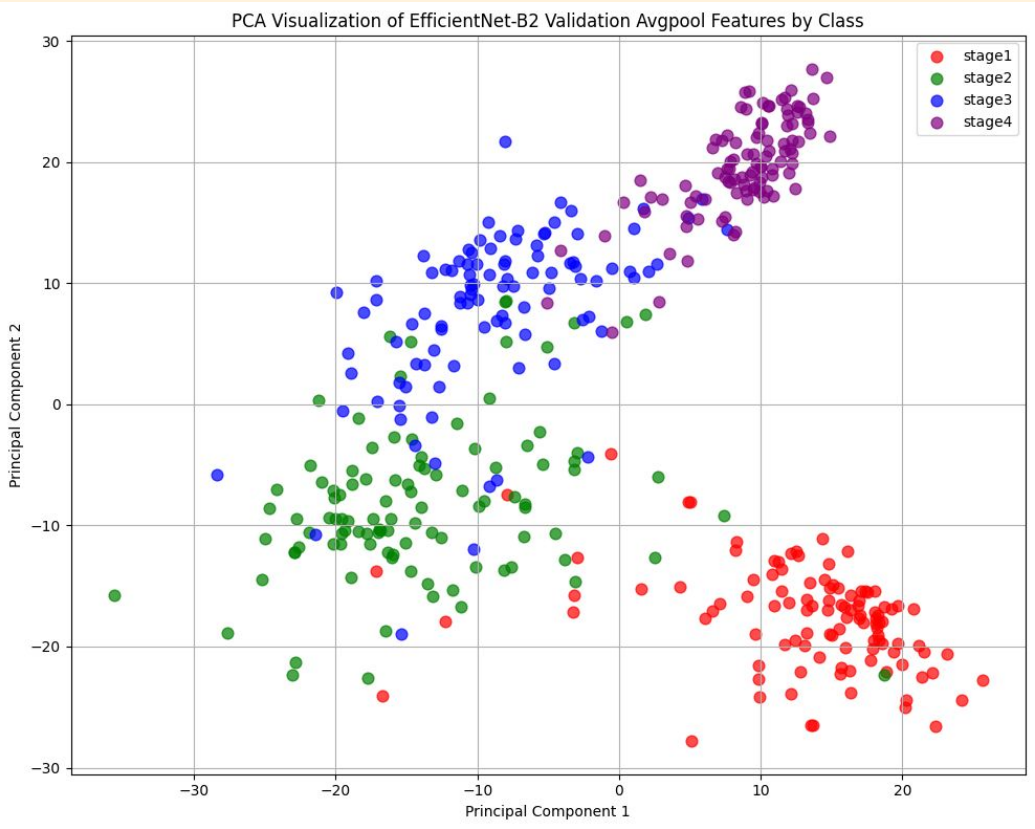


Val

EfficientNet

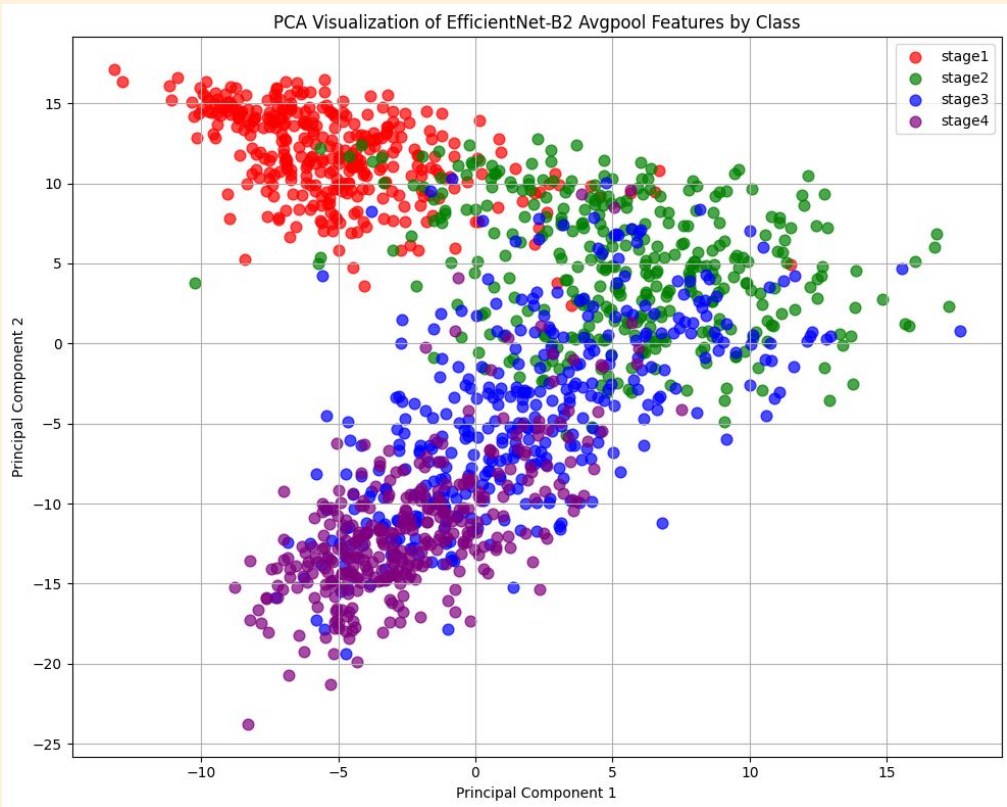


Train

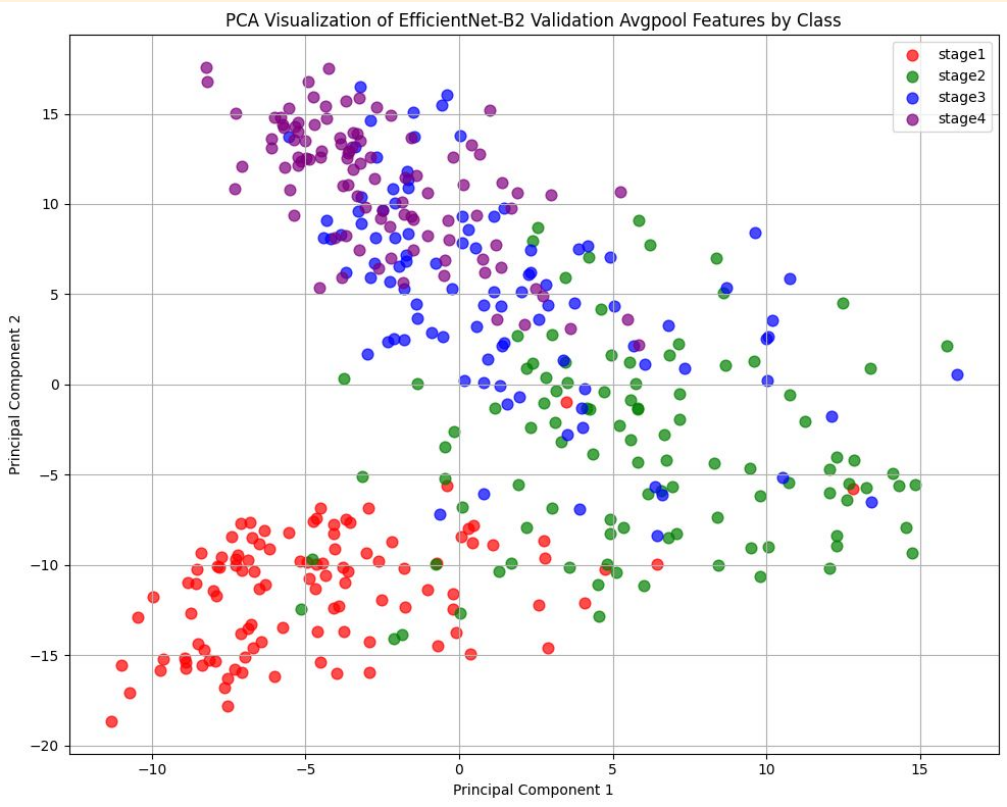


Val

ResNet

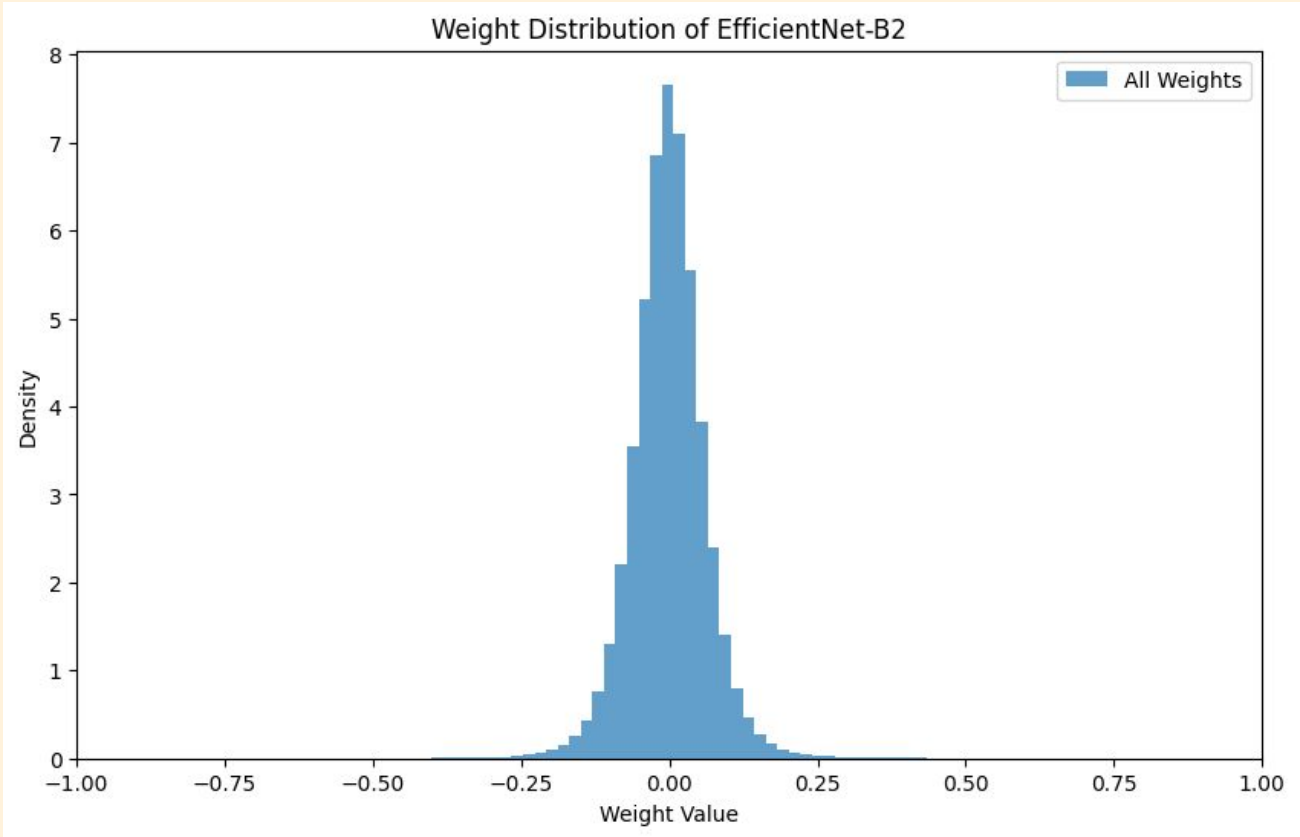


Train

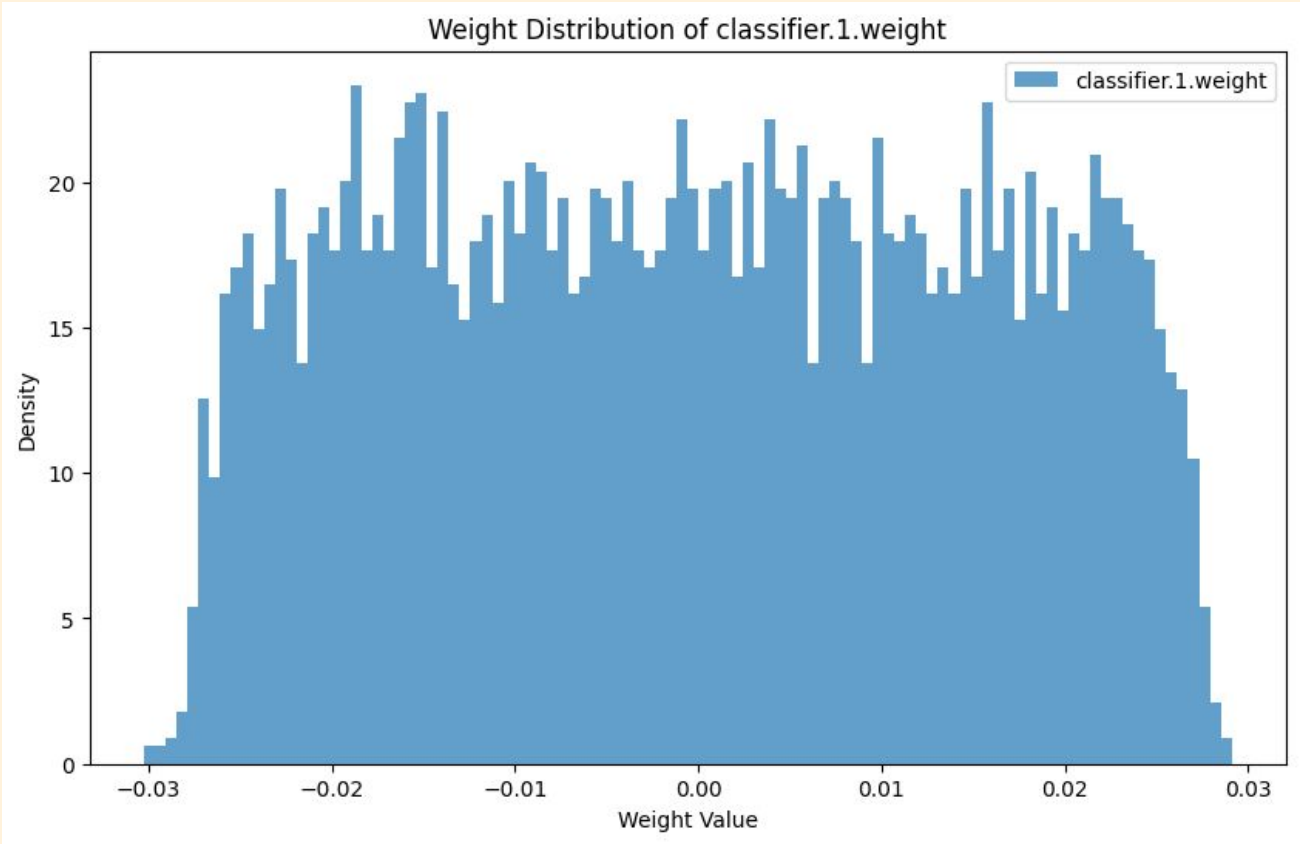


Val

EfficientNet



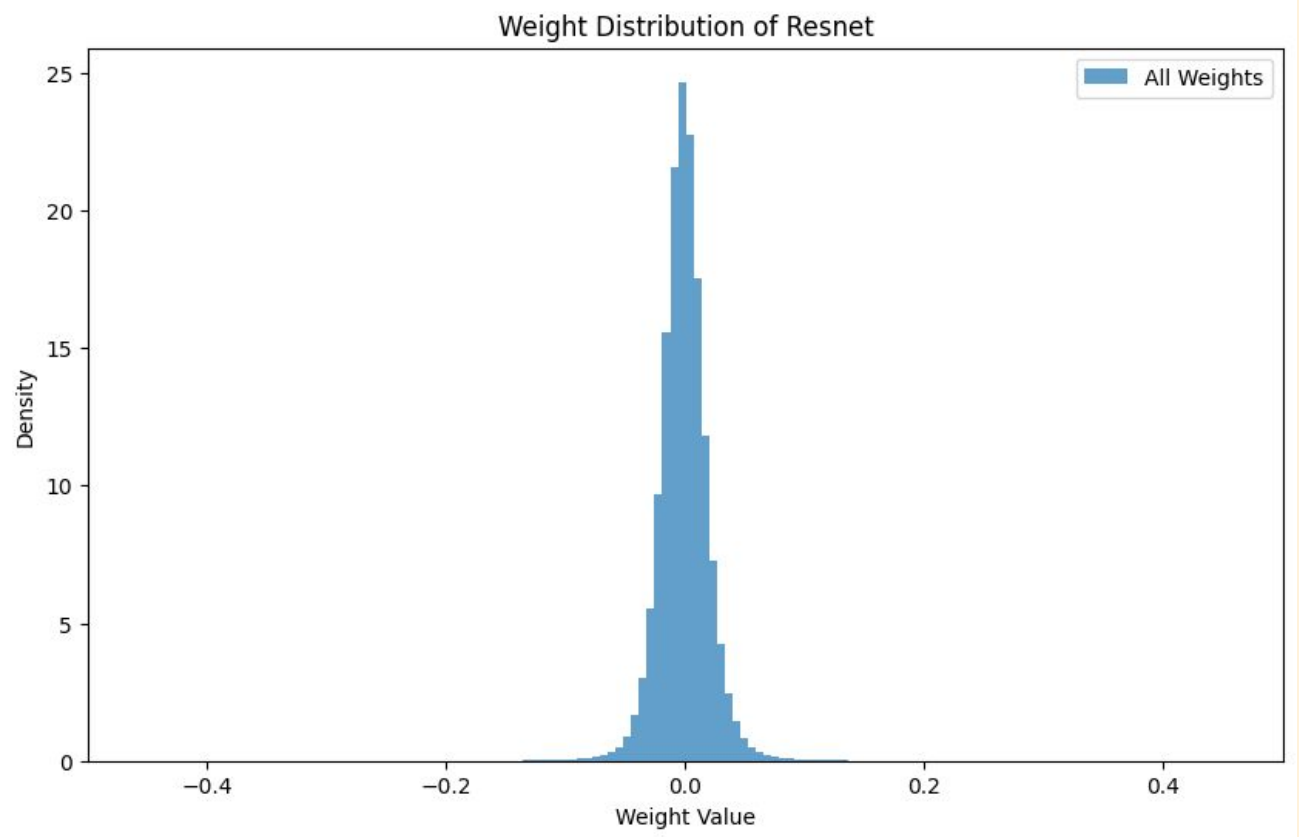
Total



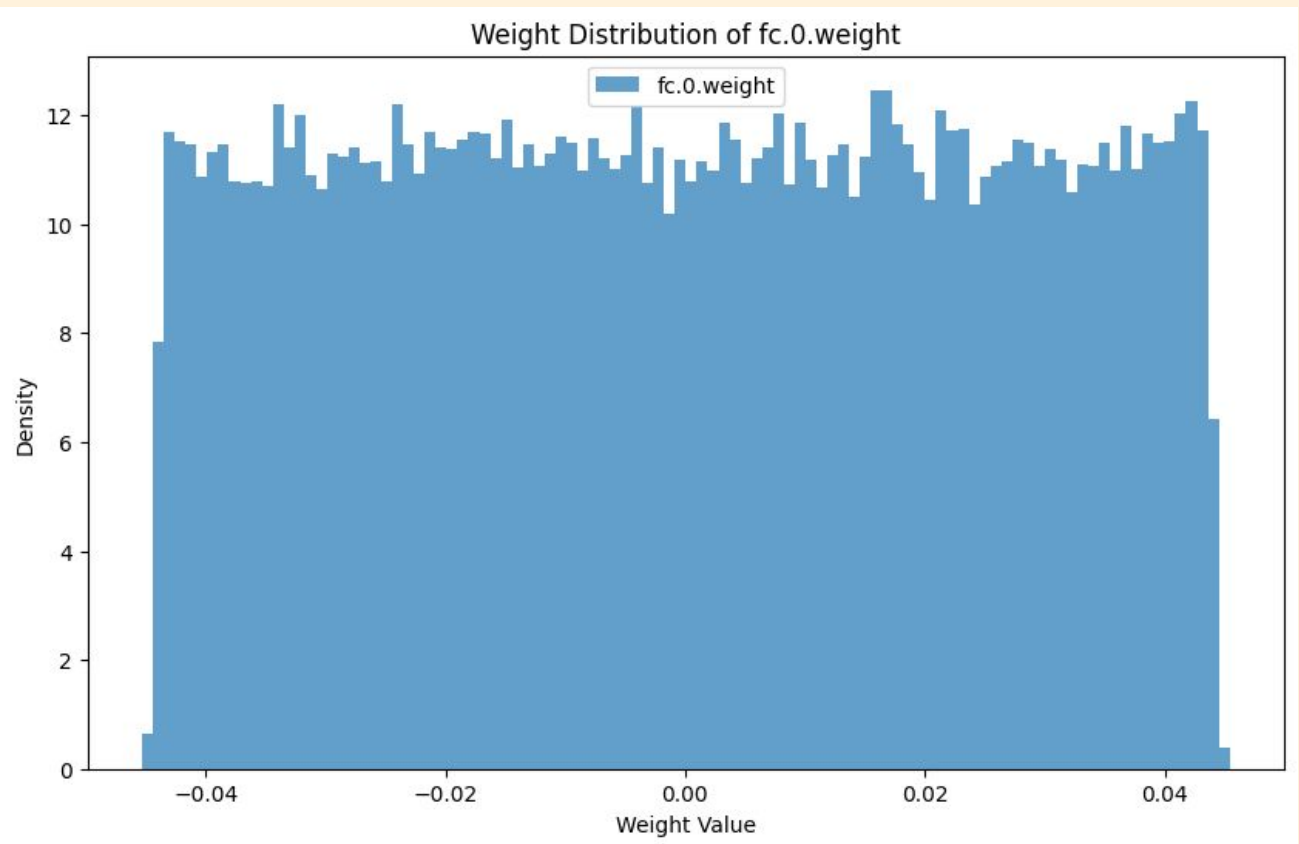
Last Layer

Roboflow 1

ResNet

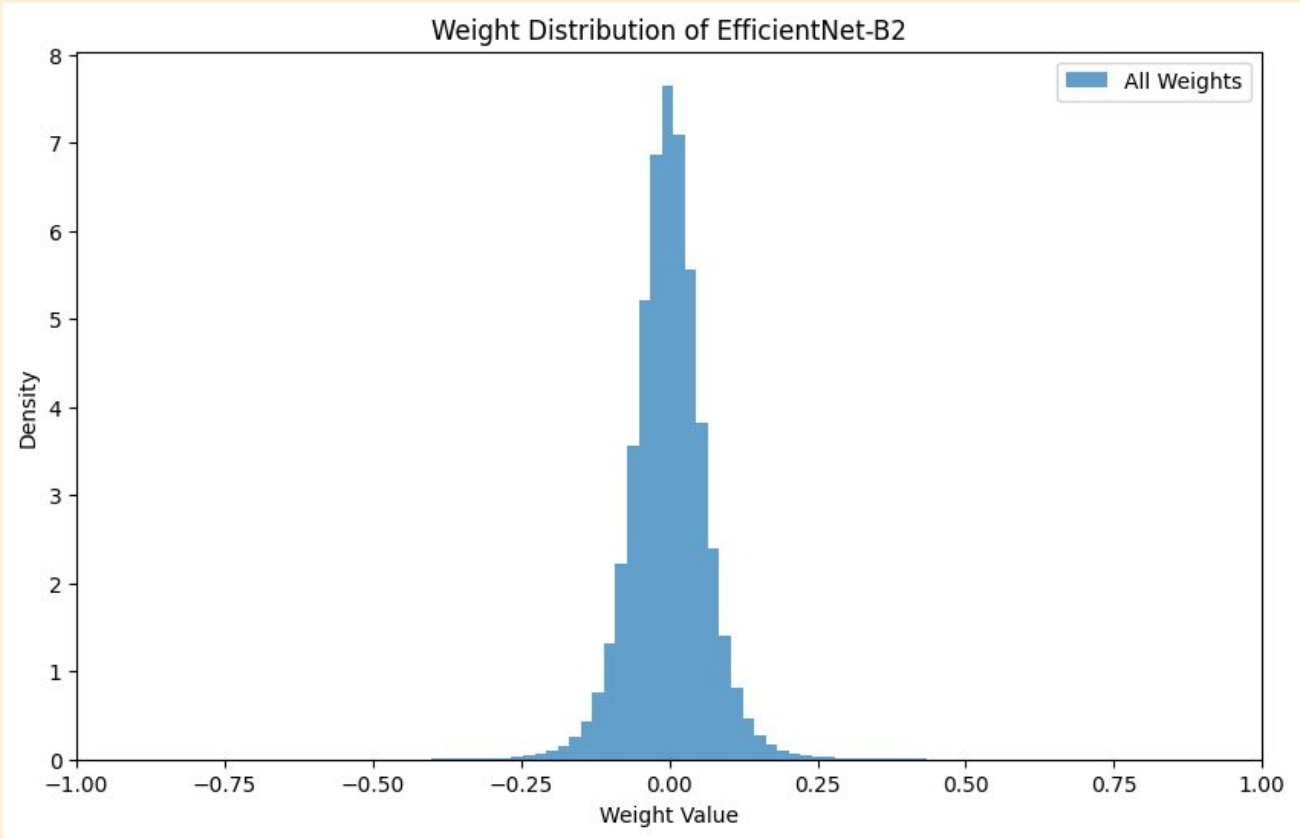


Total

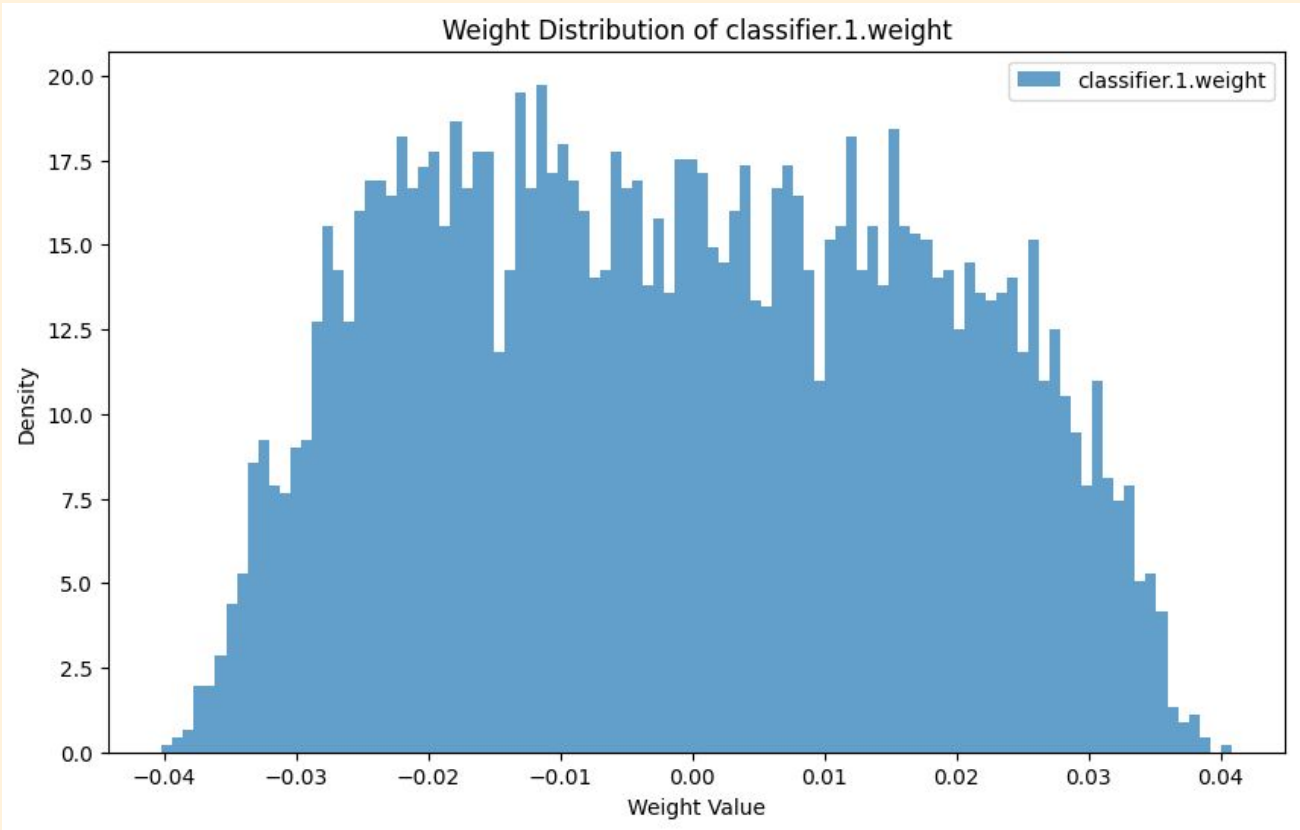


Last Layer

EfficientNet



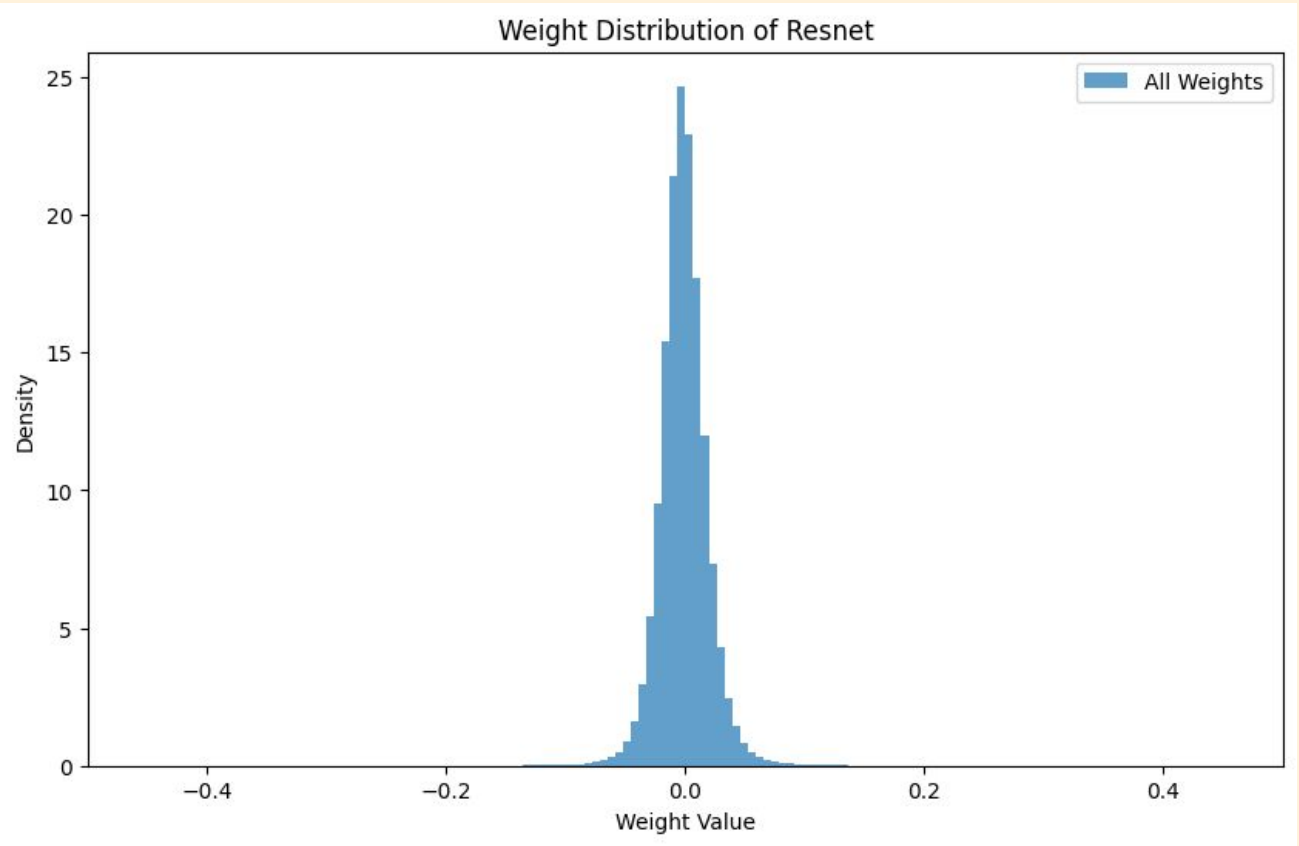
Total



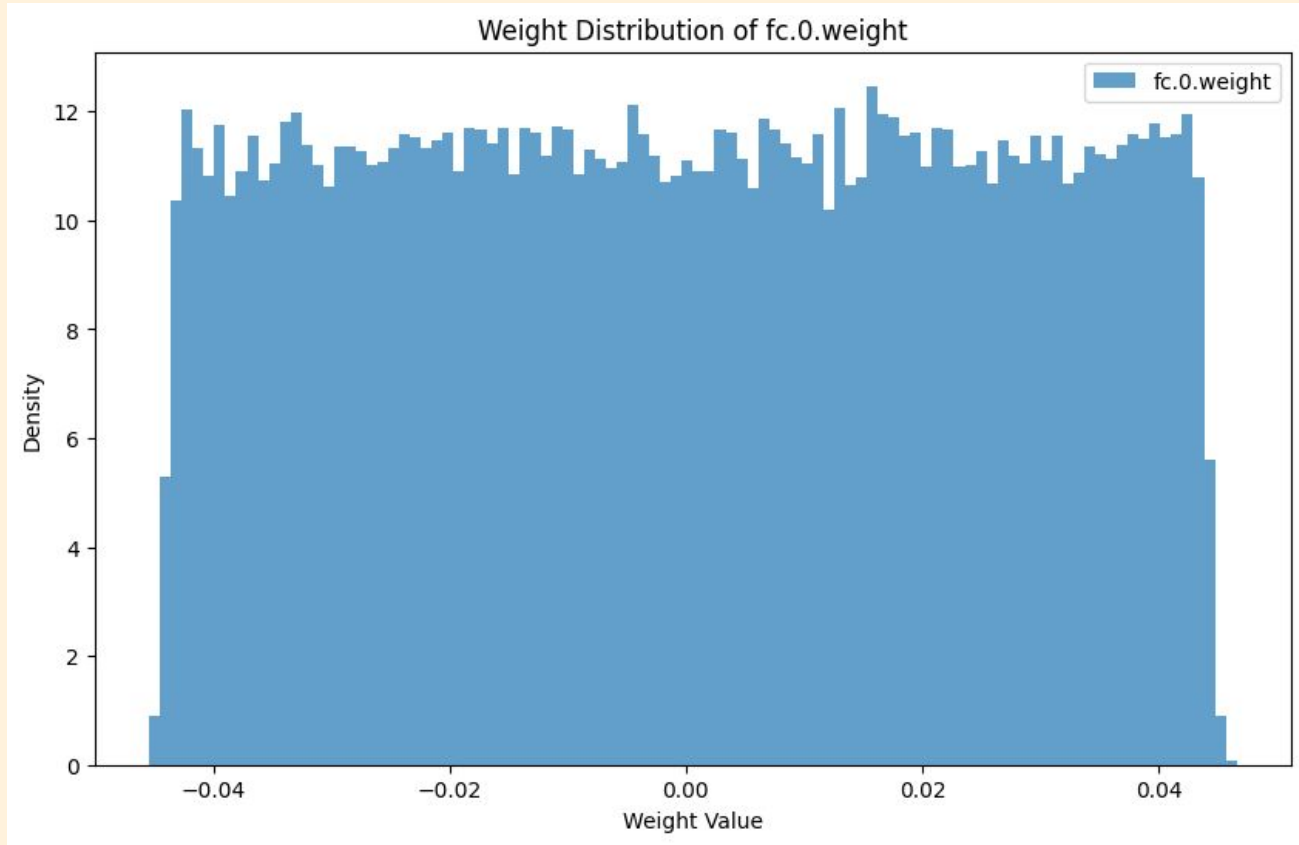
Last Layer

Roboflow 2

ResNet



Total



Last Layer

Image Reconstruction



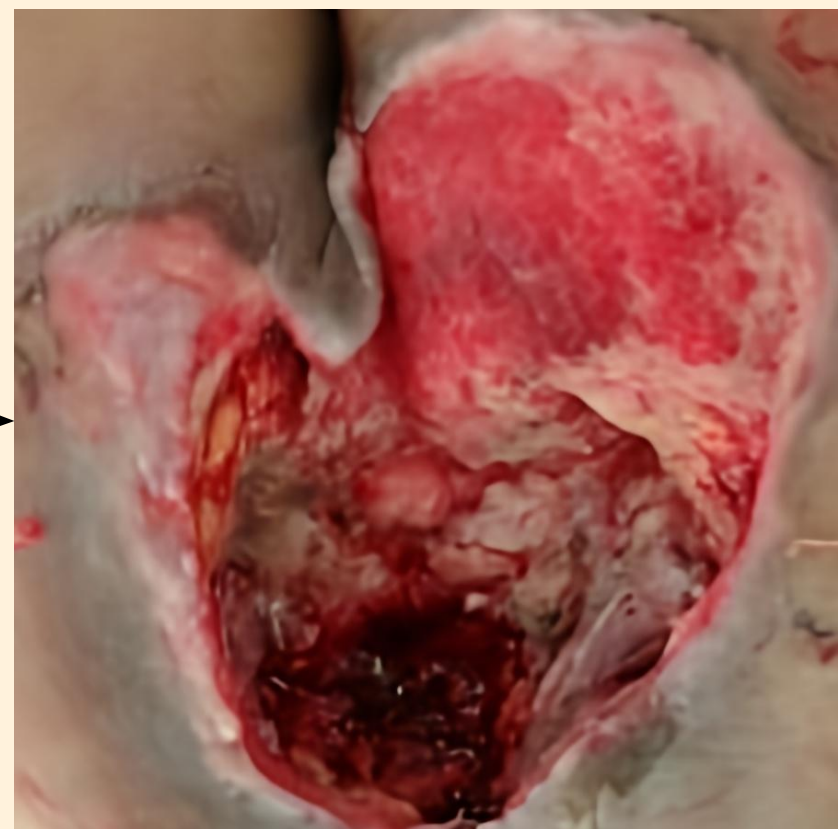
- The original photos are not clear enough.
- Attempt to restore the image details.
- Using Real-ESRGAN for processing:
 - `gt_size=192`
 - `total_iter=10000`

Image Reconstruction

- Train a model using super-resolution images and compare it with a model trained on the original images.



Raw Image



Reconstructed Image



Real-ESRGAN



Model Training

Perform a
comparison

Model Training

Image Reconstruction - EfficientNet

	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy	F1-Score
Roboflow1	0.21	0.21	0.46	0.35	0.3300	0.3088
Roboflow2	0.85	0.78	0.75	0.93	0.8241	0.8287

- ❑ Batch_Size:32,64,128

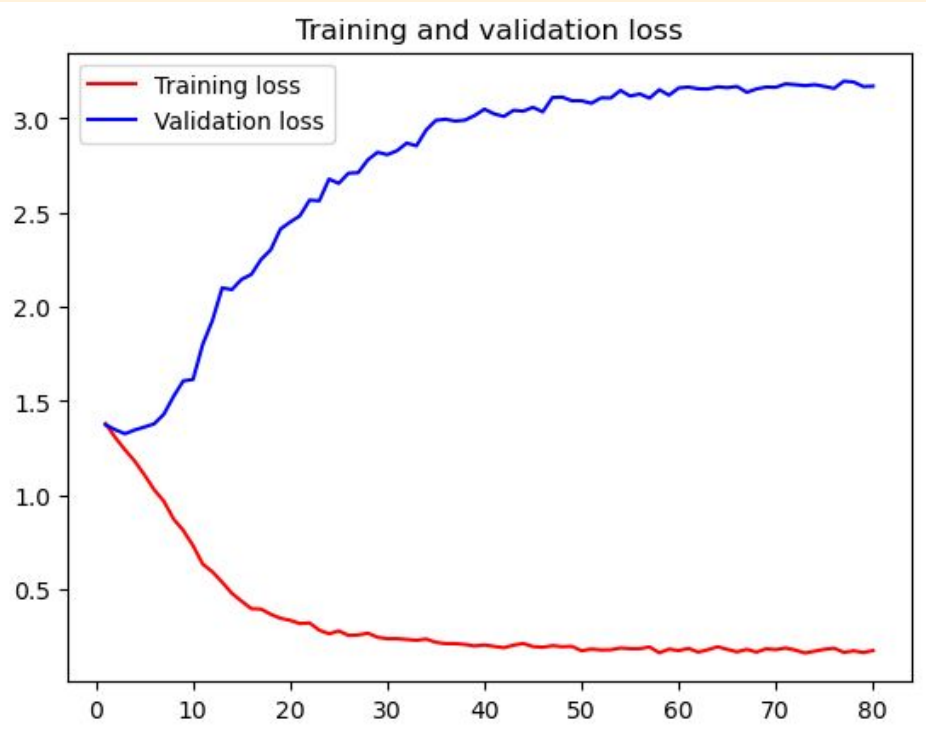
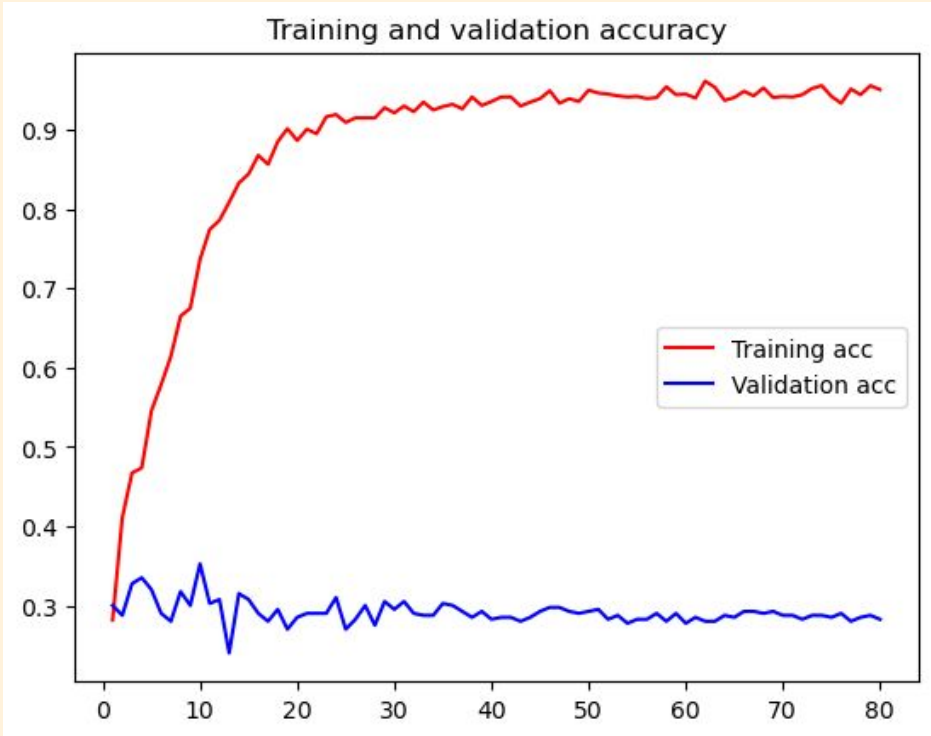
❑ Epoch:80,100
- ❑ Learning_rate:1e-4~1e-5

❑ Weight_Decay:None,2e-3
- ❑ Optimizer:Adam,SGD

❑ Scheduler:Linear,CosineAnnealingLR

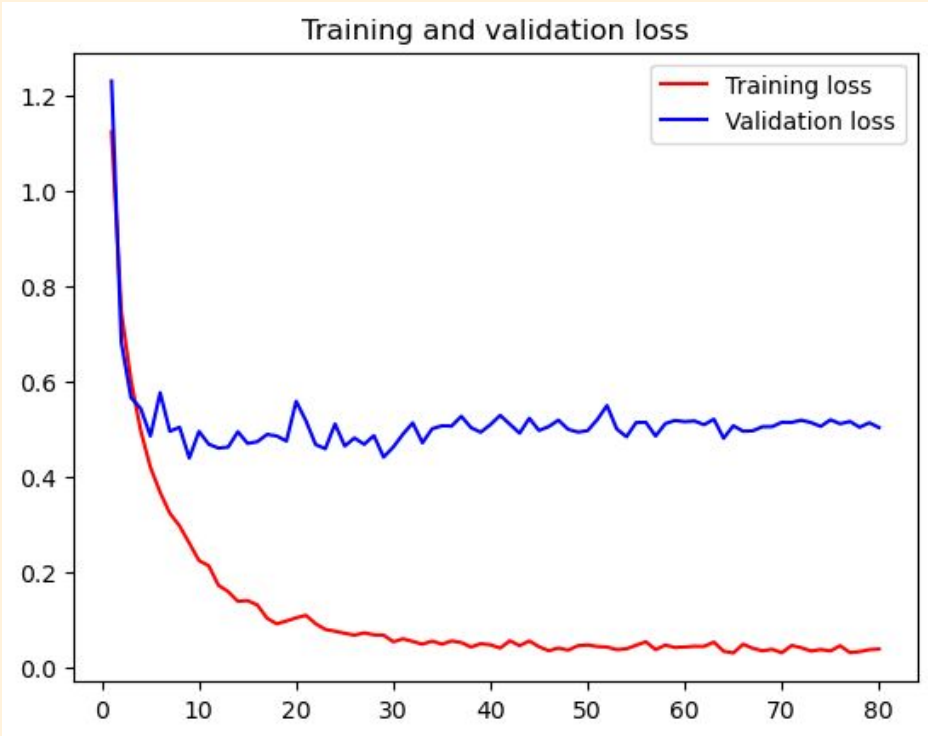
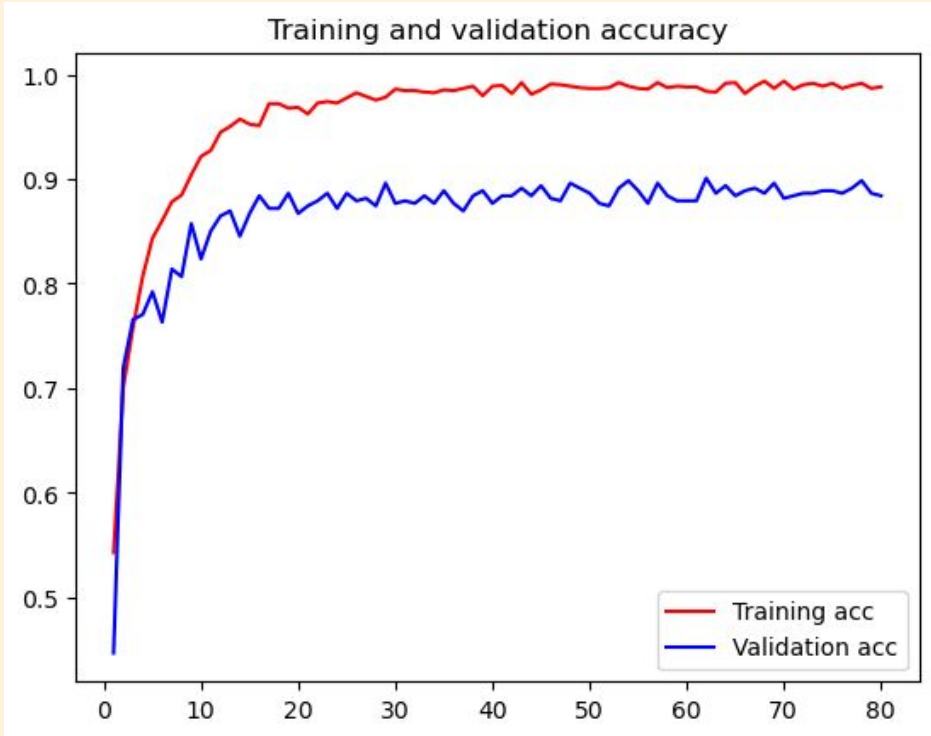
Roboflow1

Roboflow2



Train

Val



Train

Val

Image Reconstruction - ResNet

	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy	F1-Score
Roboflow1	0.19	0.34	0.38	0.34	0.3250	0.3132
Roboflow2	0.88	0.70	0.52	0.77	0.7269	0.7193

(+0.02)

(+0.03)

(+0.03)

(+0.02)

(+0.01)

❑ Batch_Size:32,64,128

❑ Learning_rate:1e-4~1e-5

❑ Optimizer:Adam,SGD

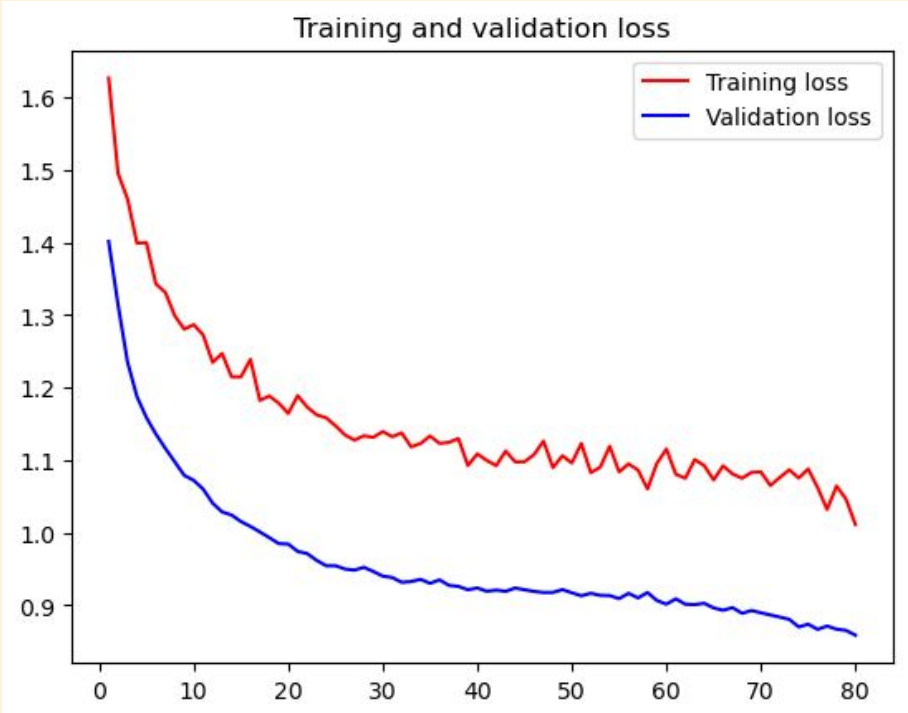
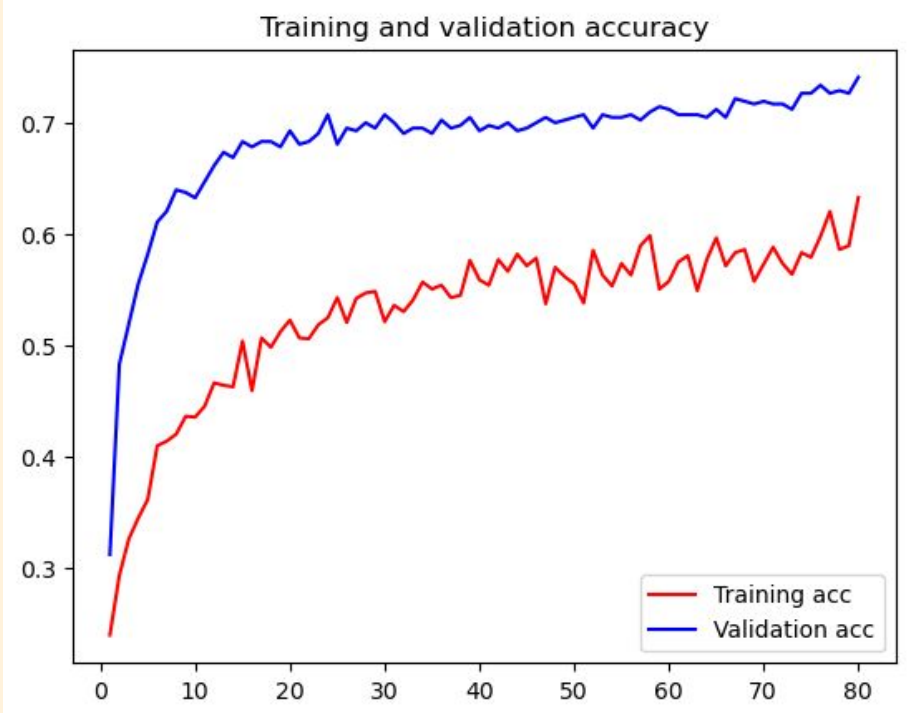
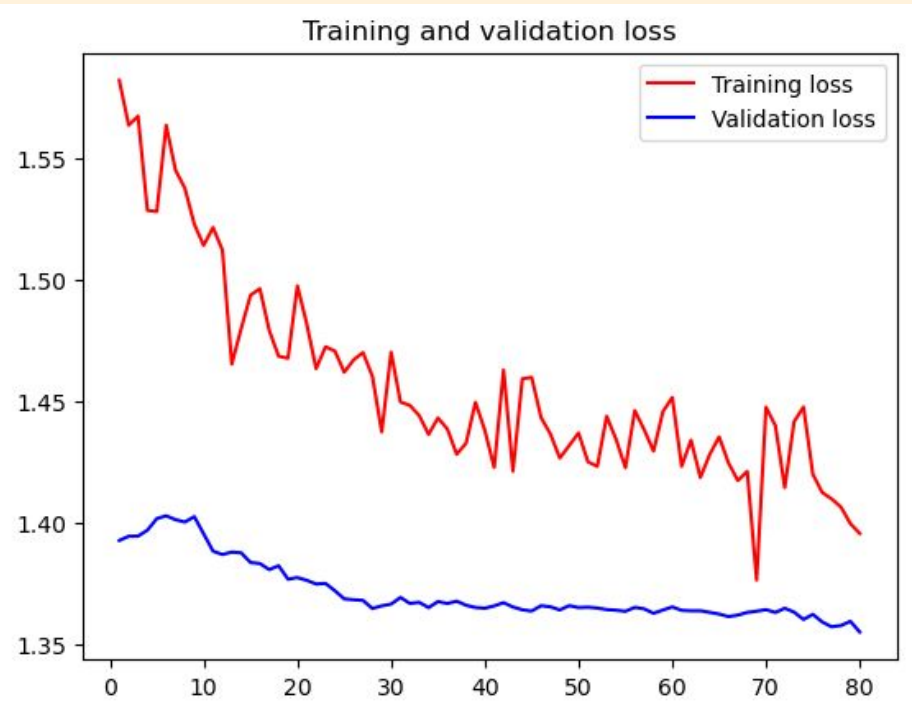
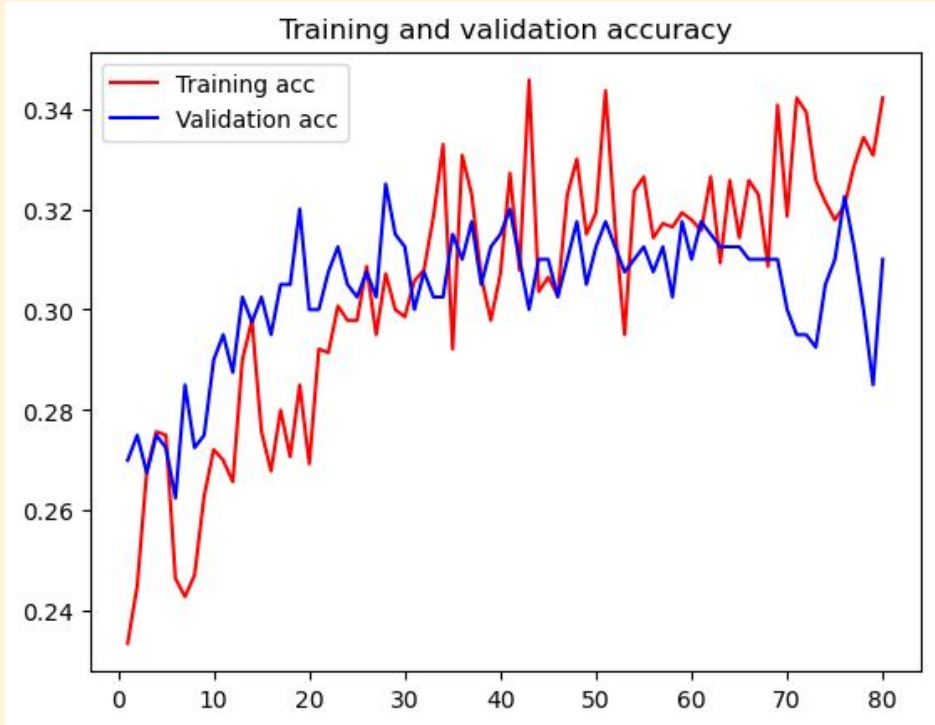
❑ Epoch:80,100

❑ Weight_Decay:None,2e-3

❑ Scheduler:Linear,CosineAnnealingLR

Roboflow1

Roboflow2



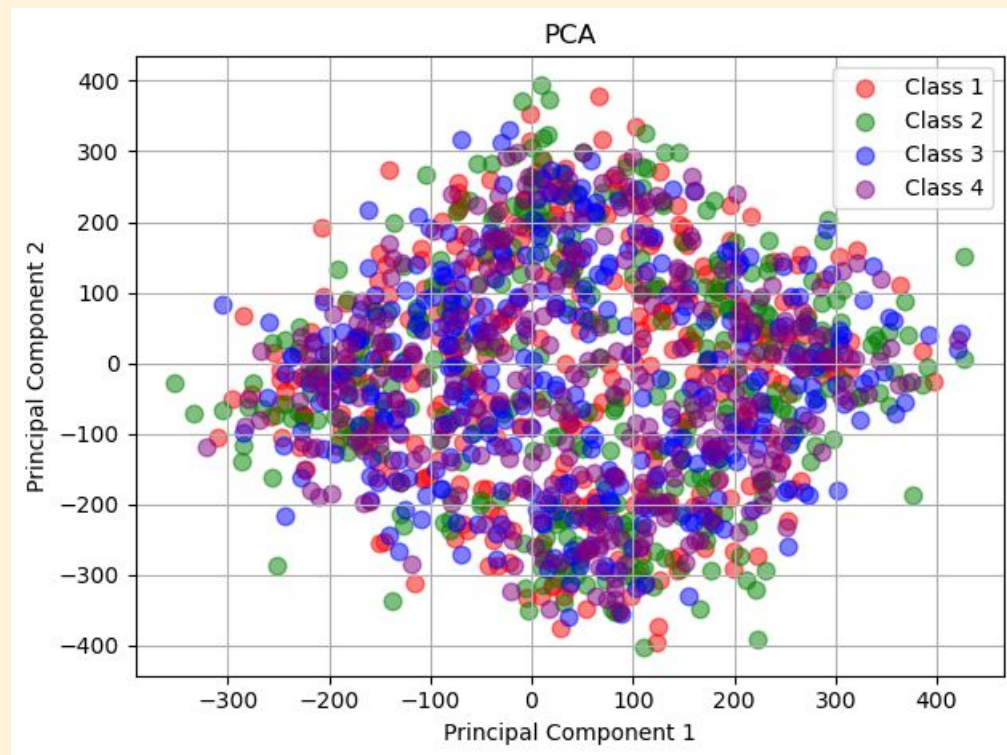
Train

Val

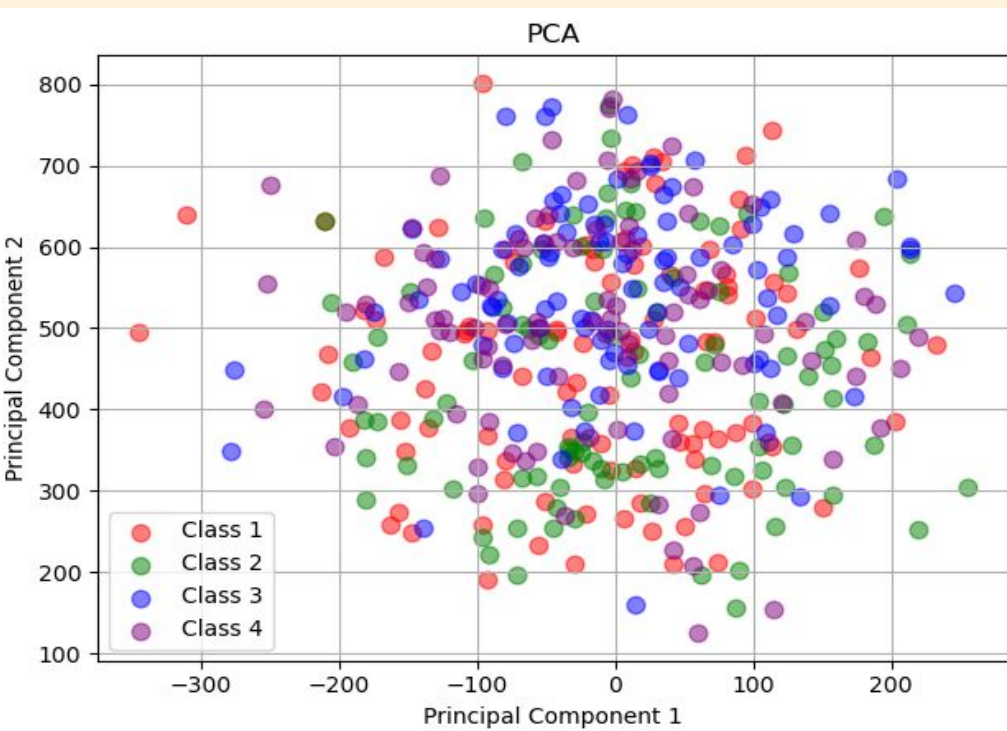
Train

Val

Roboflow 1

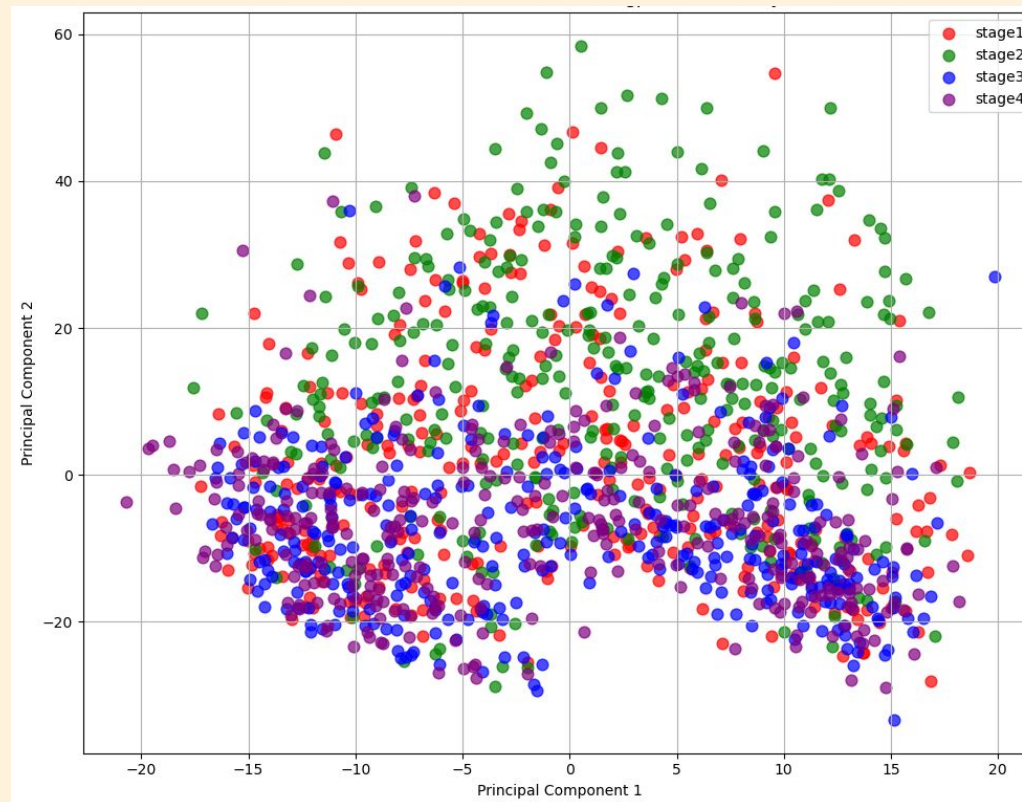


Train

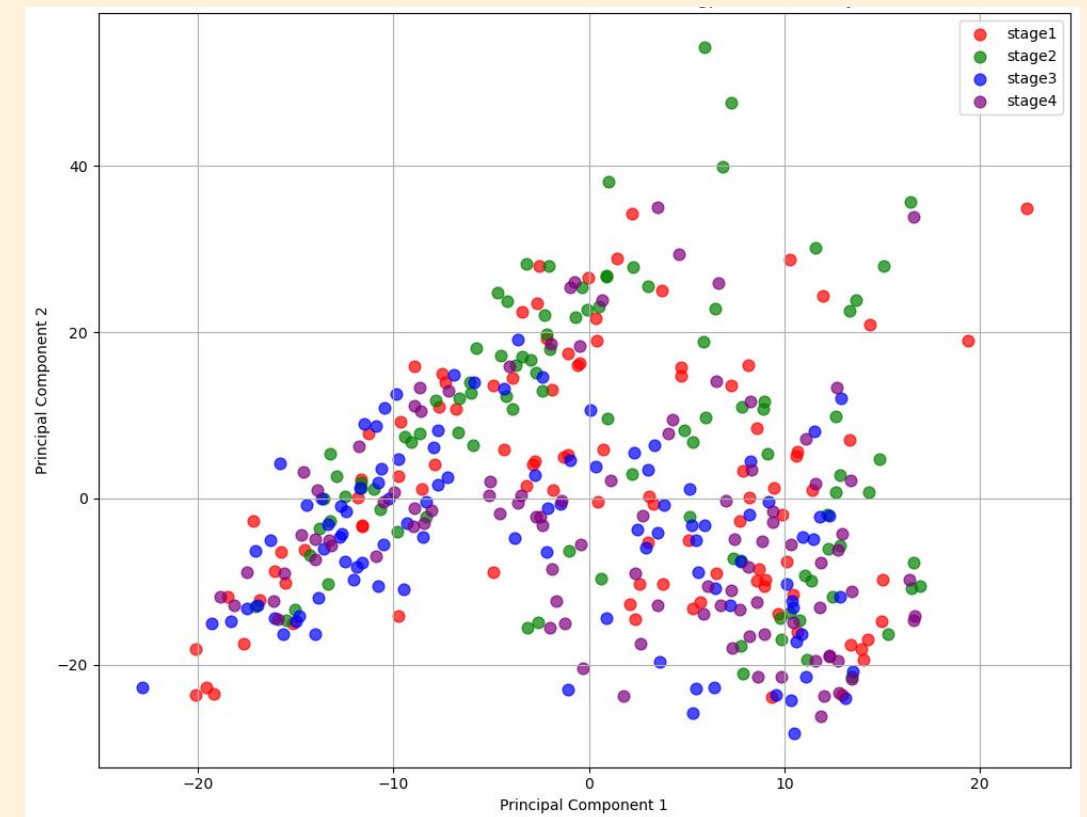


Val

EfficientNet

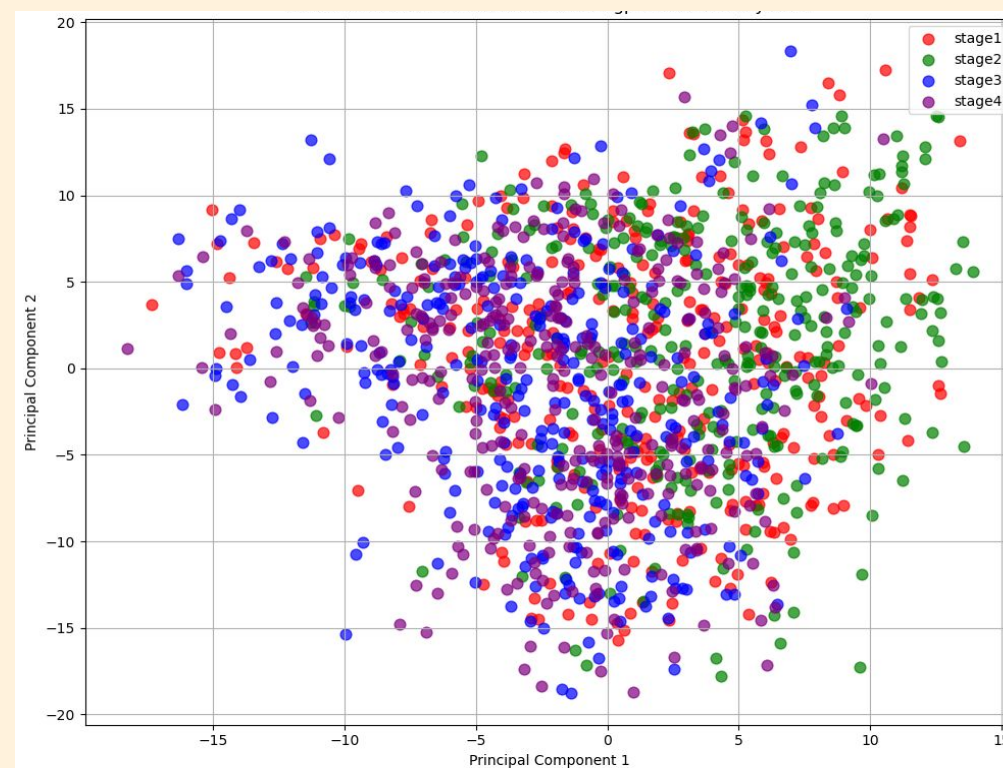


Train

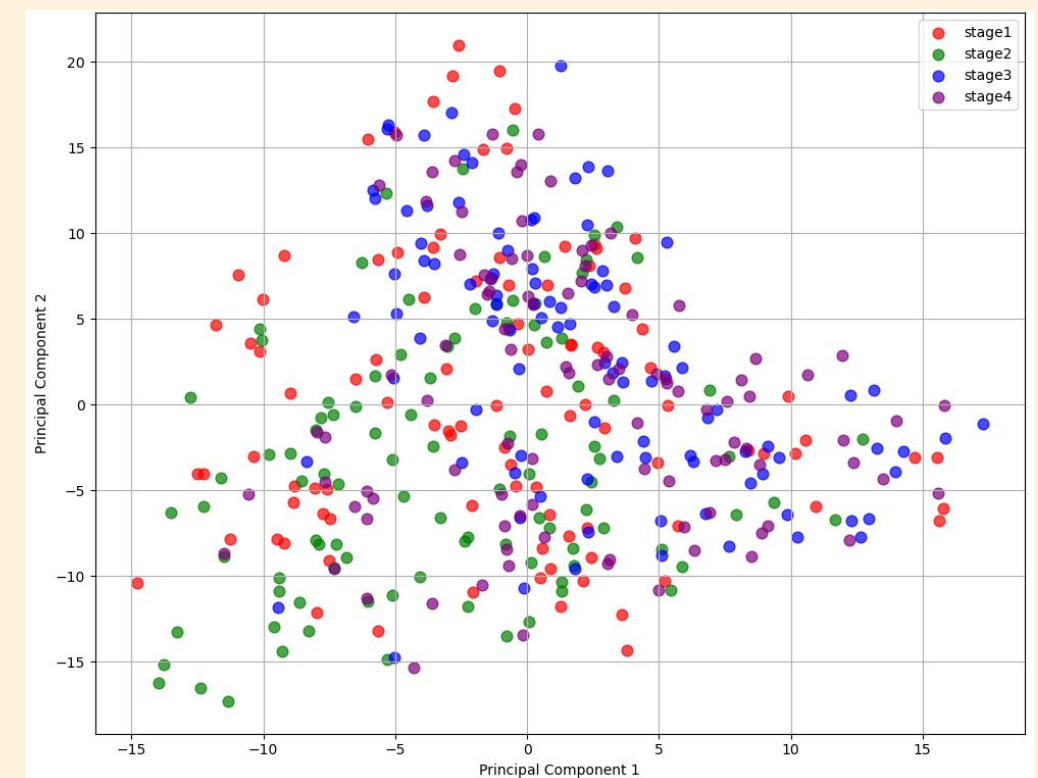


Val

ResNet

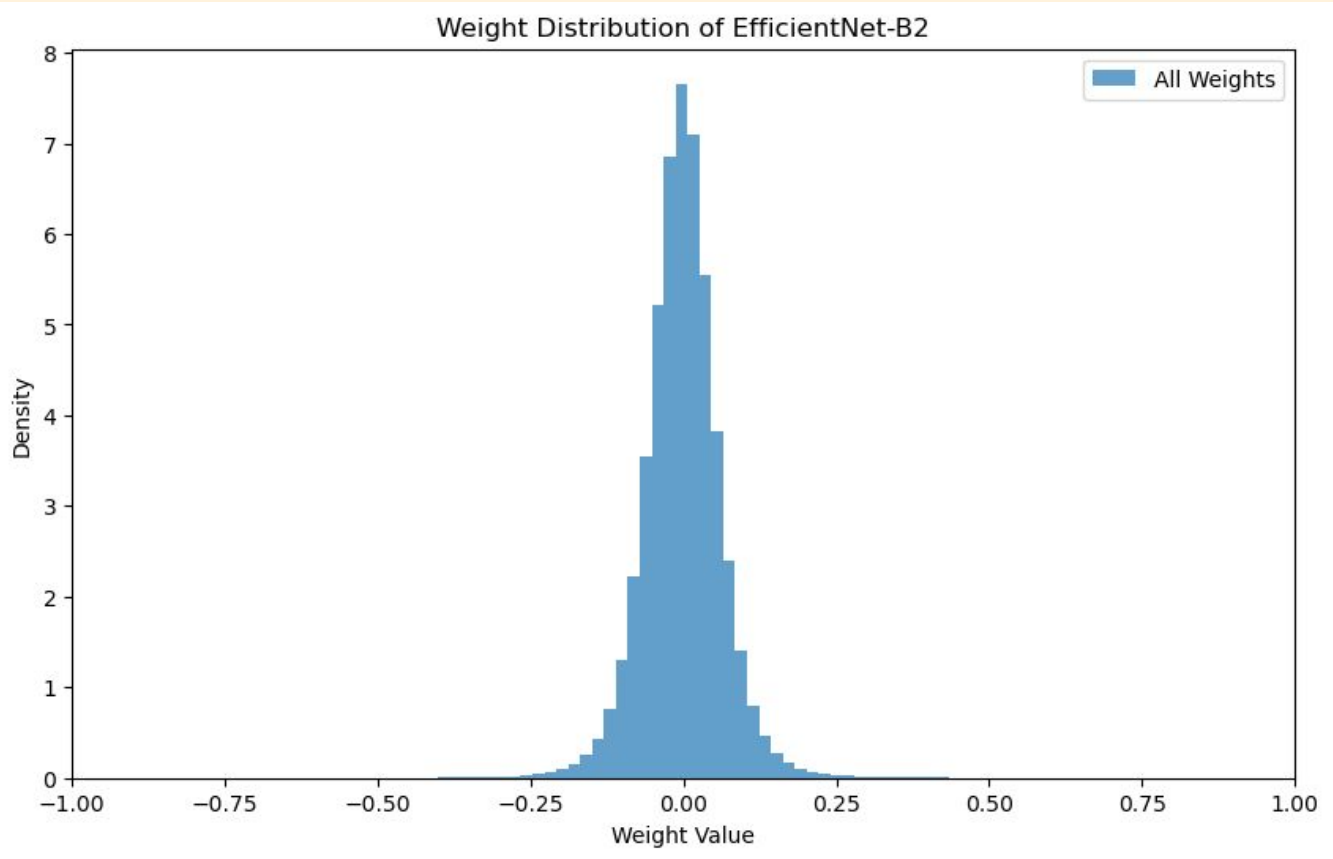


Train

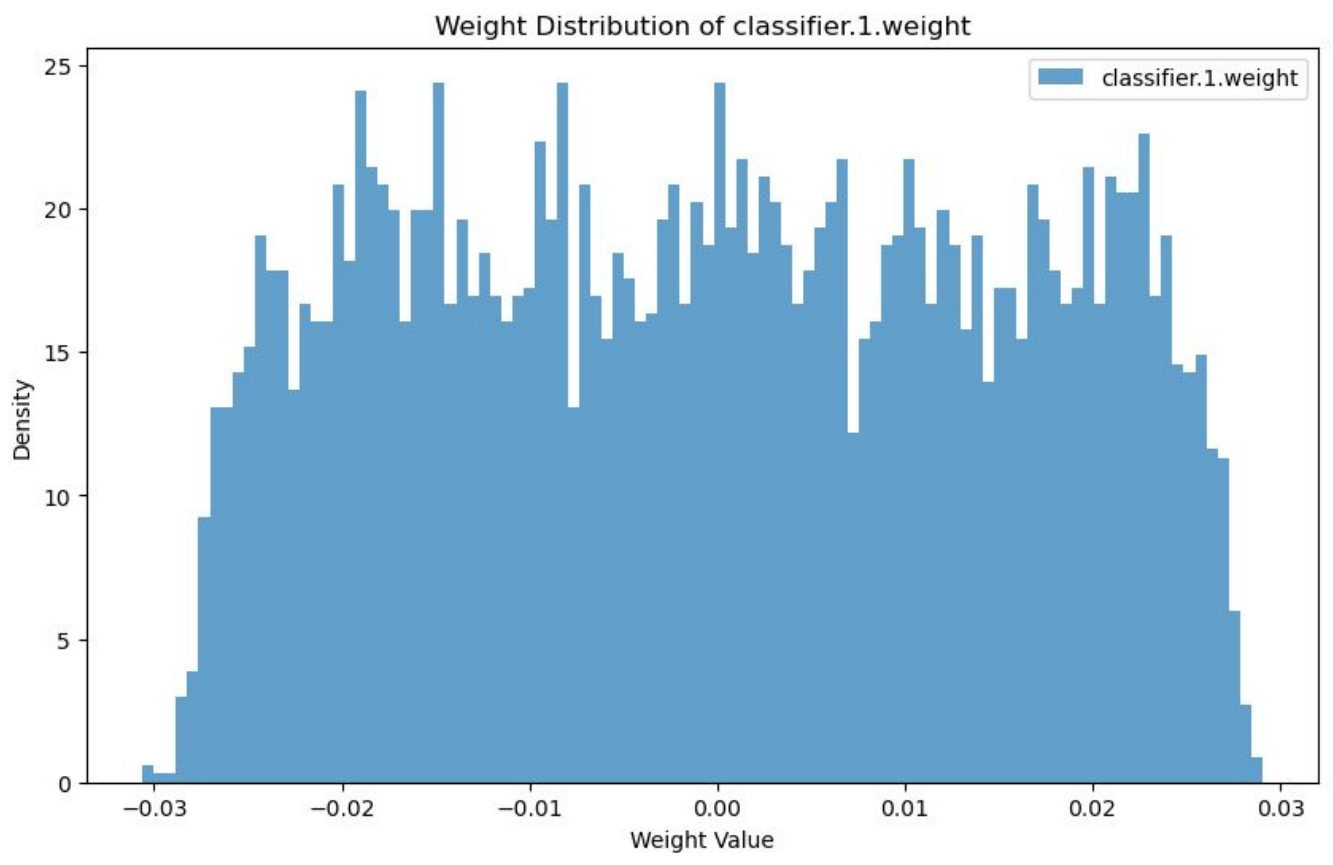


Val

EfficientNet



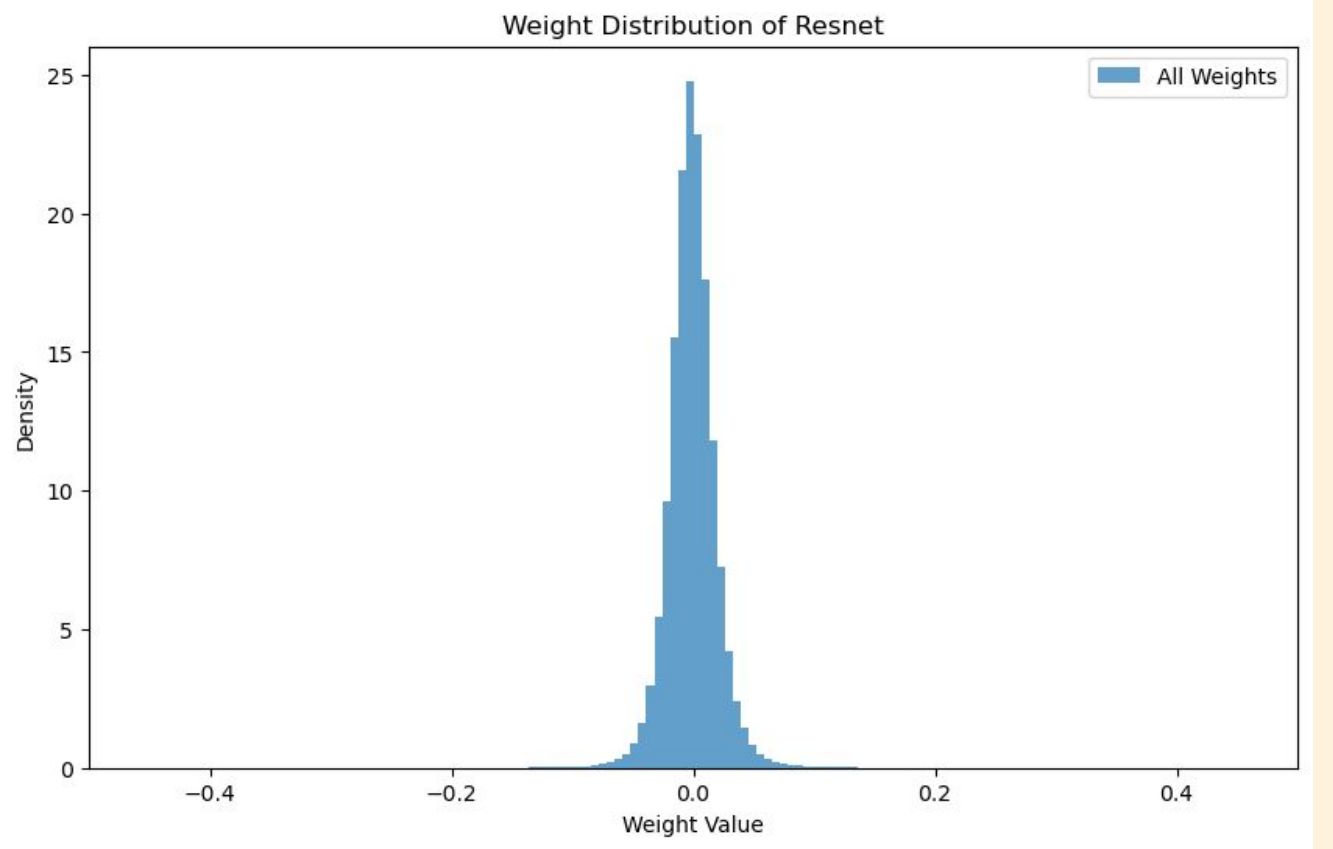
Total



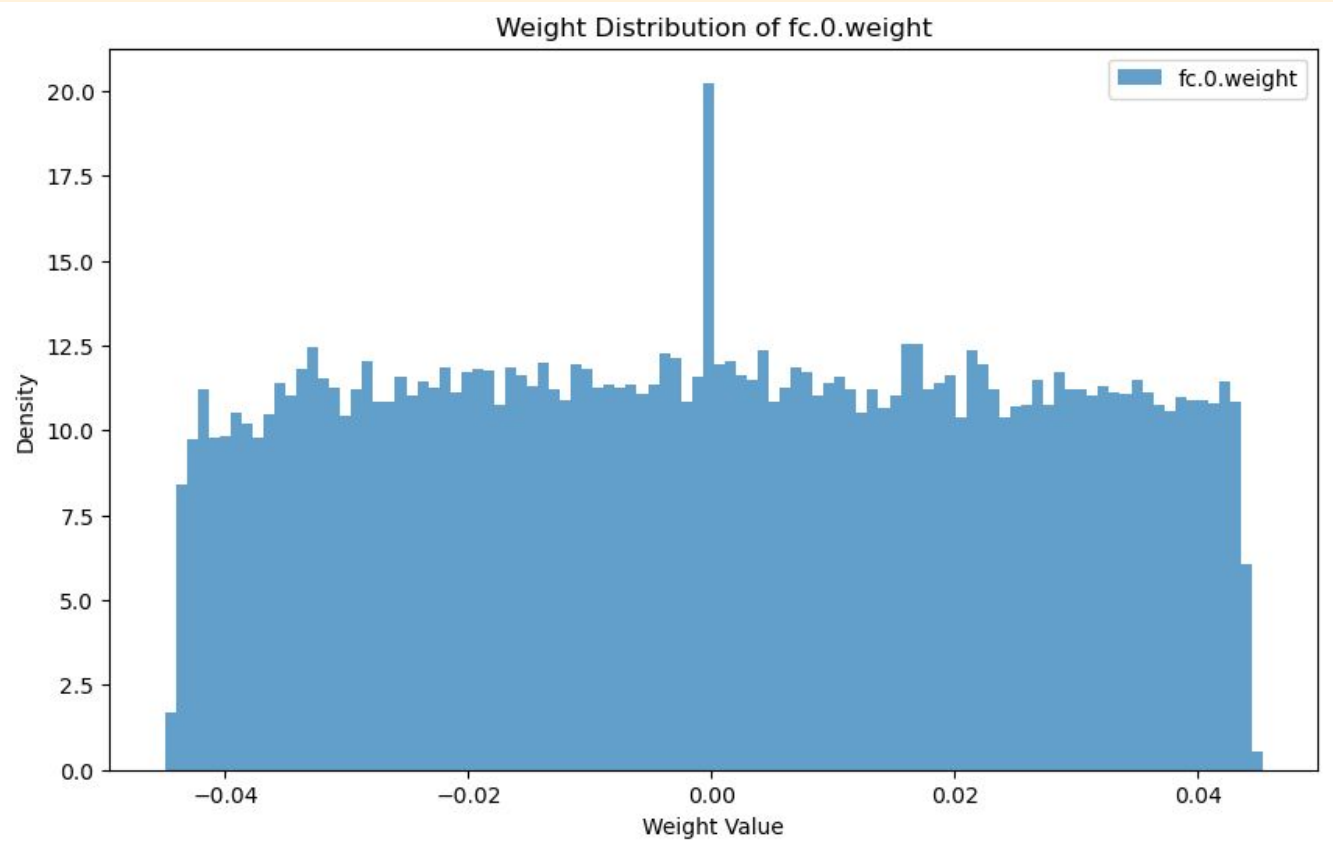
Last Layder

Roboflow 1

ResNet

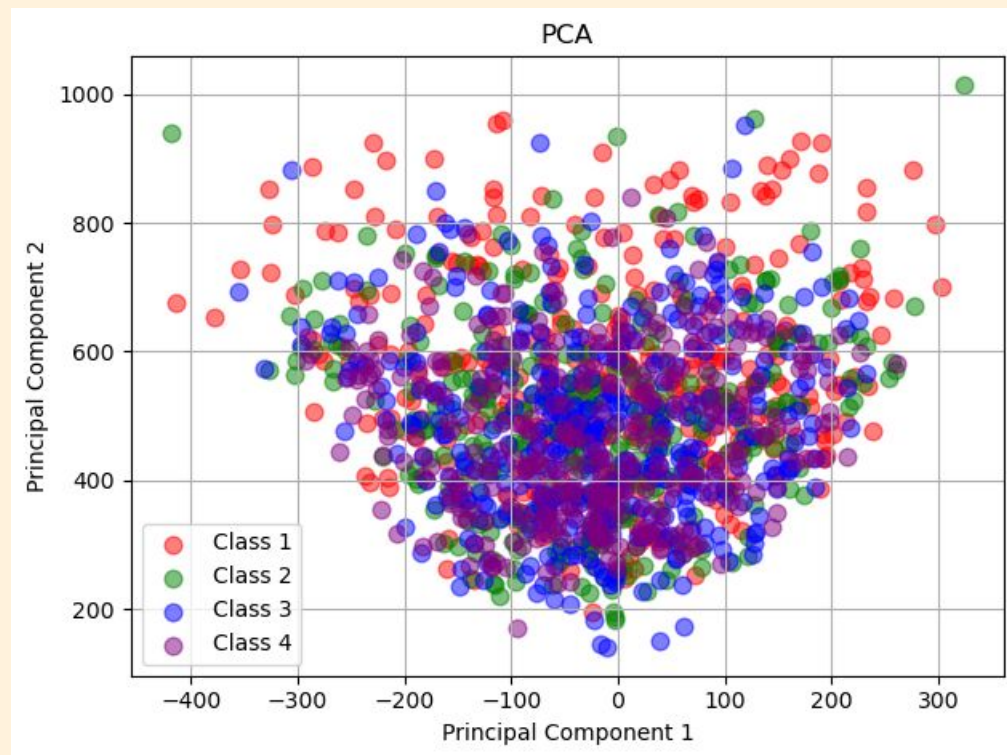


Total

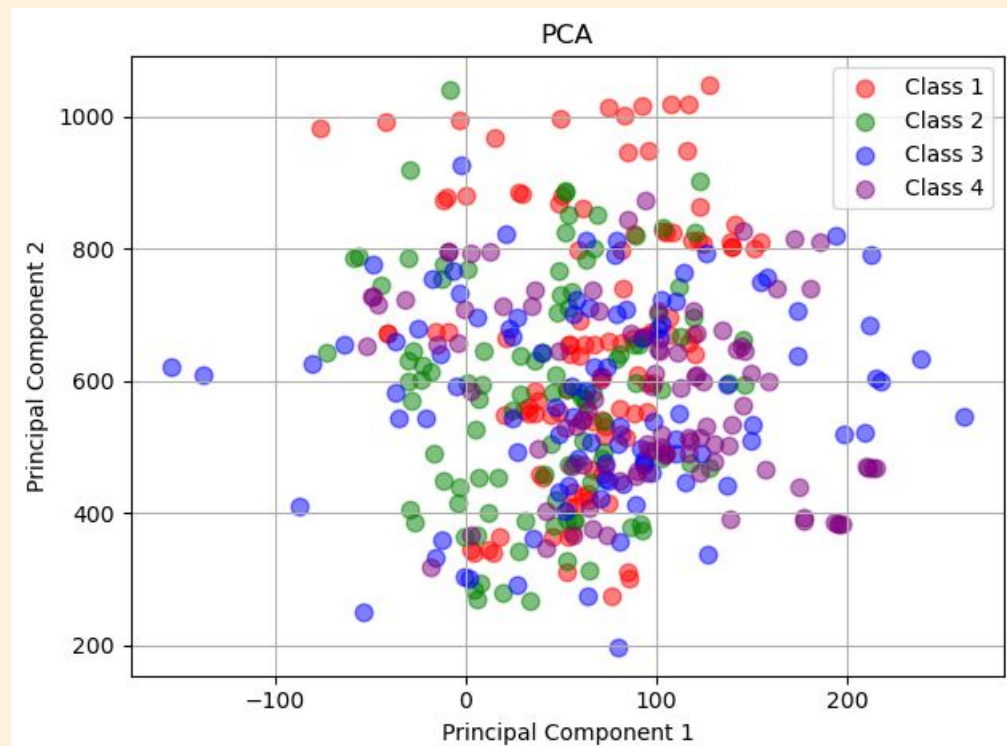


Last Layder

Roboflow 2

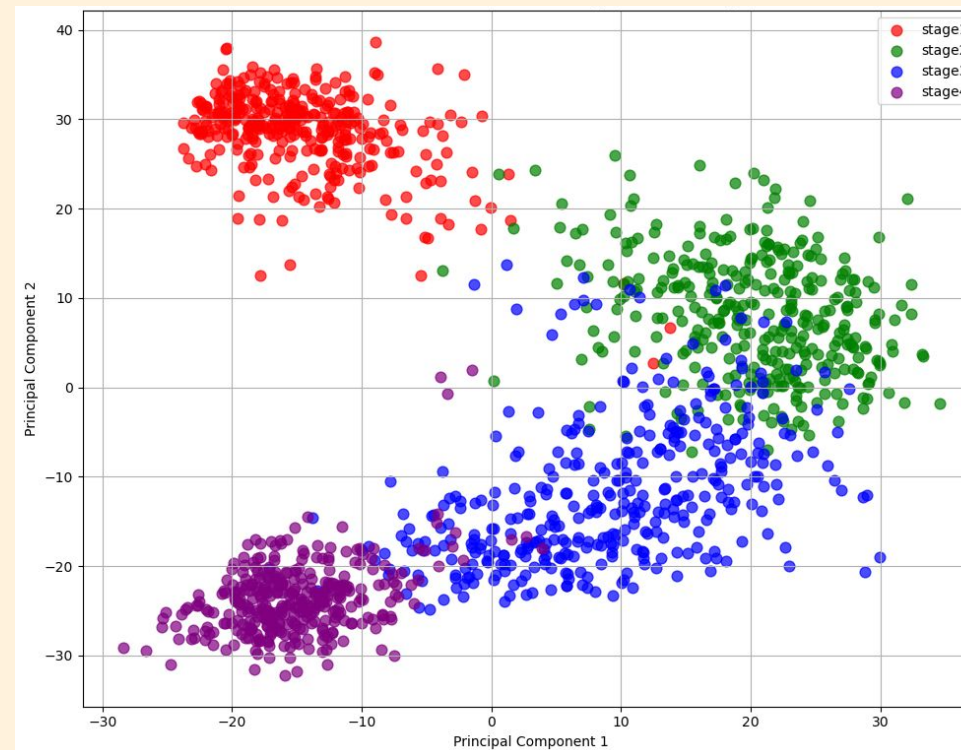


Train

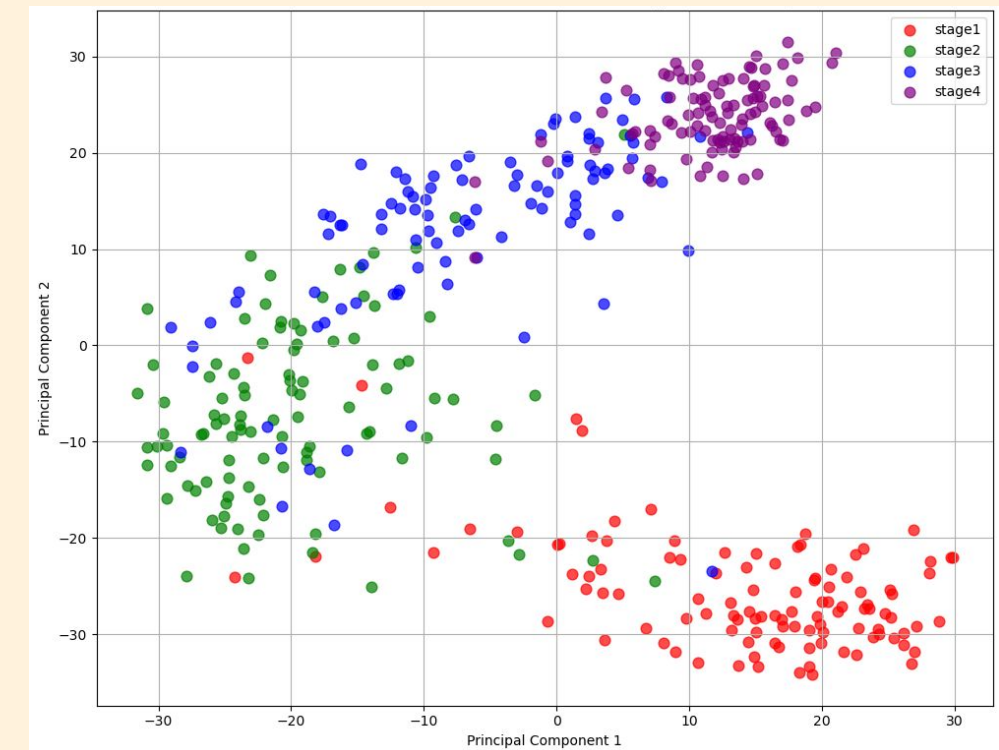


Val

EfficientNet

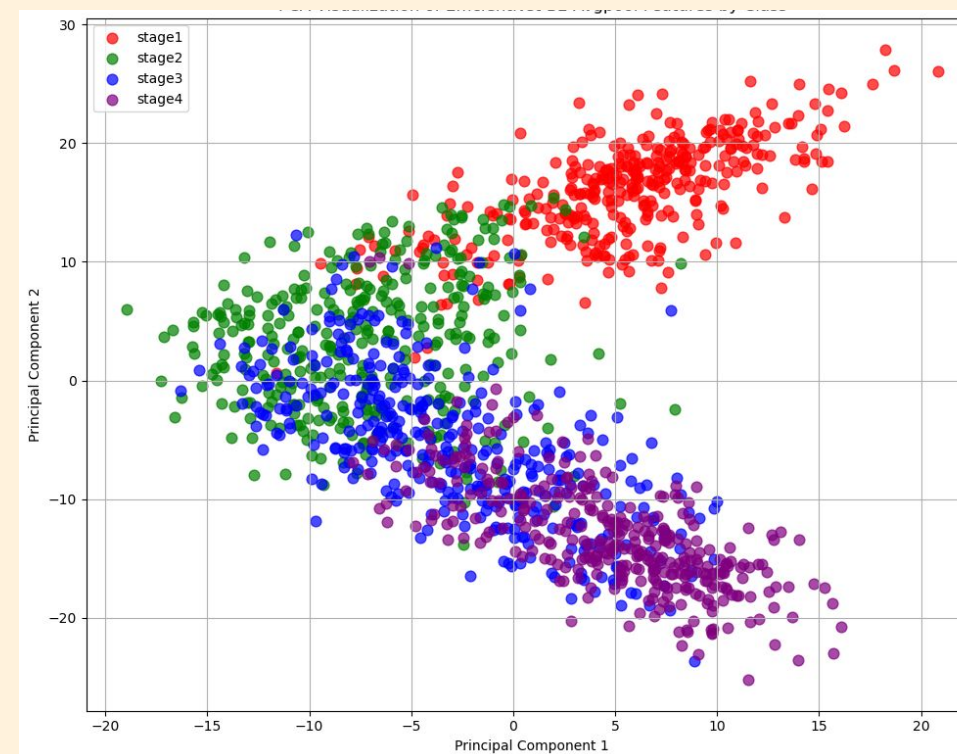


Train

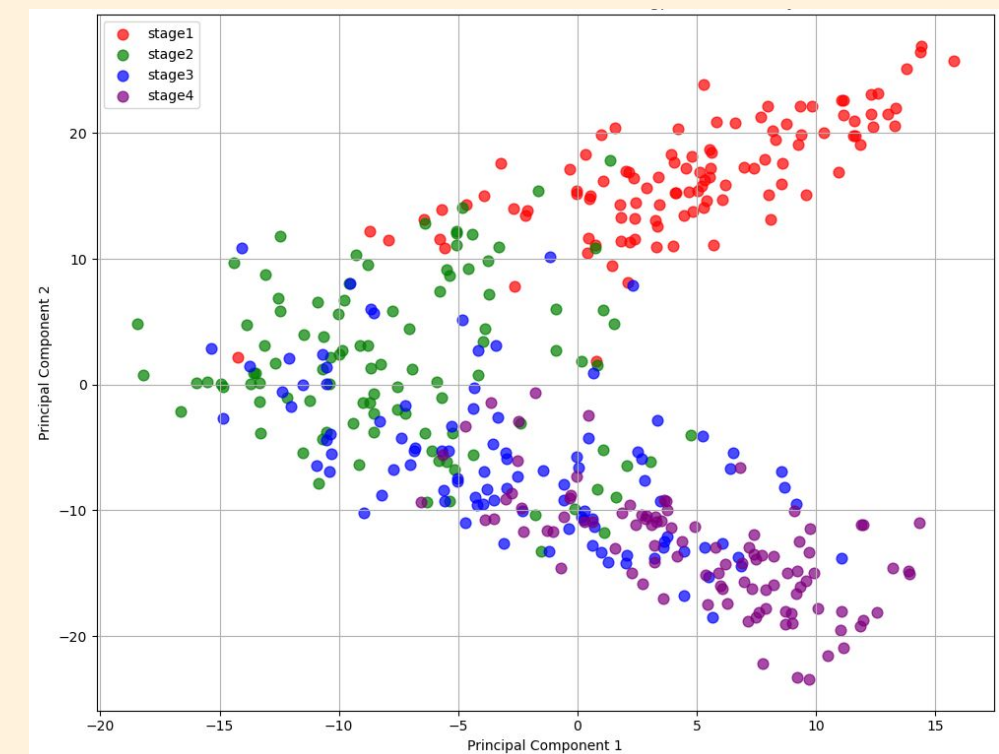


Val

ResNet

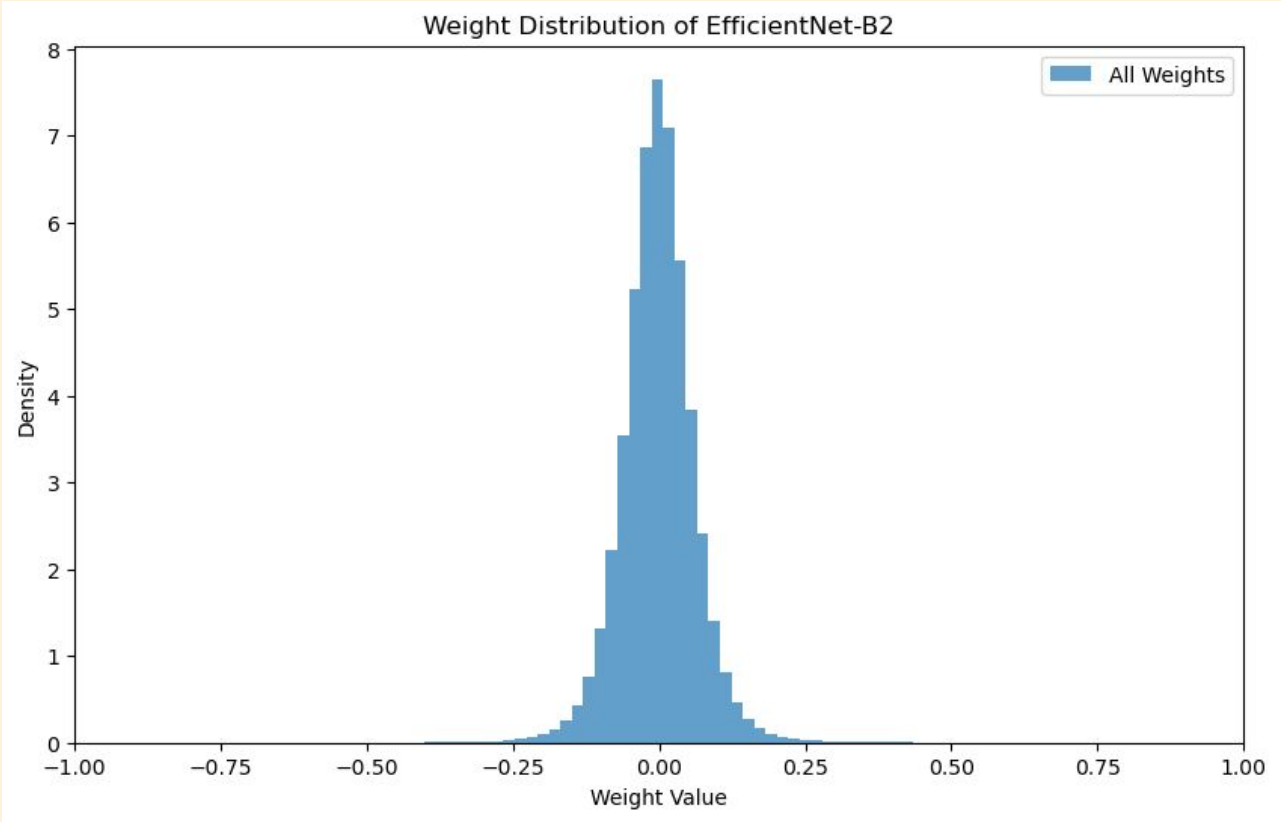


Train

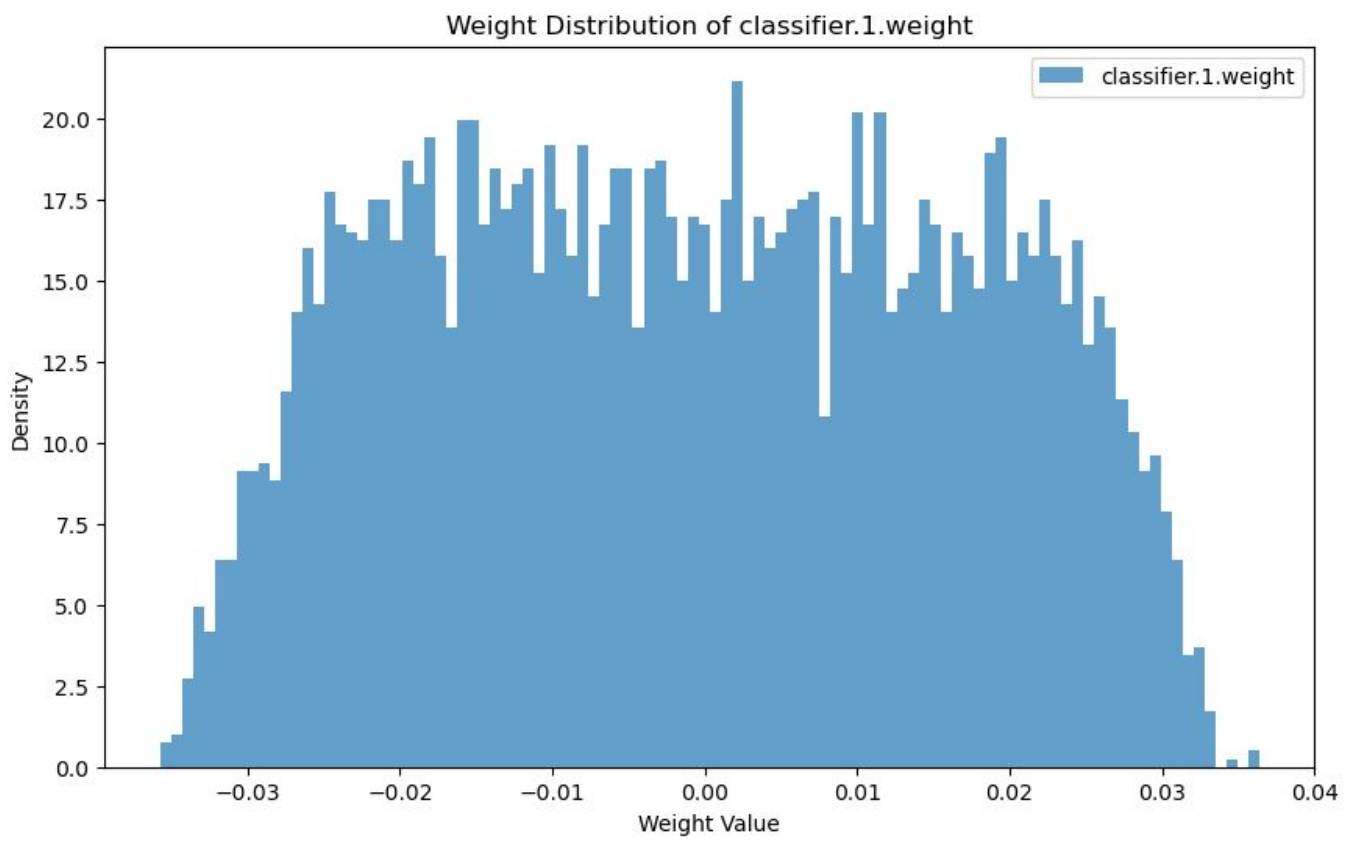


Val

EfficientNet

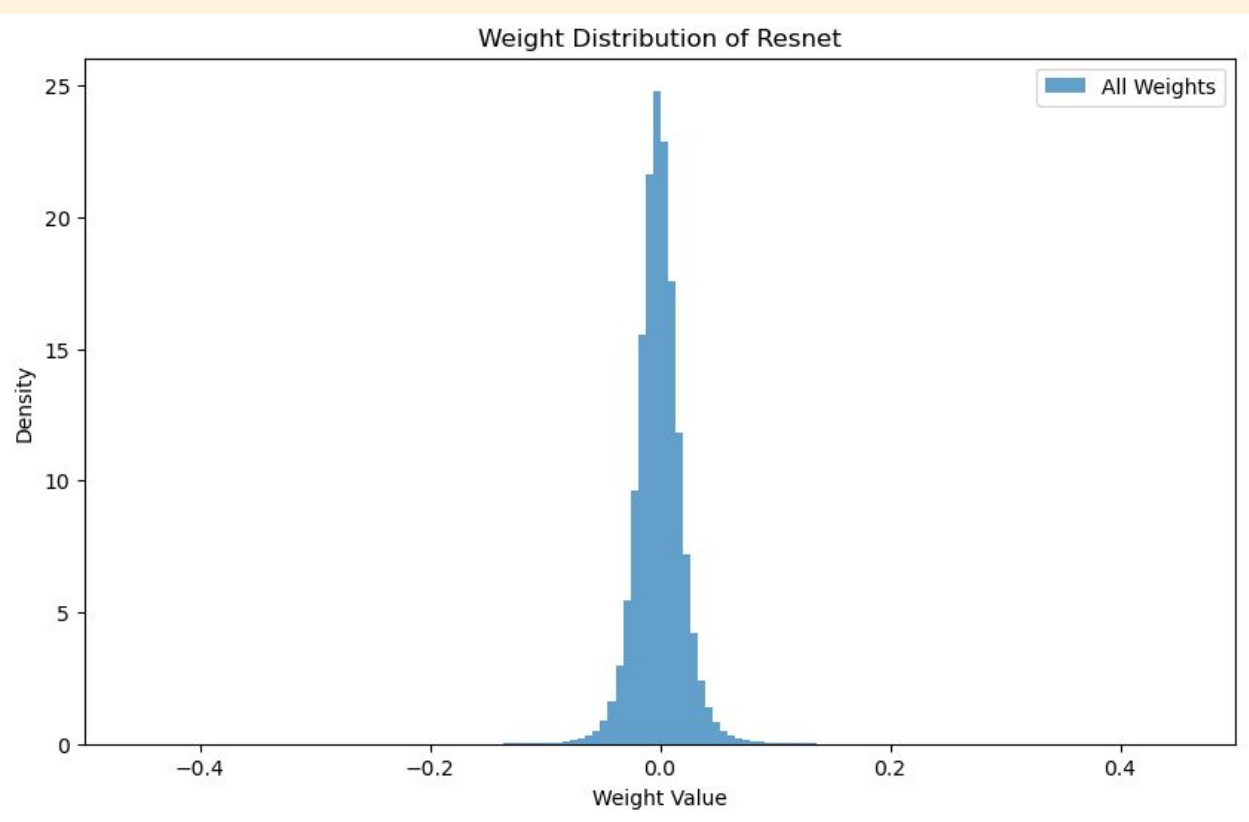


Total

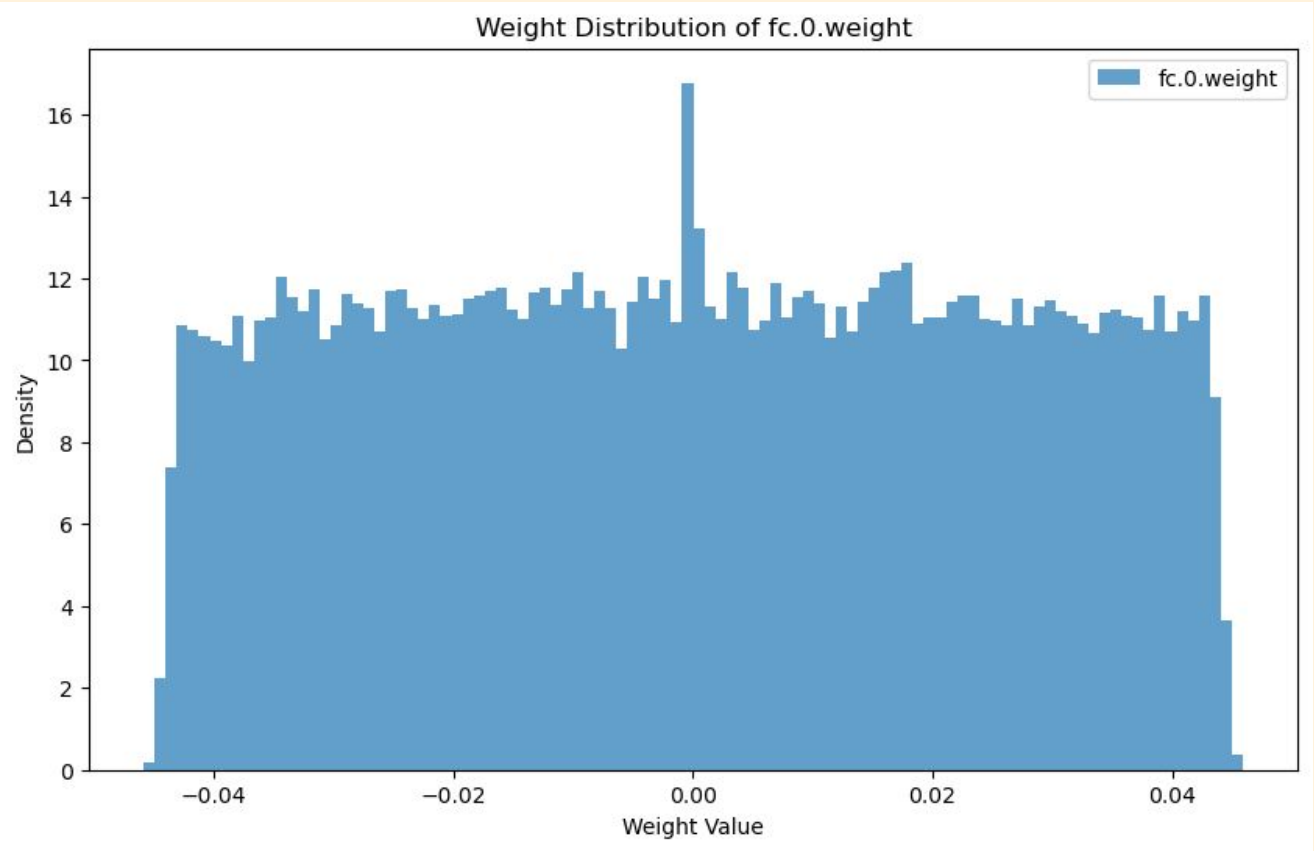


Last Layder

Roboflow 2



Total



Last Layder

Ordinal Classification



- The deterioration of pressure ulcer wounds increases in severity over time.
- Directly classifying them into discrete categories can easily lead to confusion at category boundaries
- Using an Ordinal model to learn the classification sequence can address this issue.

Ordinal Classification

EfficientNet F1-Score

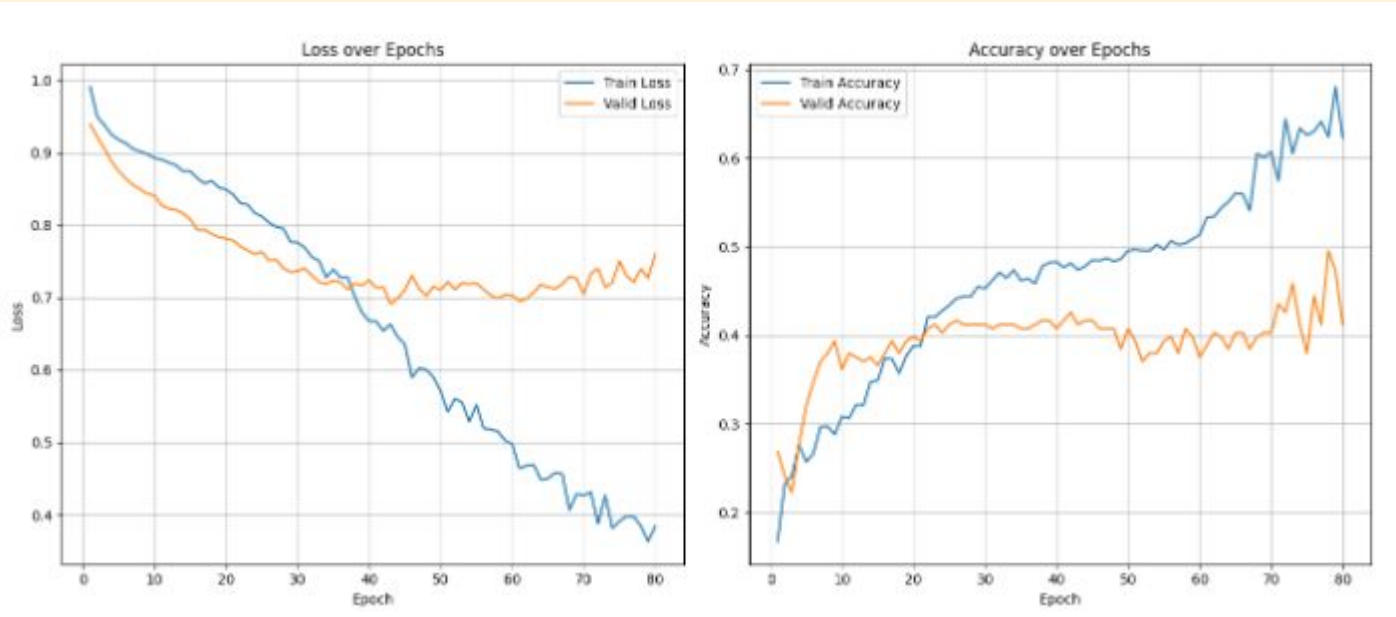
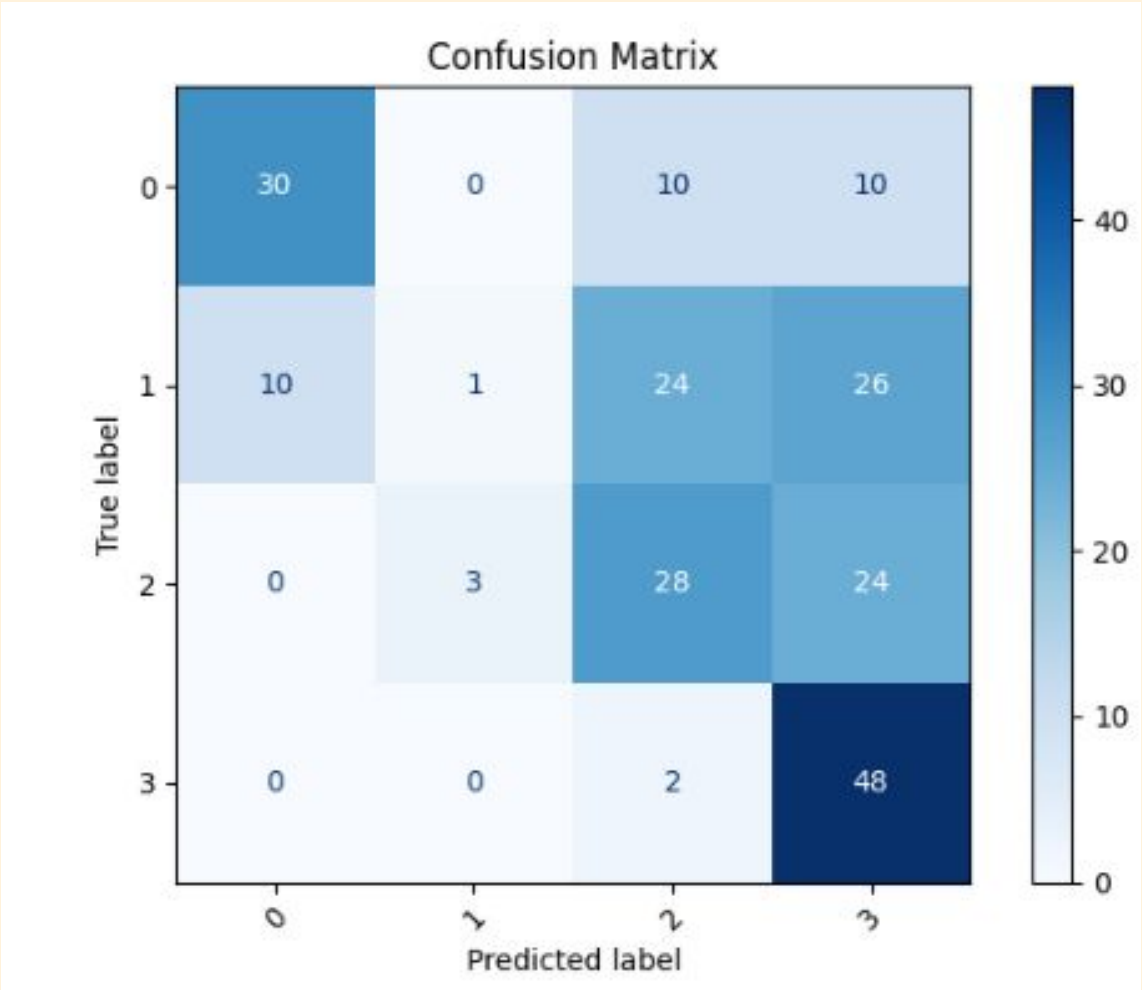
	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy
Roboflow2	0.67	0.03	0.47	0.61	0.50
Roboflow1	0.42	0.10	0.15	0.34	0.31

ResNet F1-Score

	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy
Roboflow2	0.57	0.00	0.00	0.57	0.48
Roboflow1	0.42	0.00	0.00	0.34	0.30

超參數調整項目：

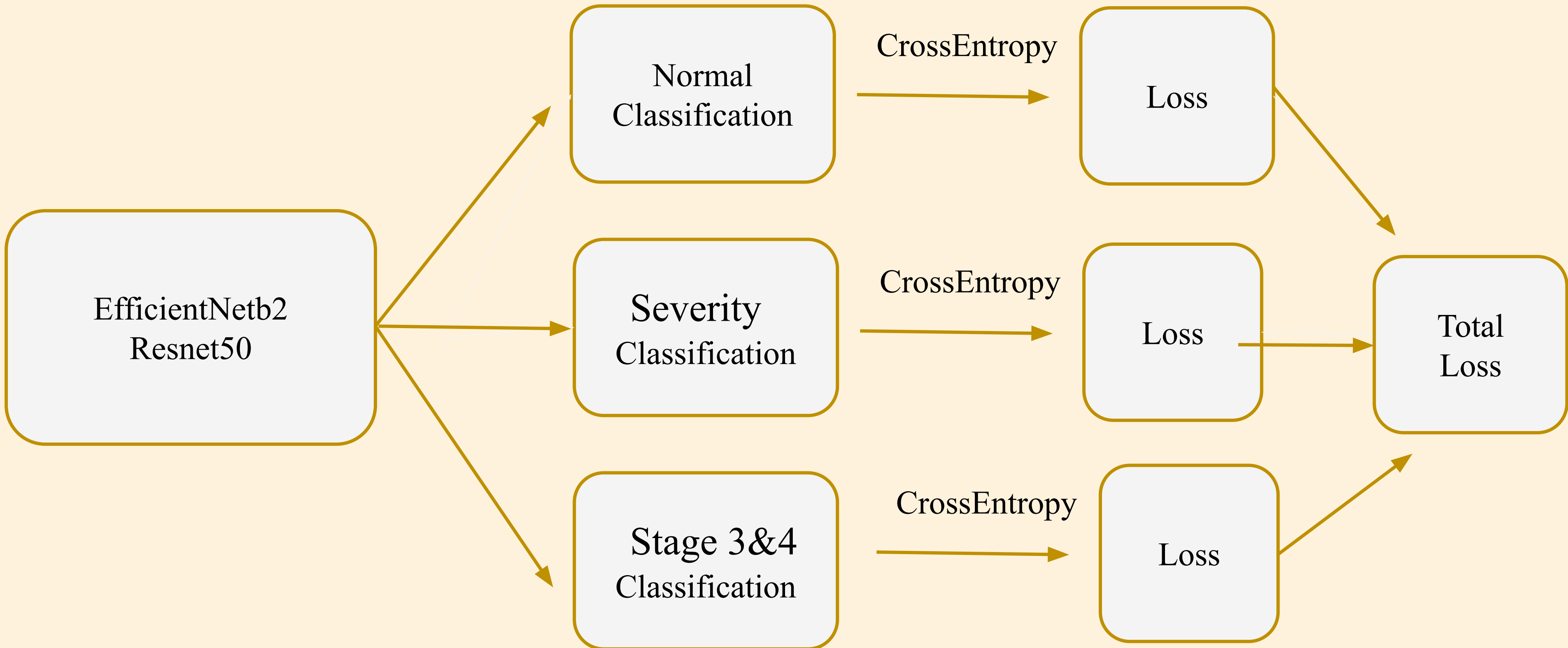
- α_2 (0.1~0.9) > 0.6
- η (0.1~0.9) > 0.6
- $\text{optimizer_weight_decay}$ (1e-3~1e-5) > 5e-4
- epochs (80)
- batch_size (32~256) > 64
- lr (1e-3~1e-5) > 1e-4



Multi-task model



- Encoder: ResNet 50 、 EfficientNet_B2
- Decoder: Normal Classification、 Severity Classification、 Stage 3&4 Classification
- Multi-task learning enables the model to capture diverse types of details by exposing it to various tasks simultaneously



Multi-task Model

EfficientNet F1-Score

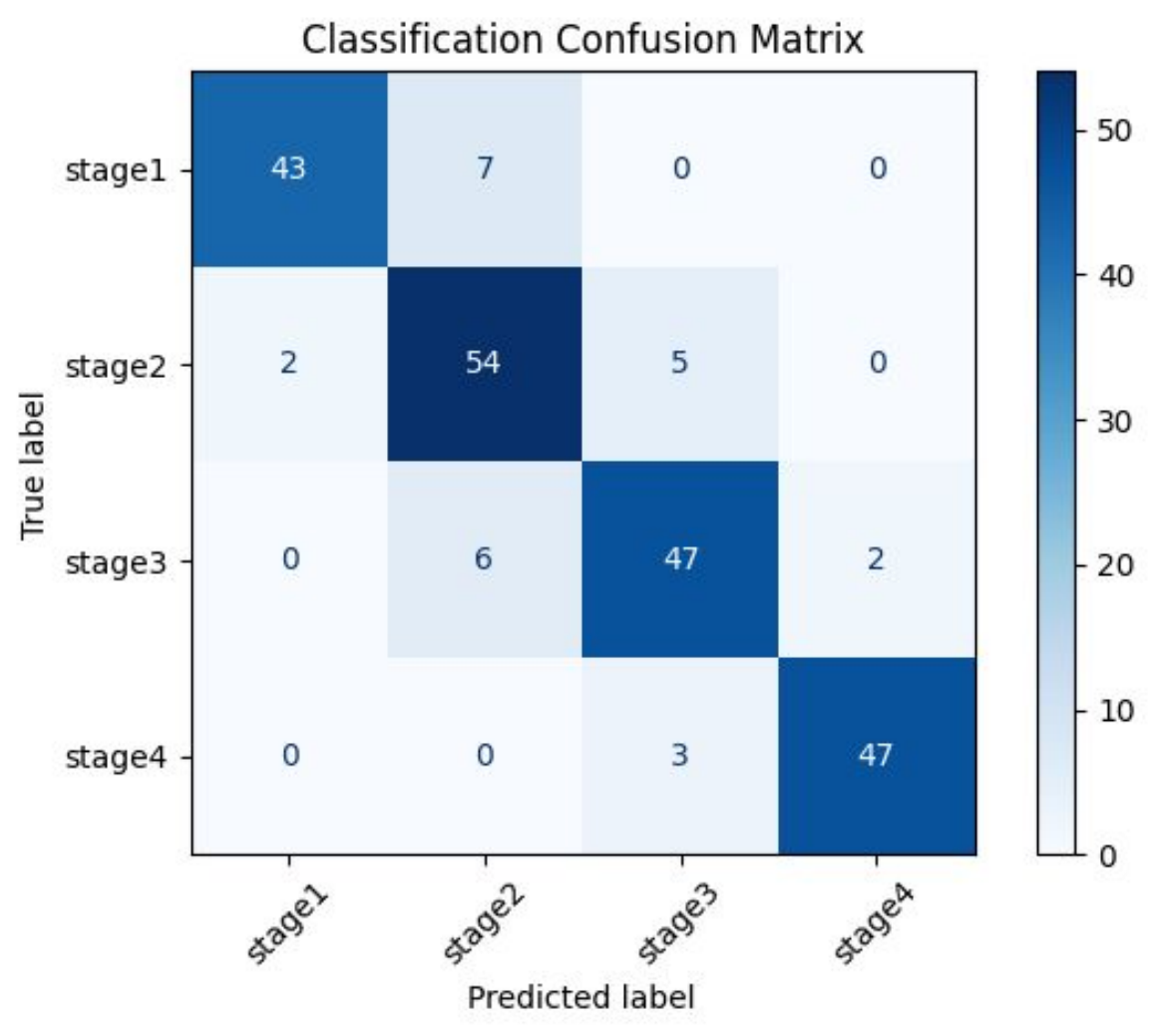
	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy
Roboflow2	0.91	0.84	0.85	0.95	0.88
Roboflow1	0.25	0.34	0.12	0.15	0.23

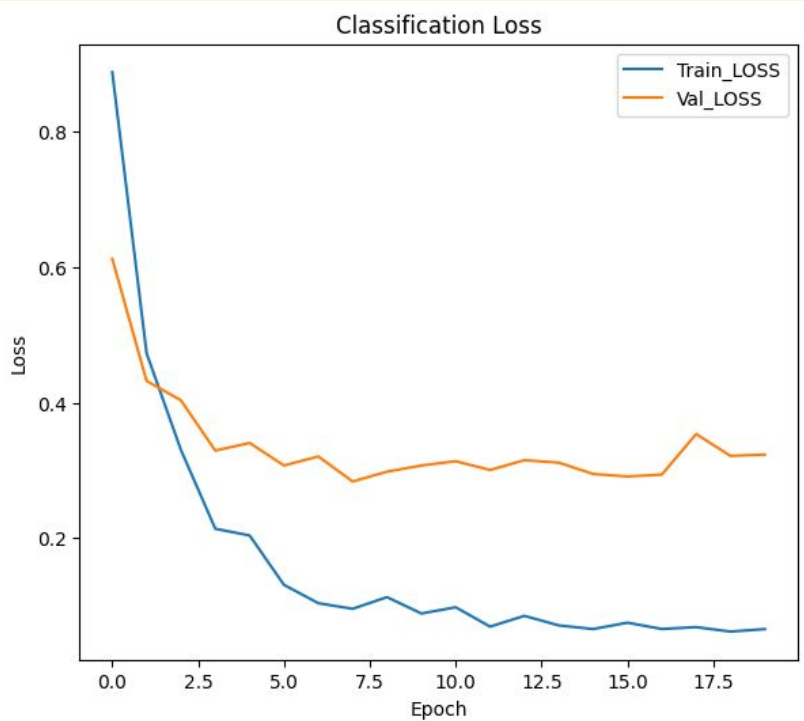
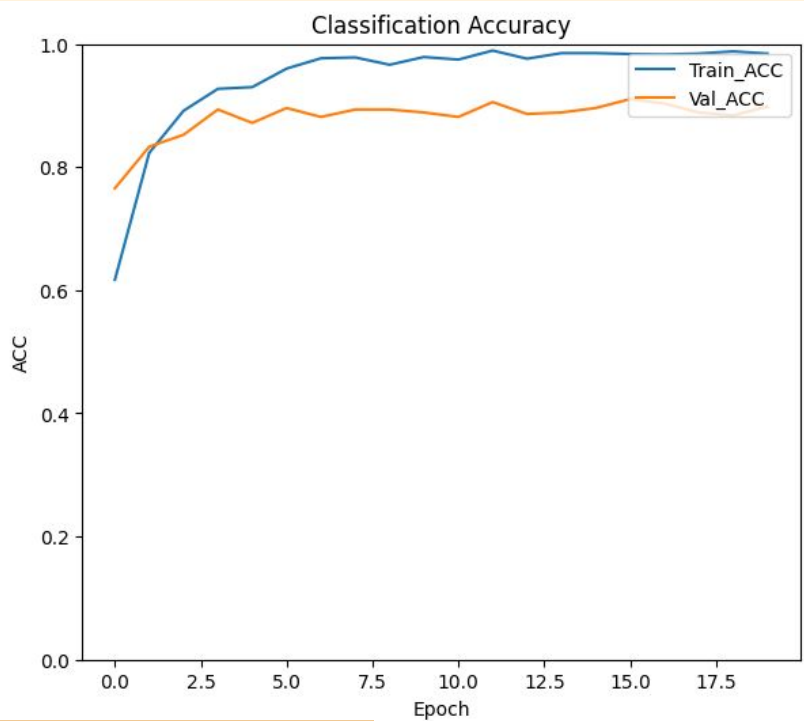
ResNet F1-Score

	Stage 1	Stage 2	Stage 3	Stage 4	Accuracy
Roboflow2	0.93	0.84	0.80	0.90	0.86
Roboflow1	0.14	0.00	0.31	0.26	0.22

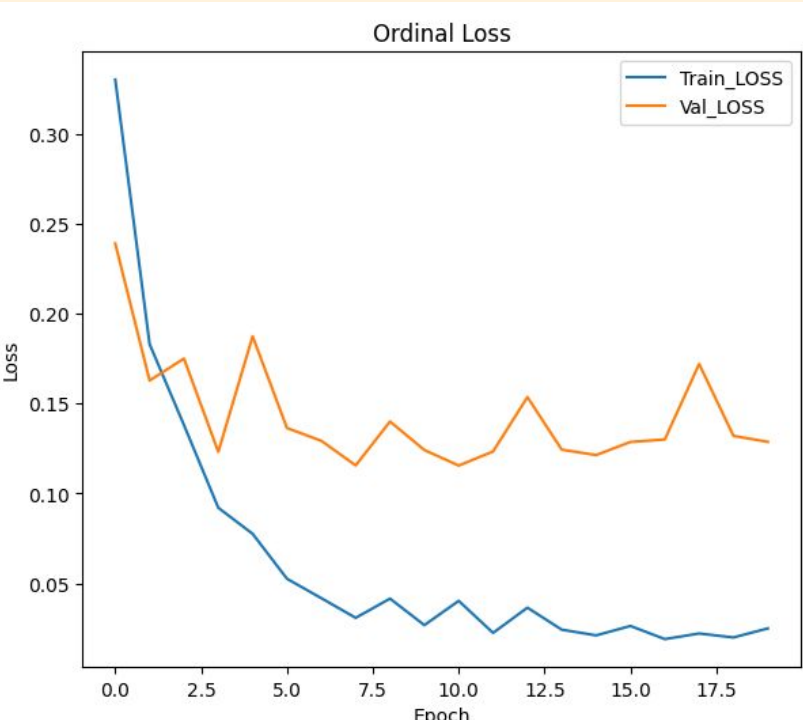
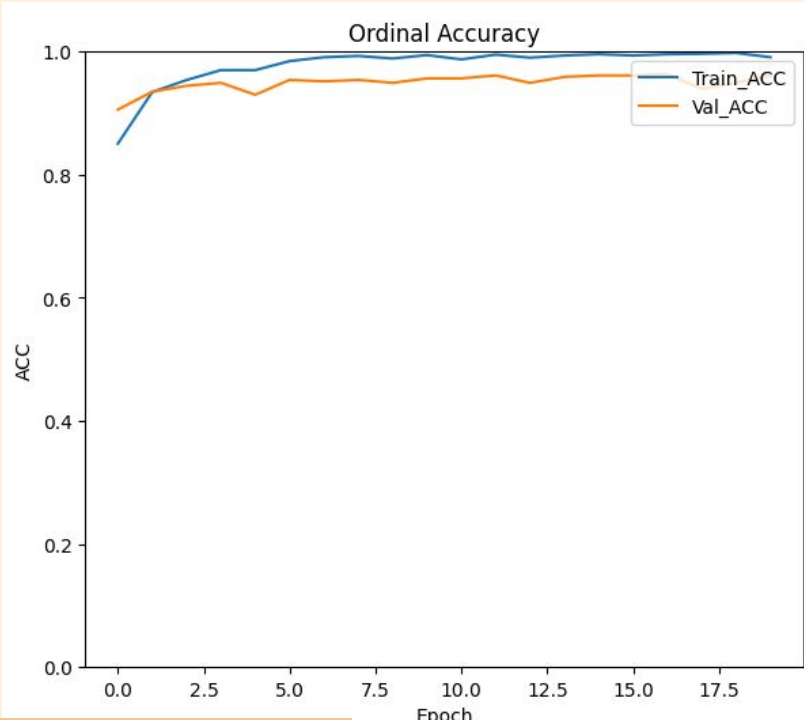
超參數調整:

	epochs / batch size	optimizer / lr / weight_decay	lr_scheduler/ step_size/gamma	loss function	Loss(weight/ label_smoothing)
Normal Classification	20/16	Adam/0.0001/1e-4	StepLR/5/0.3	CrossEntropyLoss	[1.0,1.0,1.0,1.0] / 0.0
Severity Classification	20/16	Adam/0.0001/1e-4	StepLR/5/0.3	CrossEntropyLoss	[1.0,1.0] / 0.0
Stage 3&4 Classification	20/16	Adam/0.0001/1e-4	StepLR/5/0.3	CrossEntropyLoss	[1.0,1.0,1.0] / 0.0

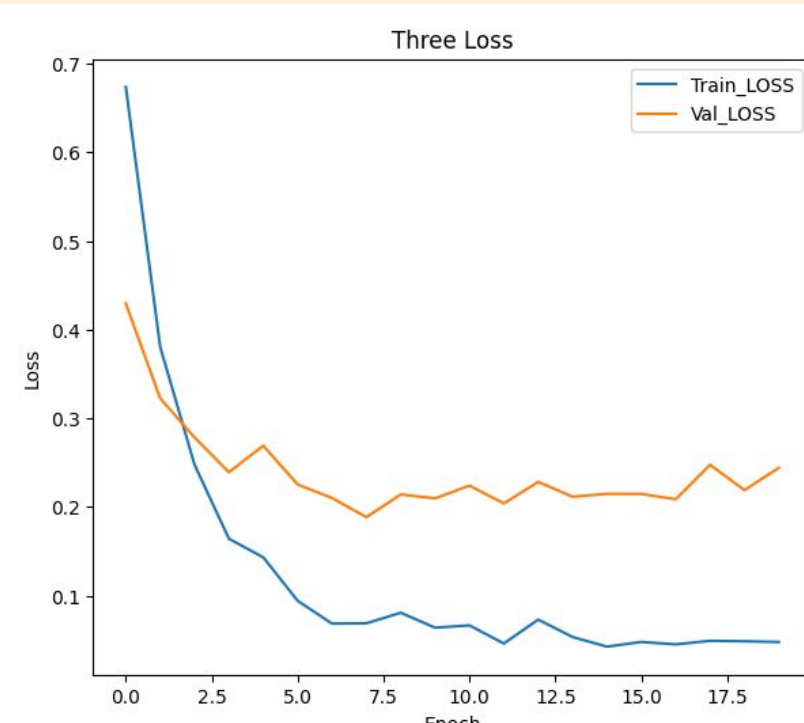
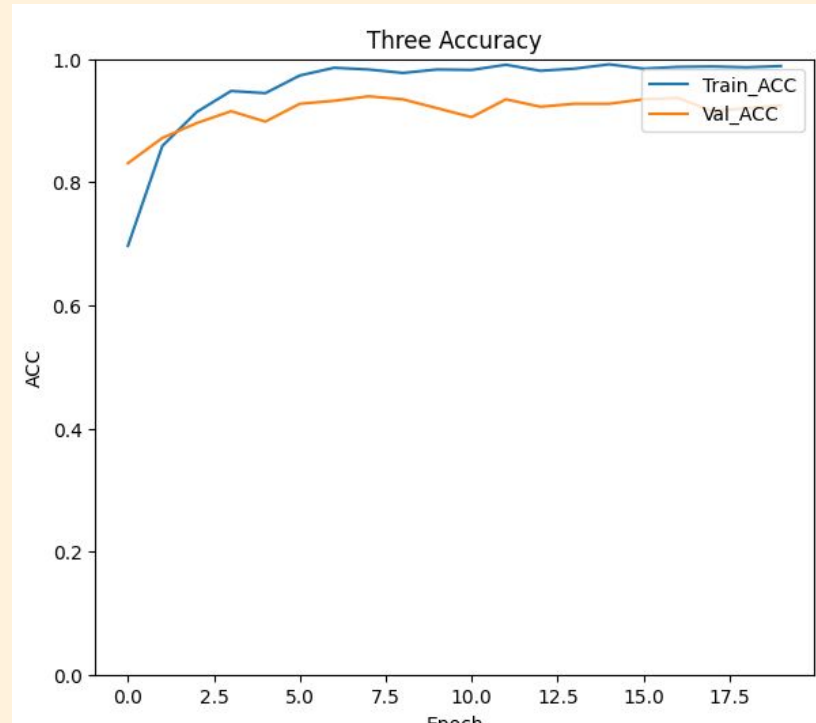




Normal
Classification



Severity
Classification



Stage 3&4
Classification

Conclusion

Model	Stage 1	Stage 2	Stage 3		Stage 4	Accuracy	Dataset
EfficientNet	0.91	0.83	0.80		0.91	0.8611	Roboflow 2
EfficientNet (Image reconstruction)	0.85	0.78	0.75		0.93	0.8241	Roboflow 2
EfficientNet (Ordinal)	0.67	0.03	0.47		0.61	0.50	Roboflow 2
EfficientNet (Multi-Task)	0.91	0.84	0.85		0.95	0.88	Roboflow 2

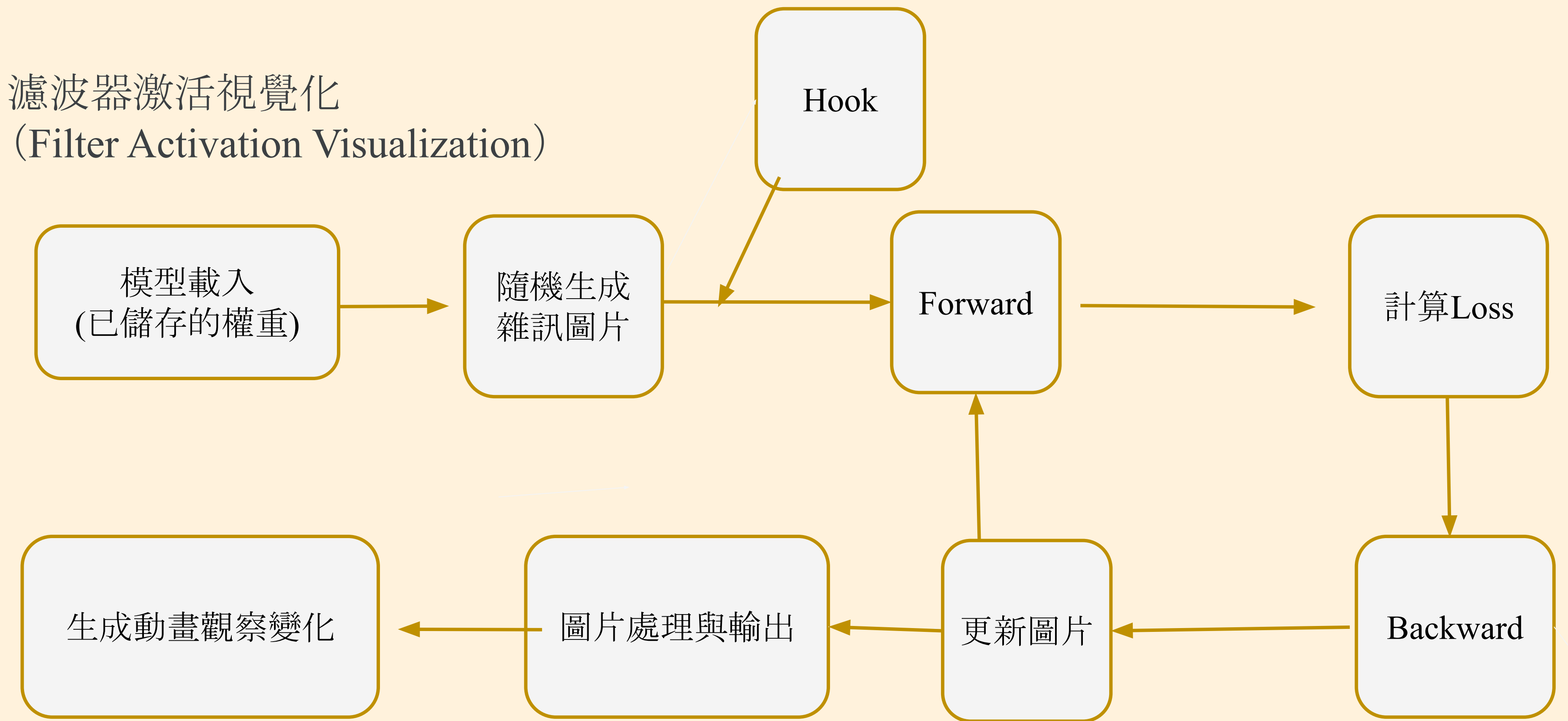
Conclusion

- The data and overall model performance of Roboflow1 are generally lower than those of Roboflow2.
- Multitask using EfficientNet achieved the best performance when trained on the Roboflow2 dataset.
- EfficientNet is better than ResNet

Multi-task 視覺化



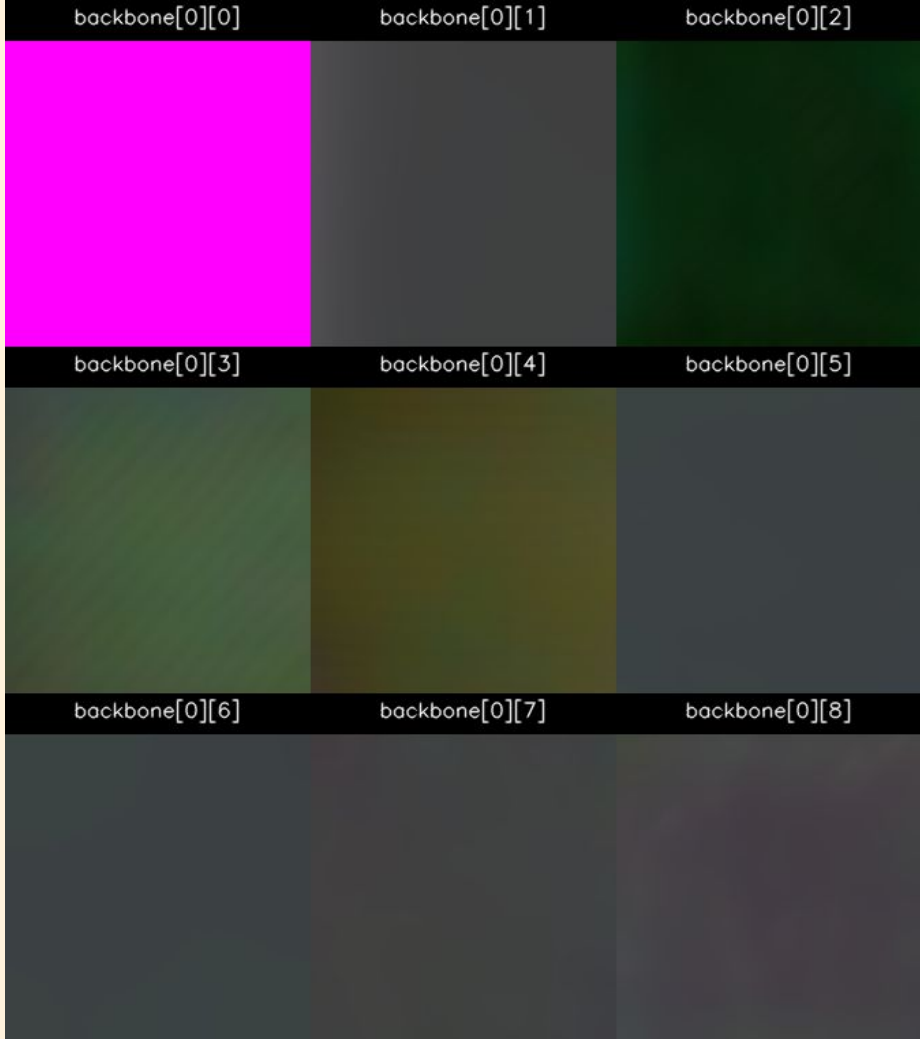
濾波器激活視覺化 (Filter Activation Visualization)



Multi-Task模型架構

- EfficientNetb2(backbone)
 - backbone[0][0] (初始卷積層) => efficientnet_blocks[0]
 - backbone[0][1] ~ backbone[0][7] => efficientnet_blocks[1]~[7]
 - MBConv (Mobile Inverted Bottleneck Convolution)
 - Conv2d(卷積)Norm(正規)Activation(激勵)
 - Squeeze(壓縮)Excitation(激勵)
 - backbone[0][8] (最終卷積層) => efficientnet_blocks[8]
- NormalClassHead
- OrdinalHead
- ThreeHead

EfficientNetb2



EfficientNetb2

+

一般分類



EfficientNetb2

+

嚴重程度分類



EfficientNetb2

+

Stage1&2

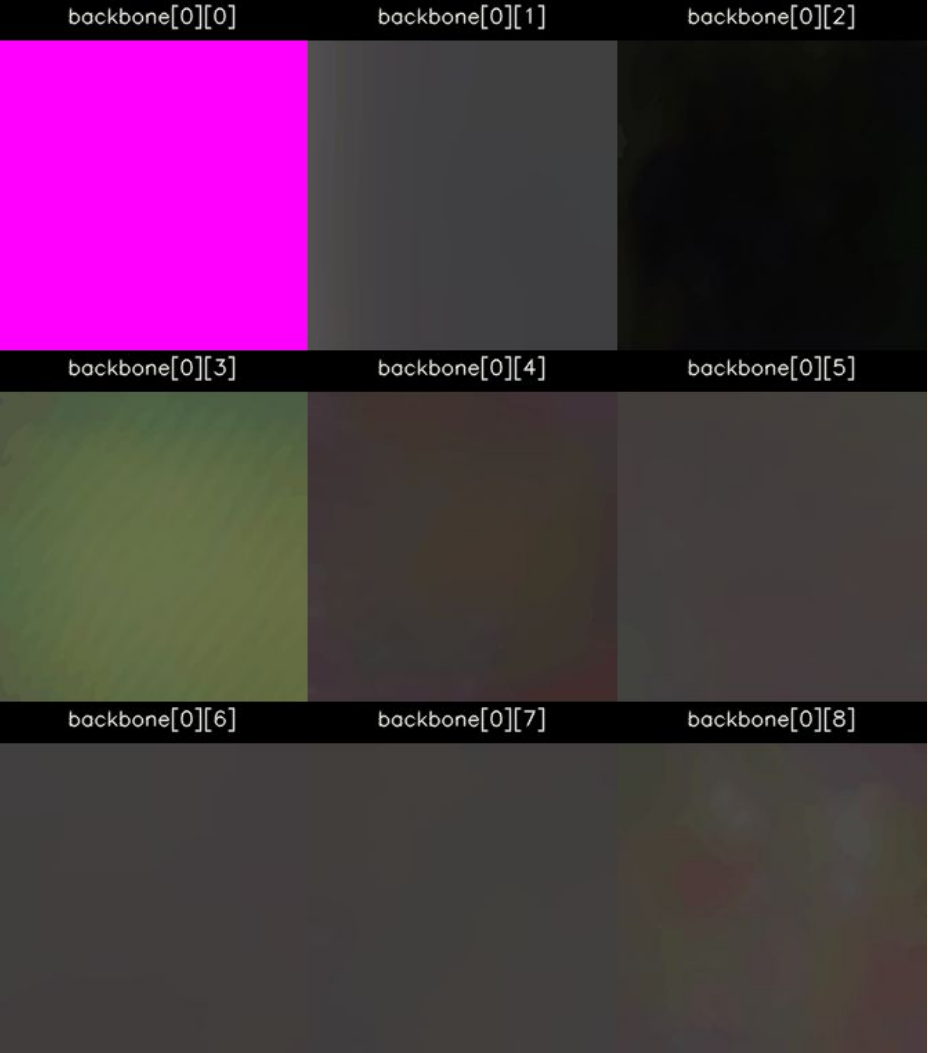
Stage3/Stage4

分類



每輪epoch

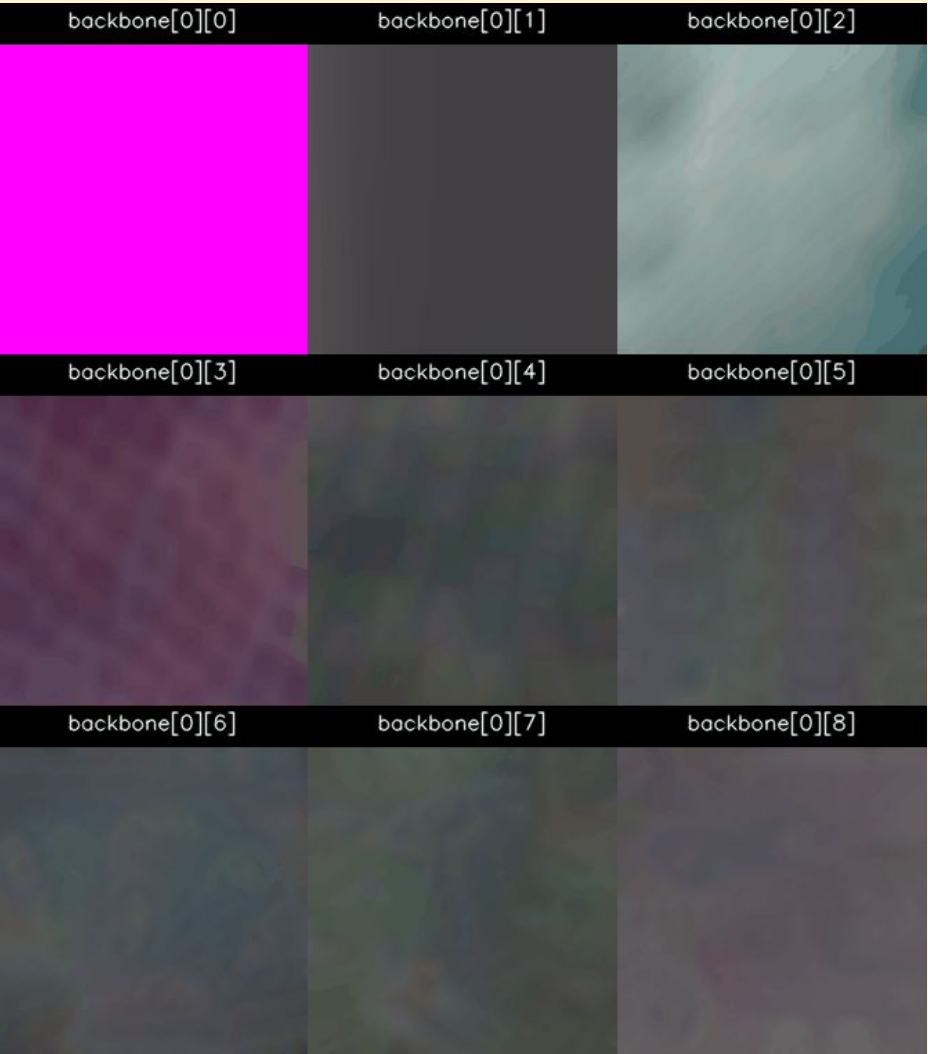
EfficientNetb2



EfficientNetb2

+

一般分類



[如若gif無法播放, 請點擊](#)

EfficientNetb2

+

嚴重程度分類



EfficientNetb2

+

Stage1&2

Stage3/Stage4

分類



剪枝pruning

- 模型參數量:

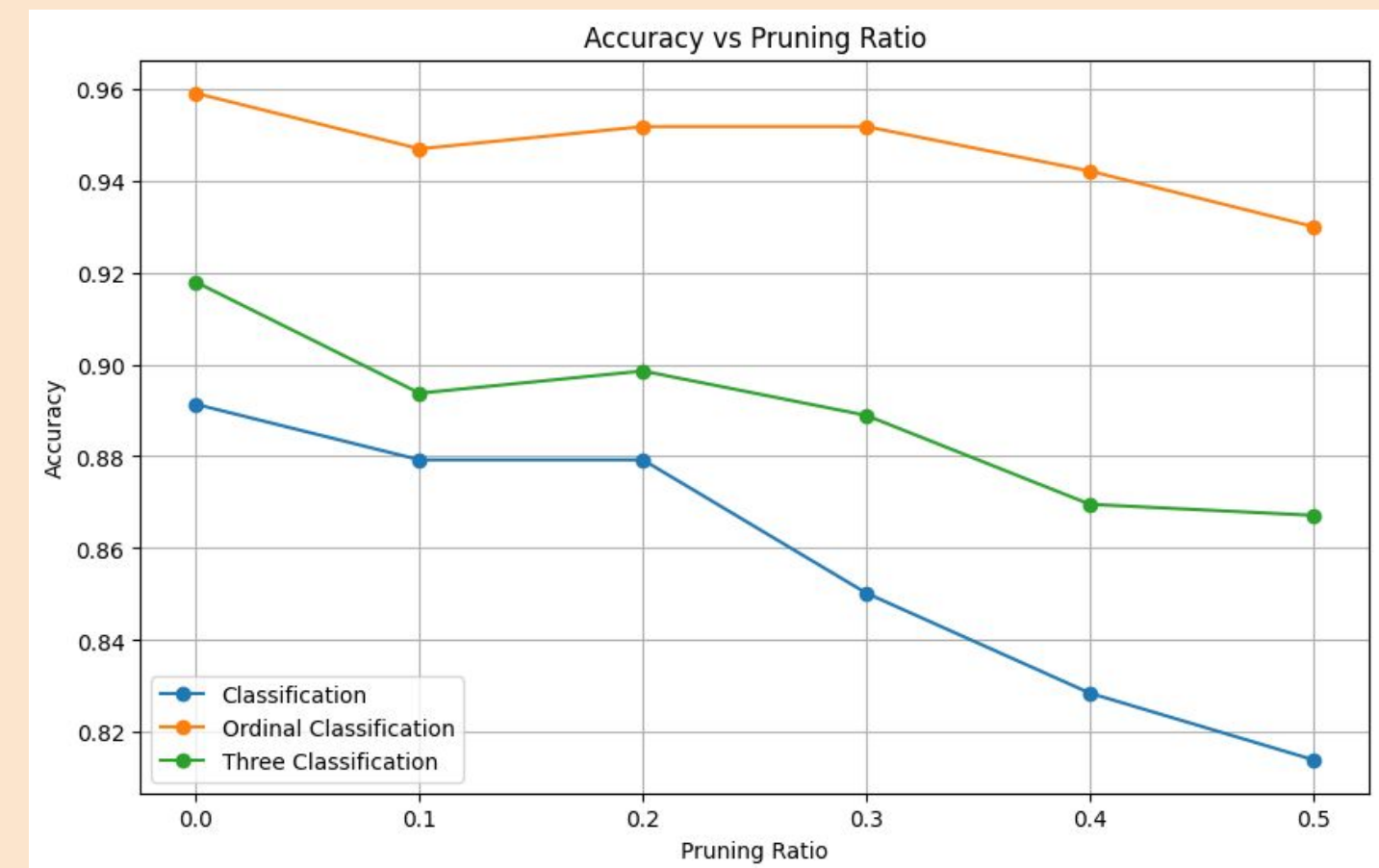
SharedEfficientNetBackbone Parameters: 7,700,994 → **6,160,795**

NormalClassHead Parameters: 724,484

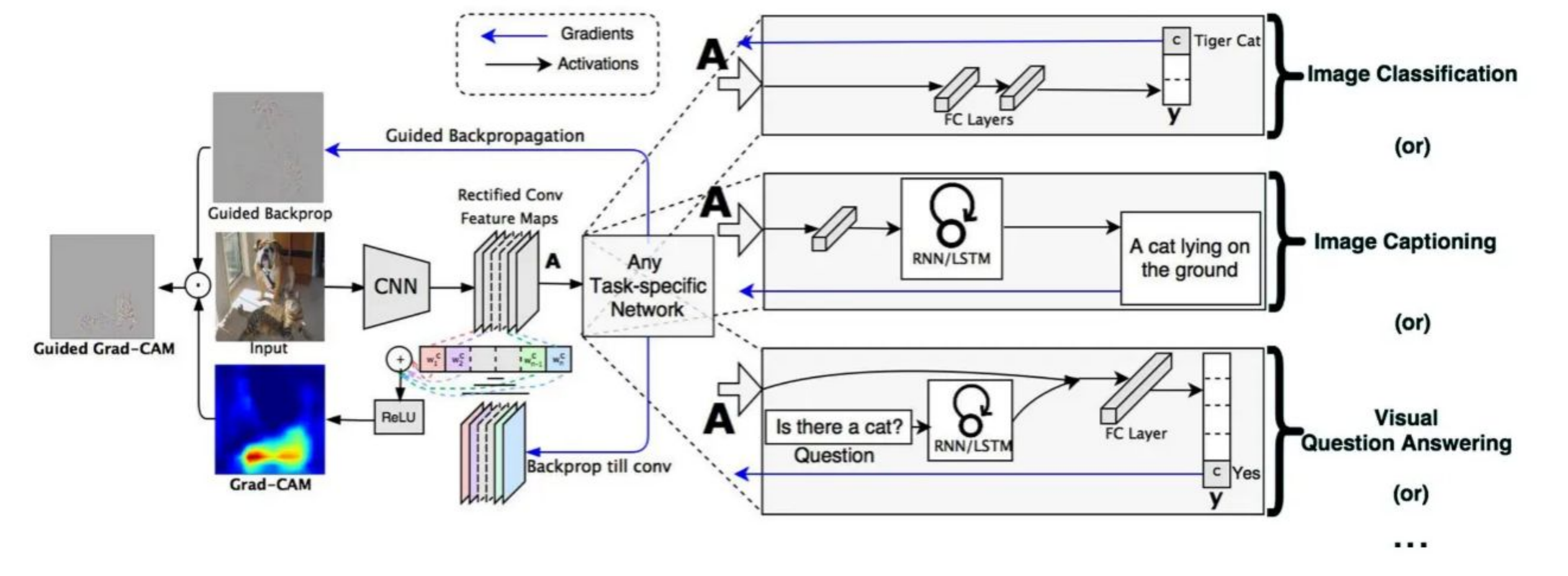
OrdinalHead Parameters: 723,458

ThreeHead Parameters: 723,971

Total Parameters: 9,872,907

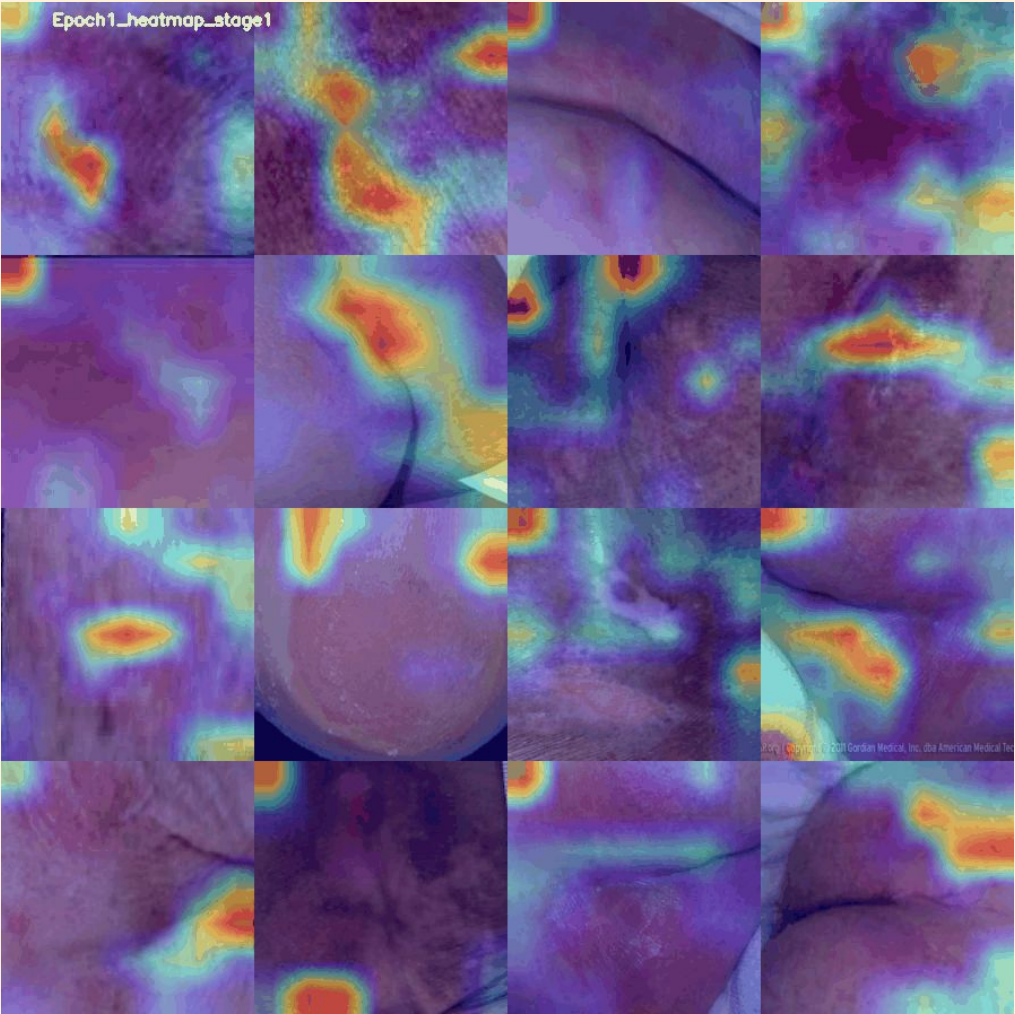


分類關注區域

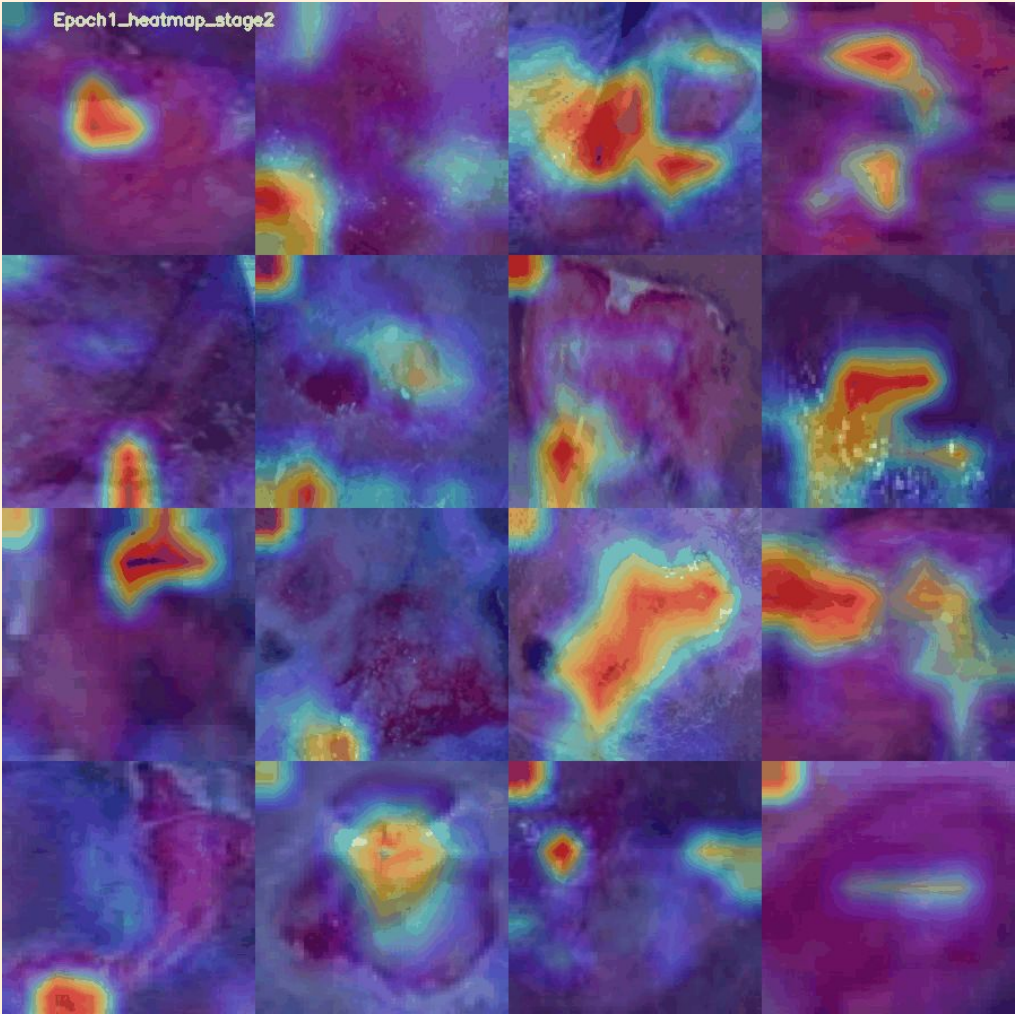


一般

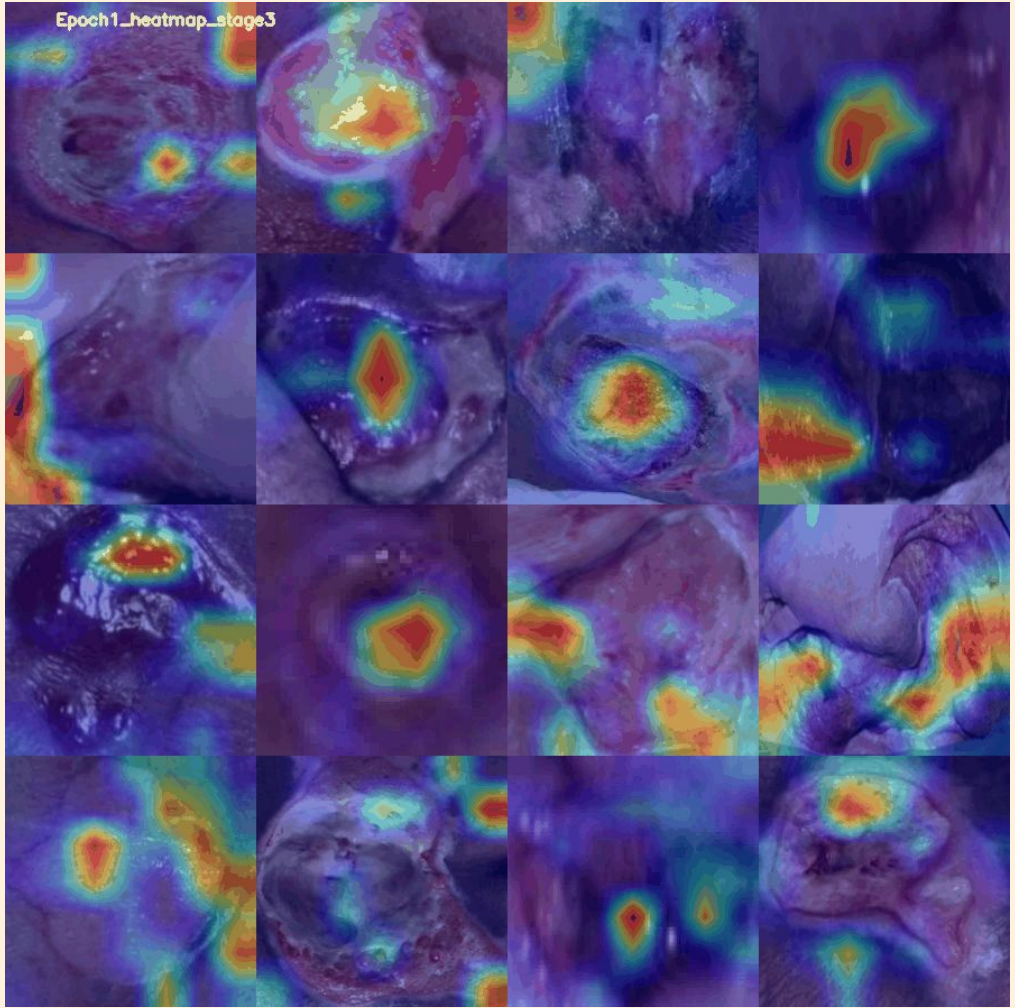
Stage1



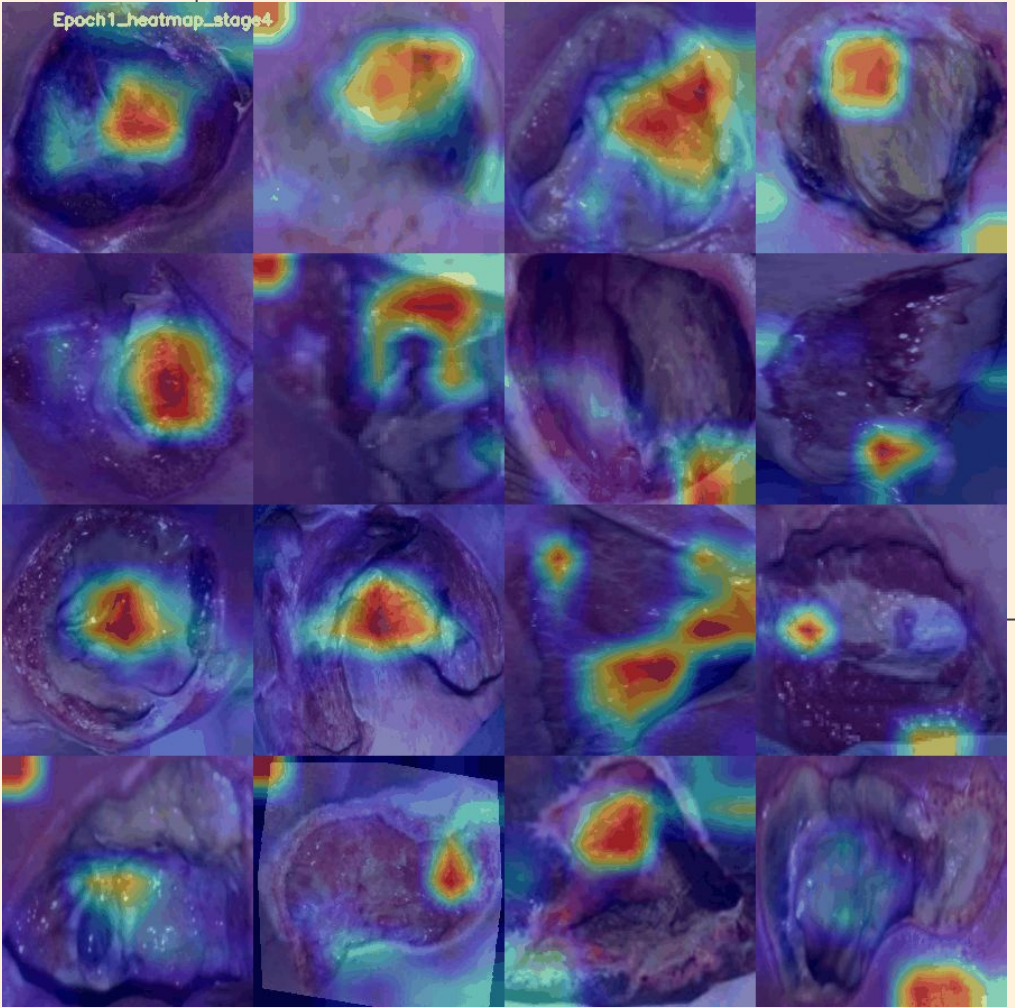
Stage2



Stage3

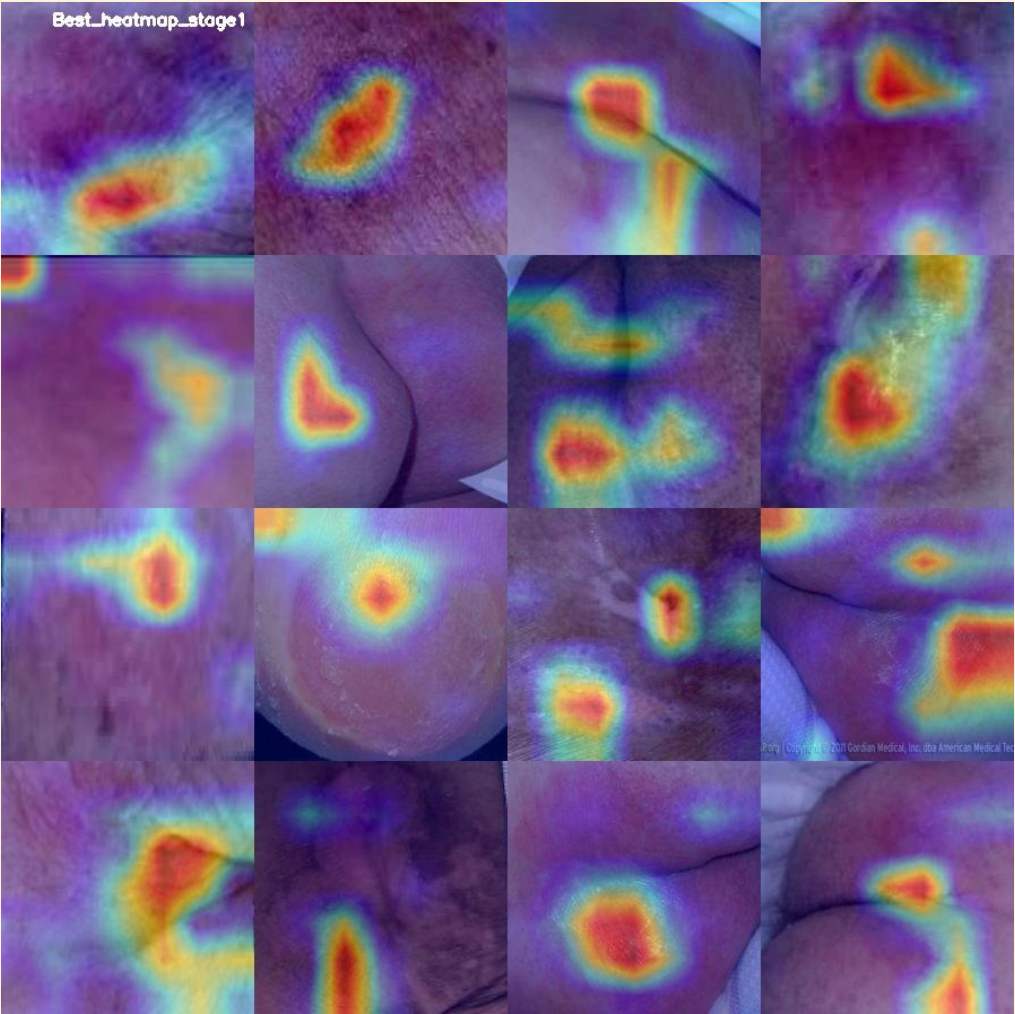


Stage4

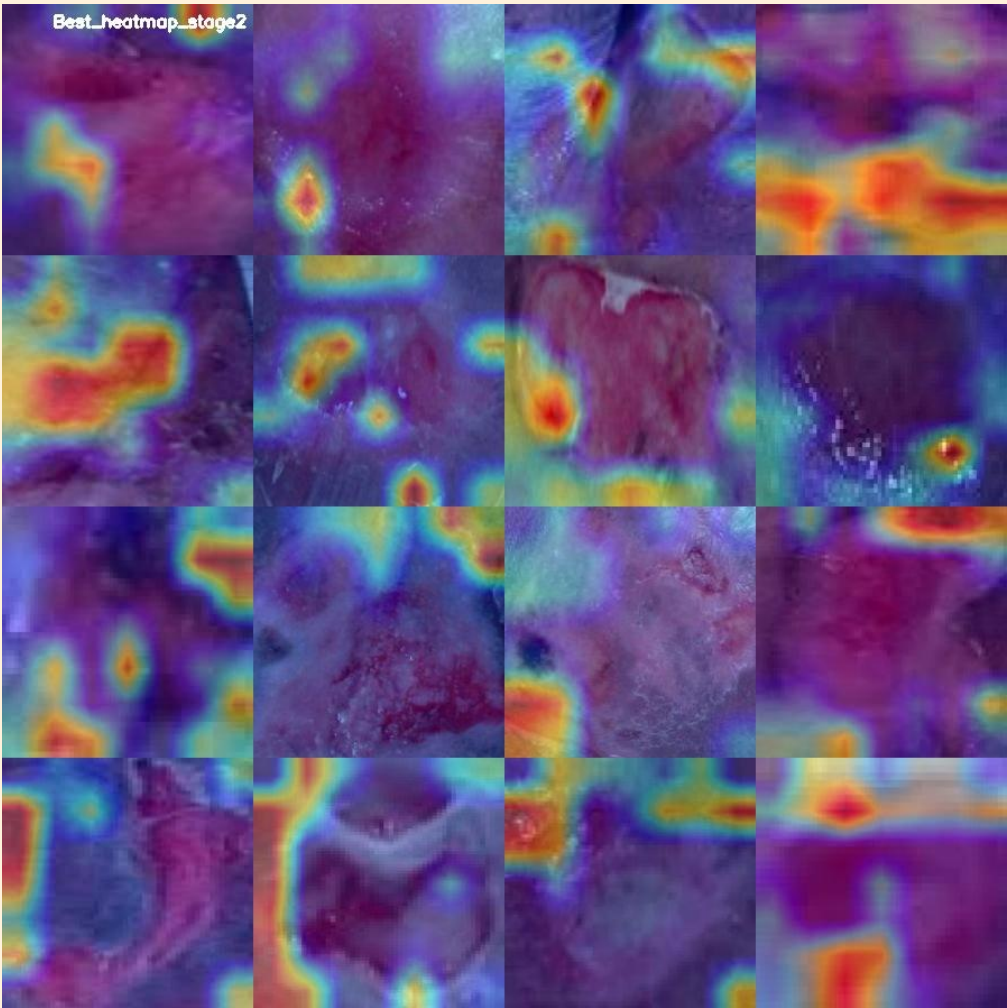


一般Best

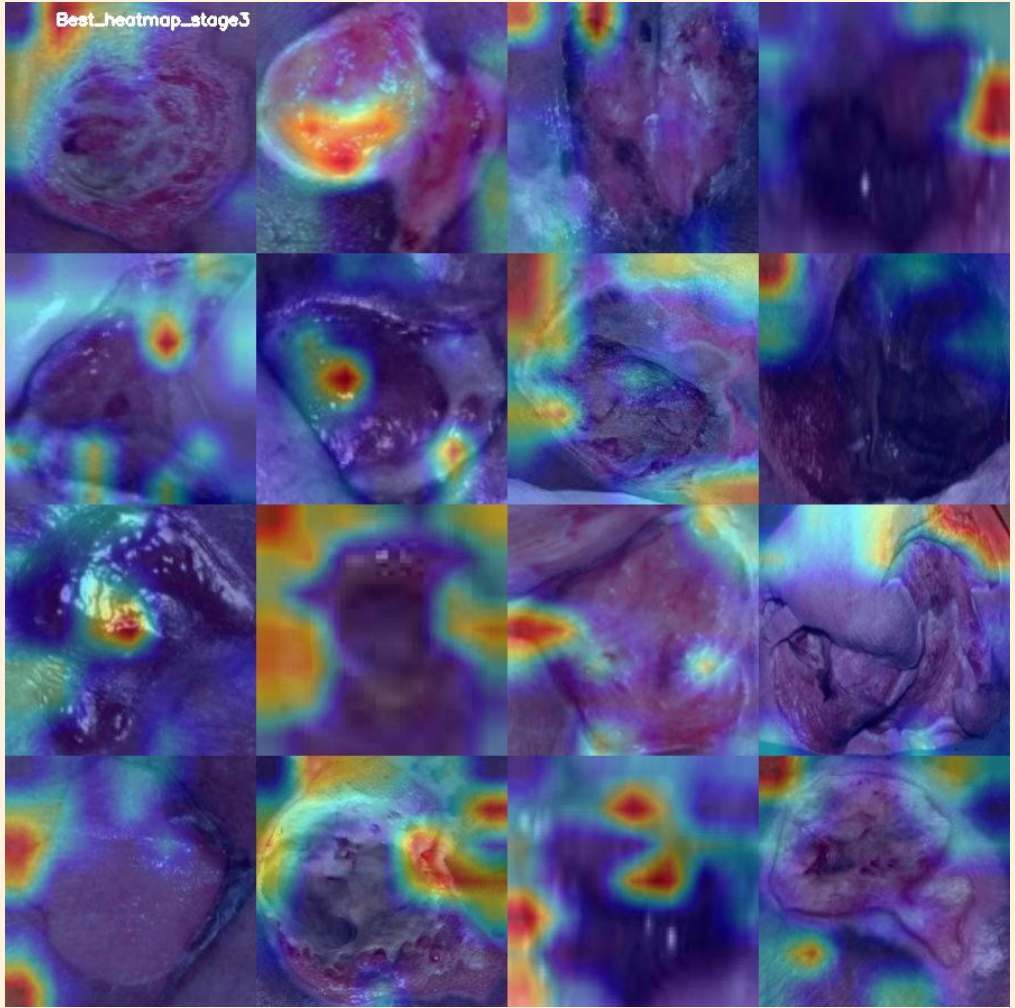
Stage1



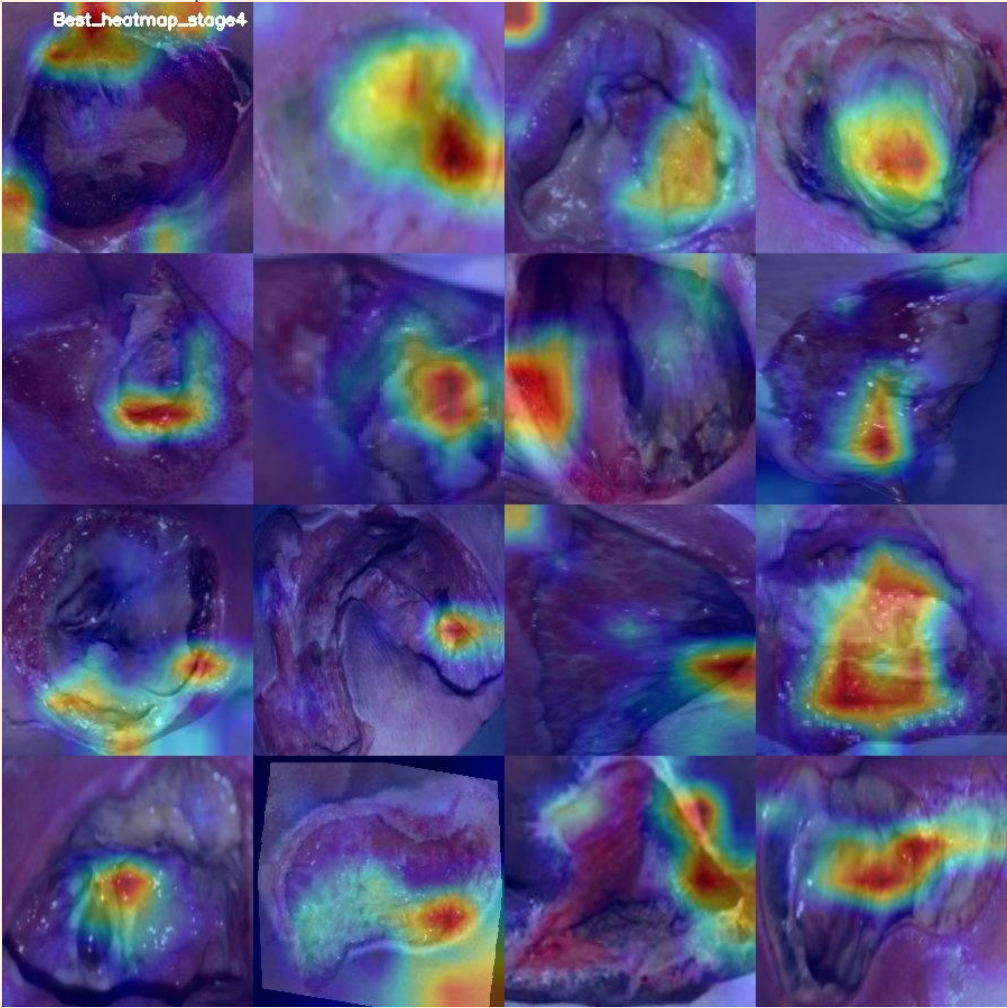
Stage2



Stage3

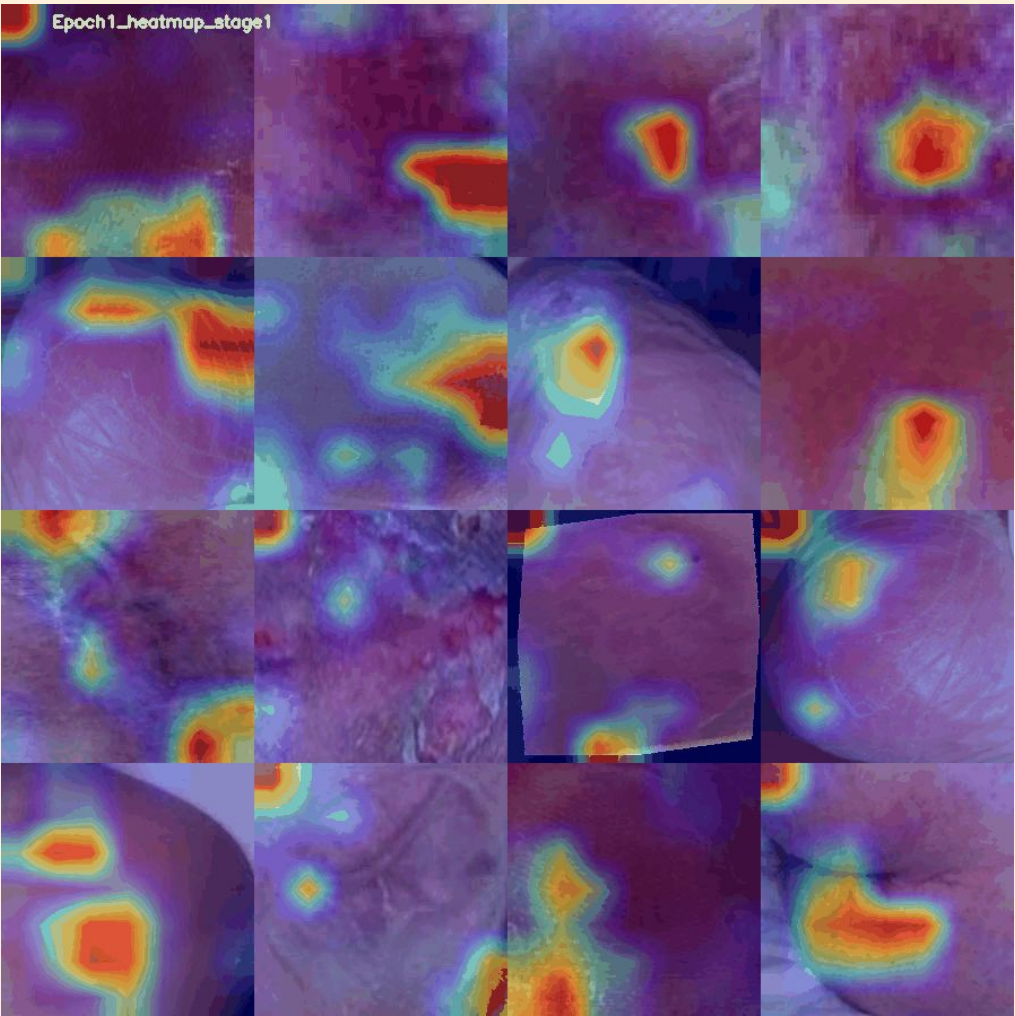


Stage4

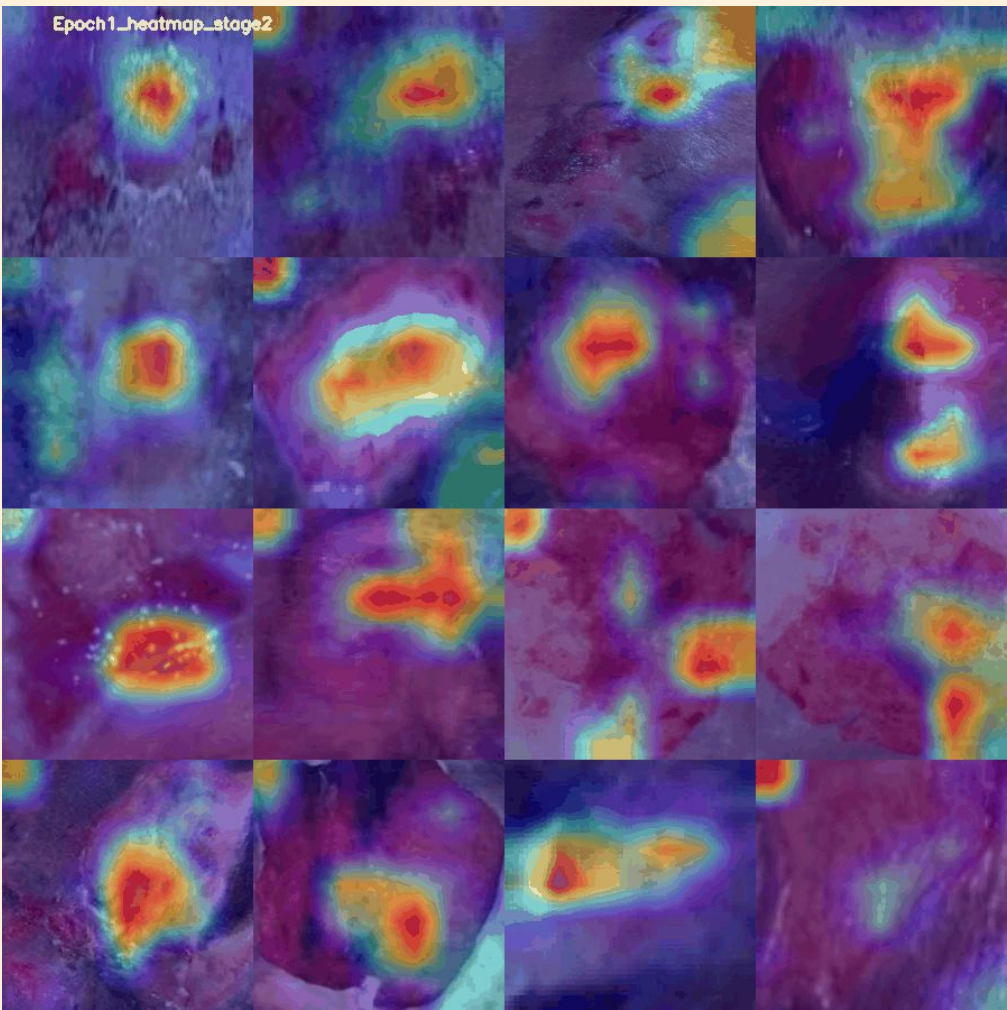


輕重度

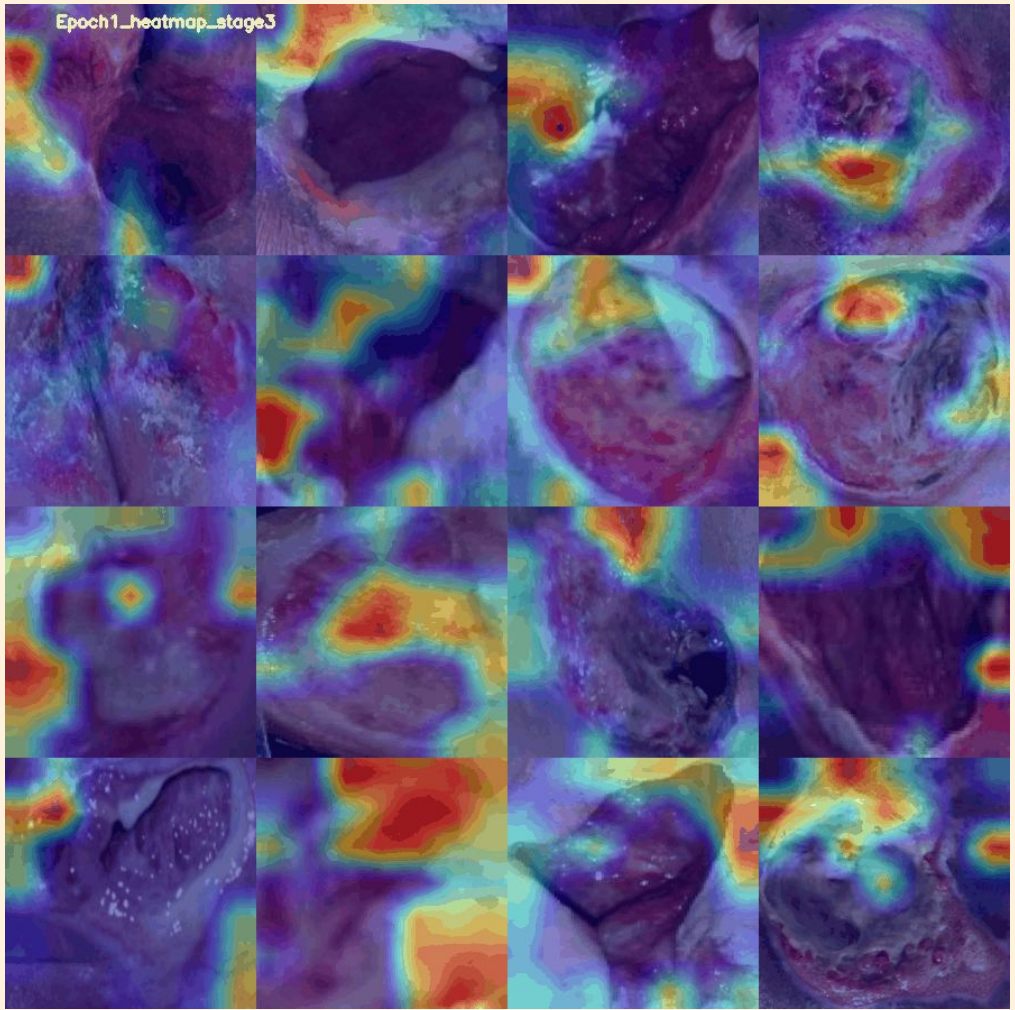
Stage1



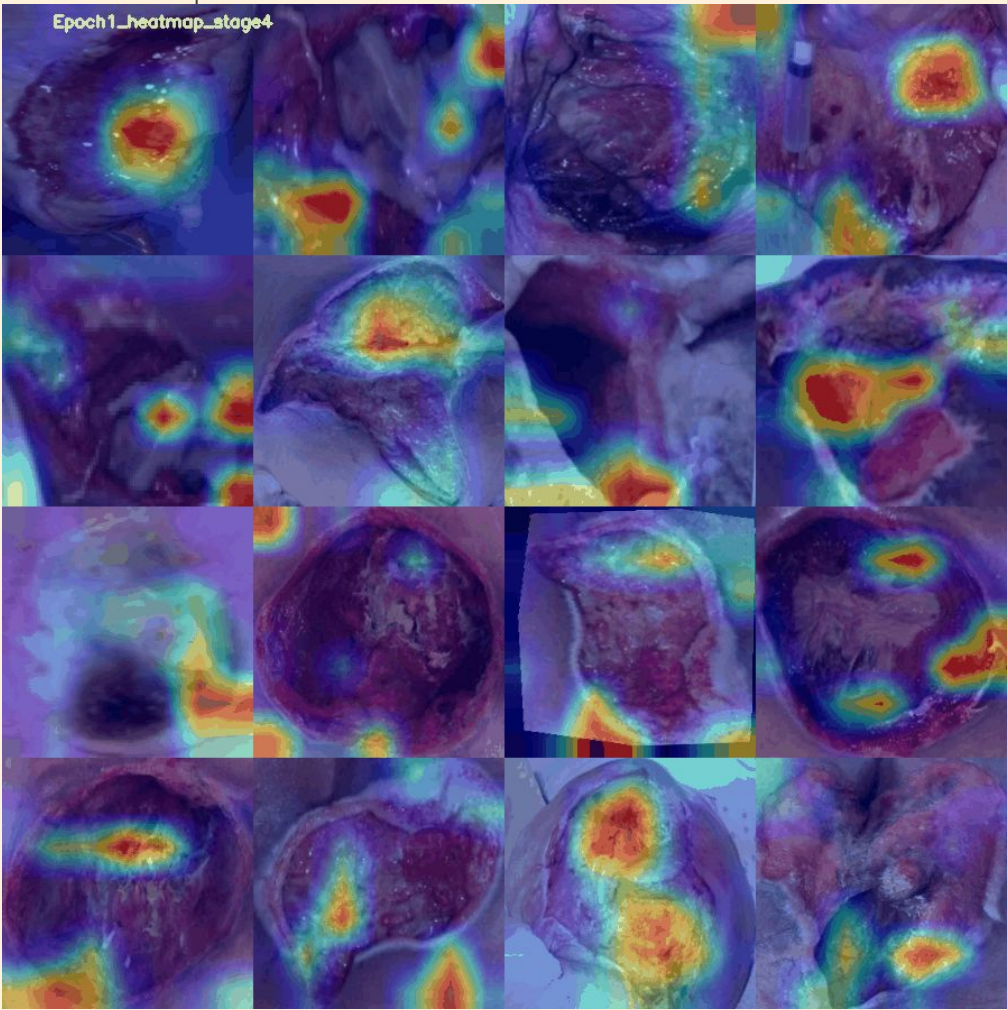
Stage2



Stage3

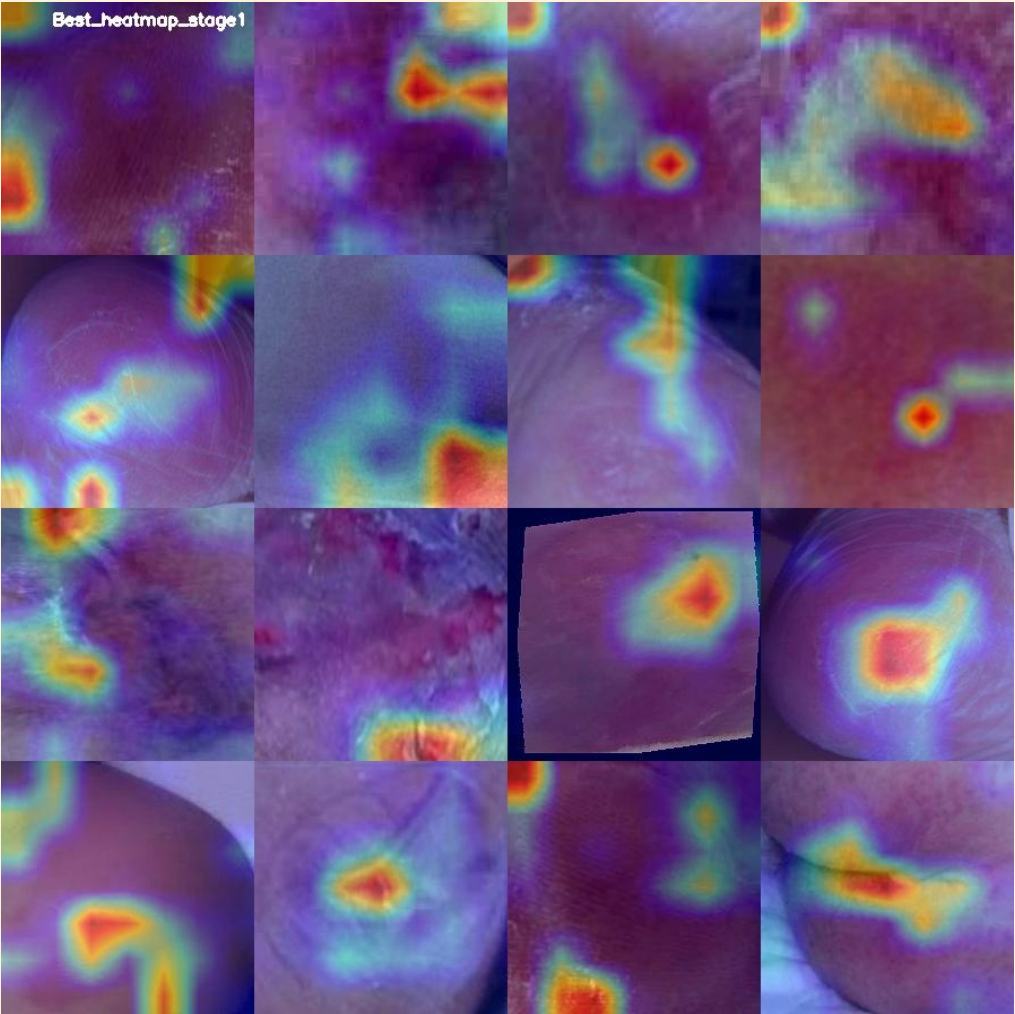


Stage4

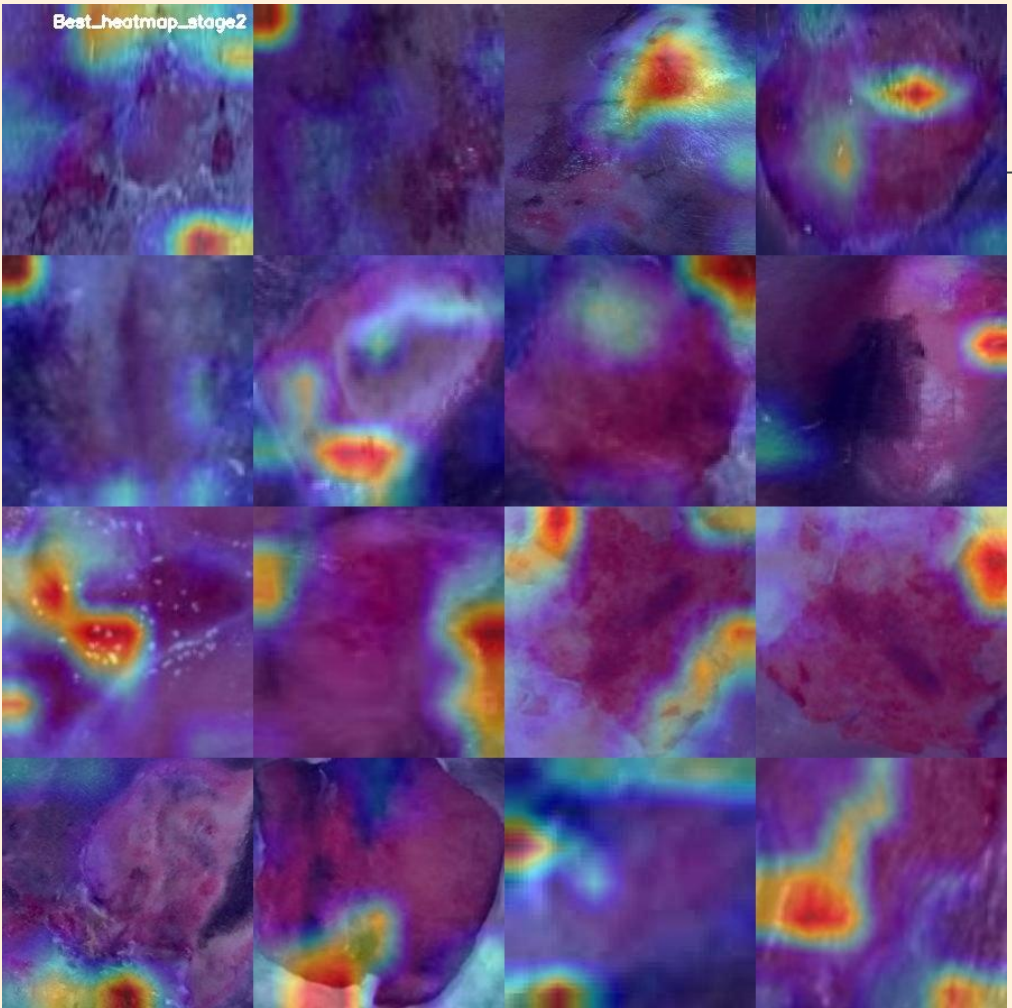


輕重度Best

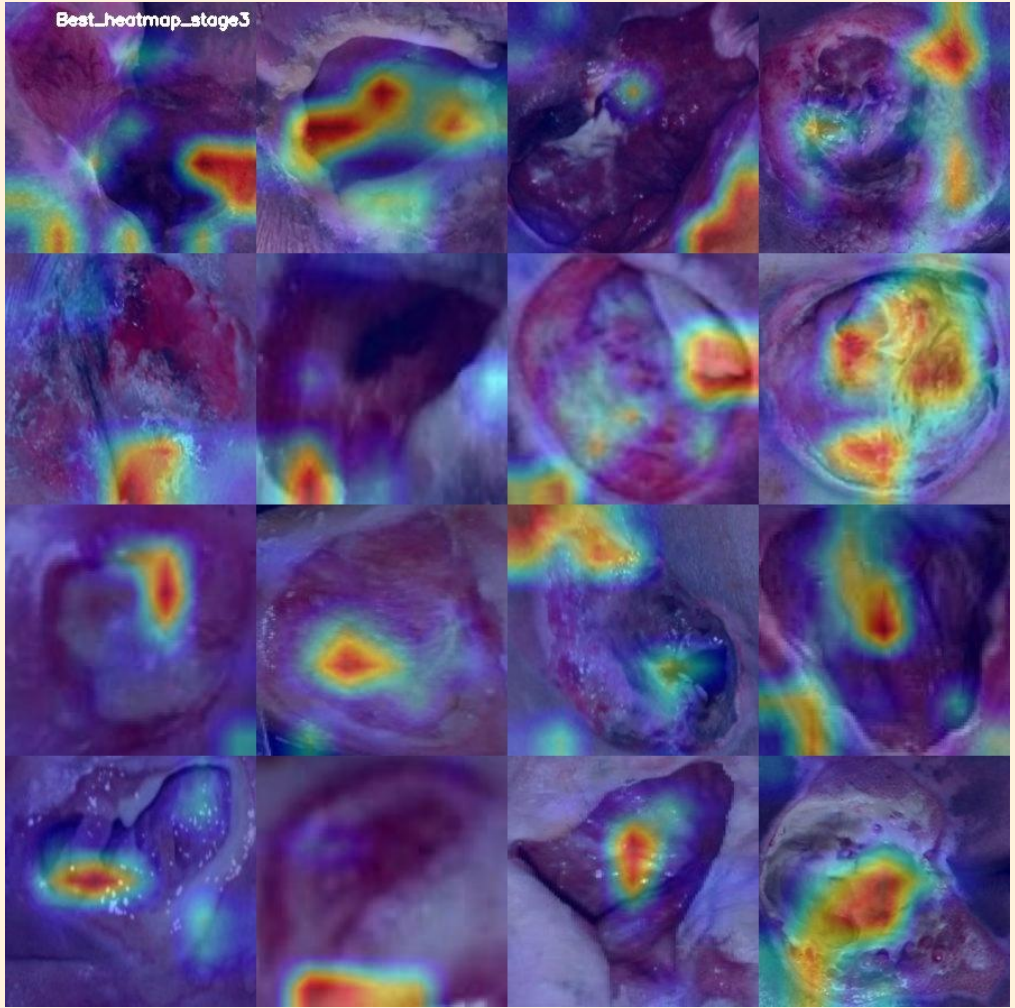
Stage1



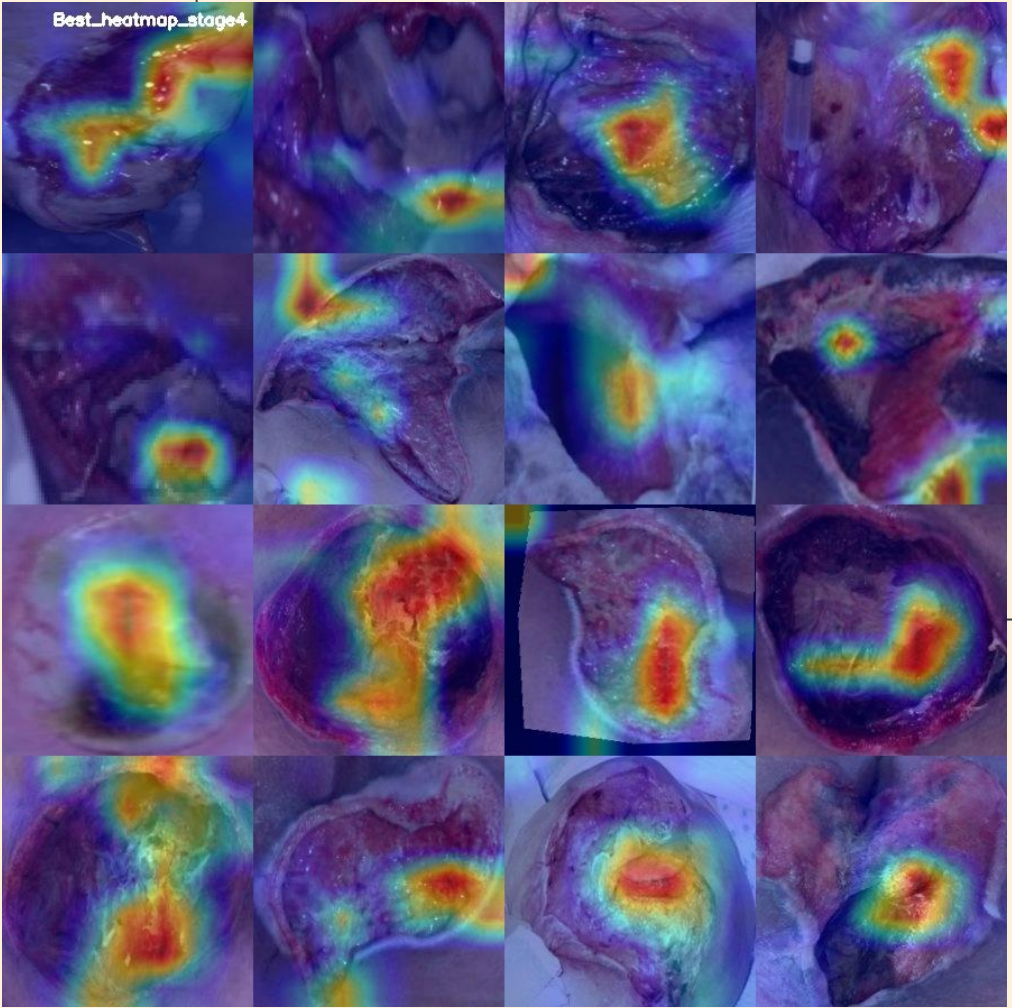
Stage2



Stage3

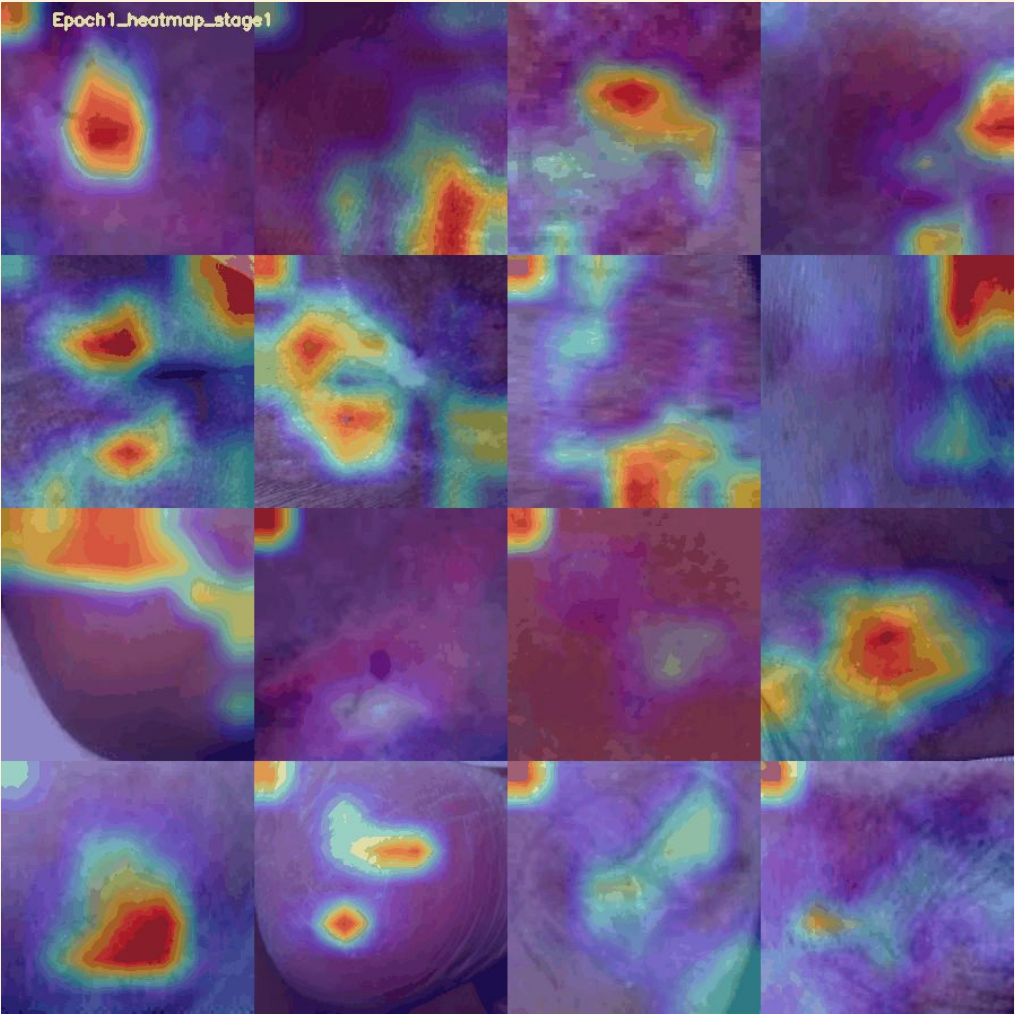


Stage4

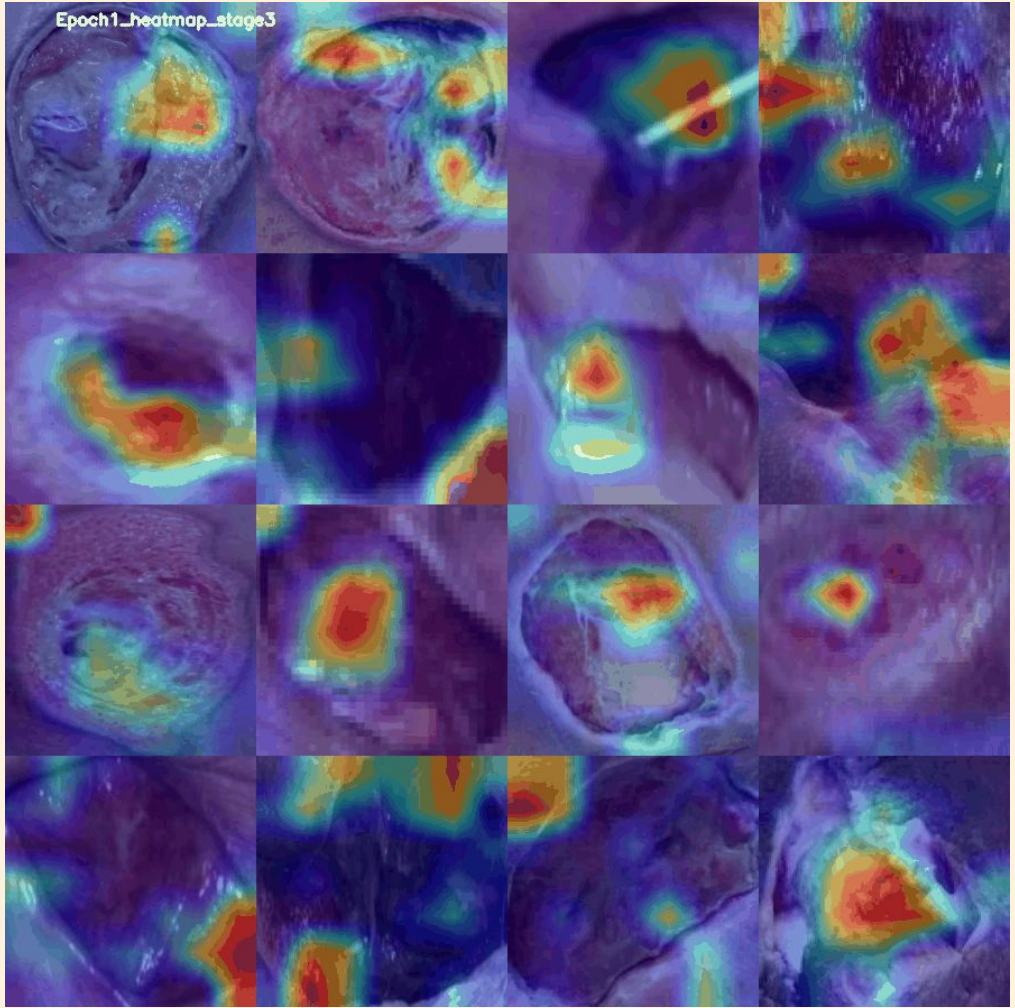


1&2級合併

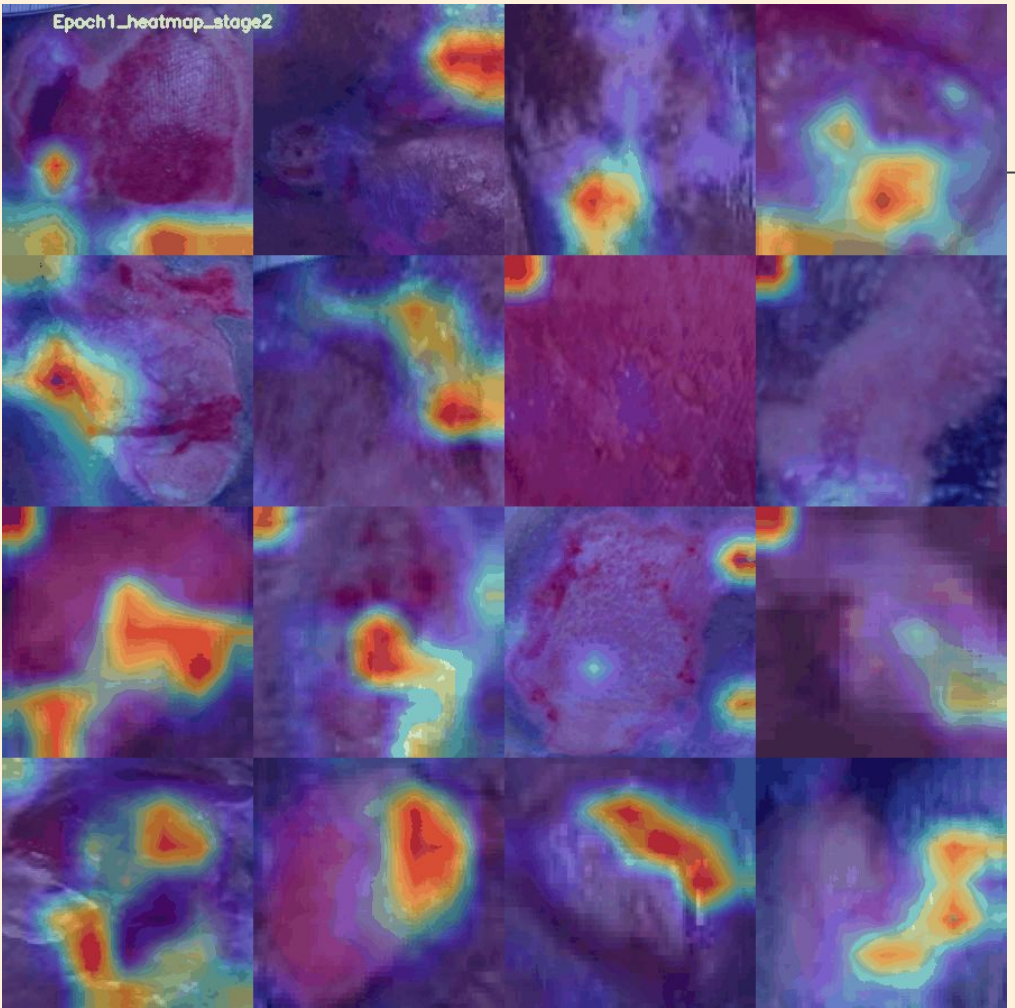
Stage1



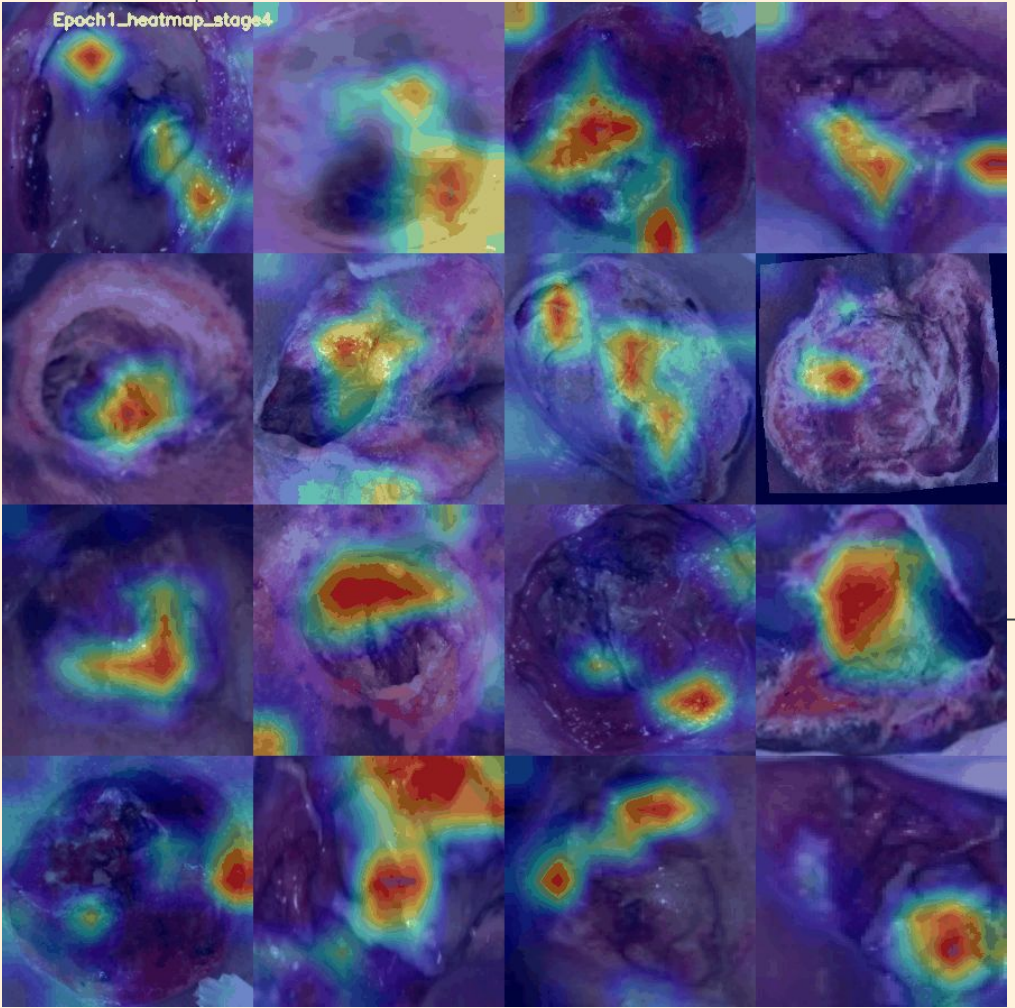
Stage3



Stage2

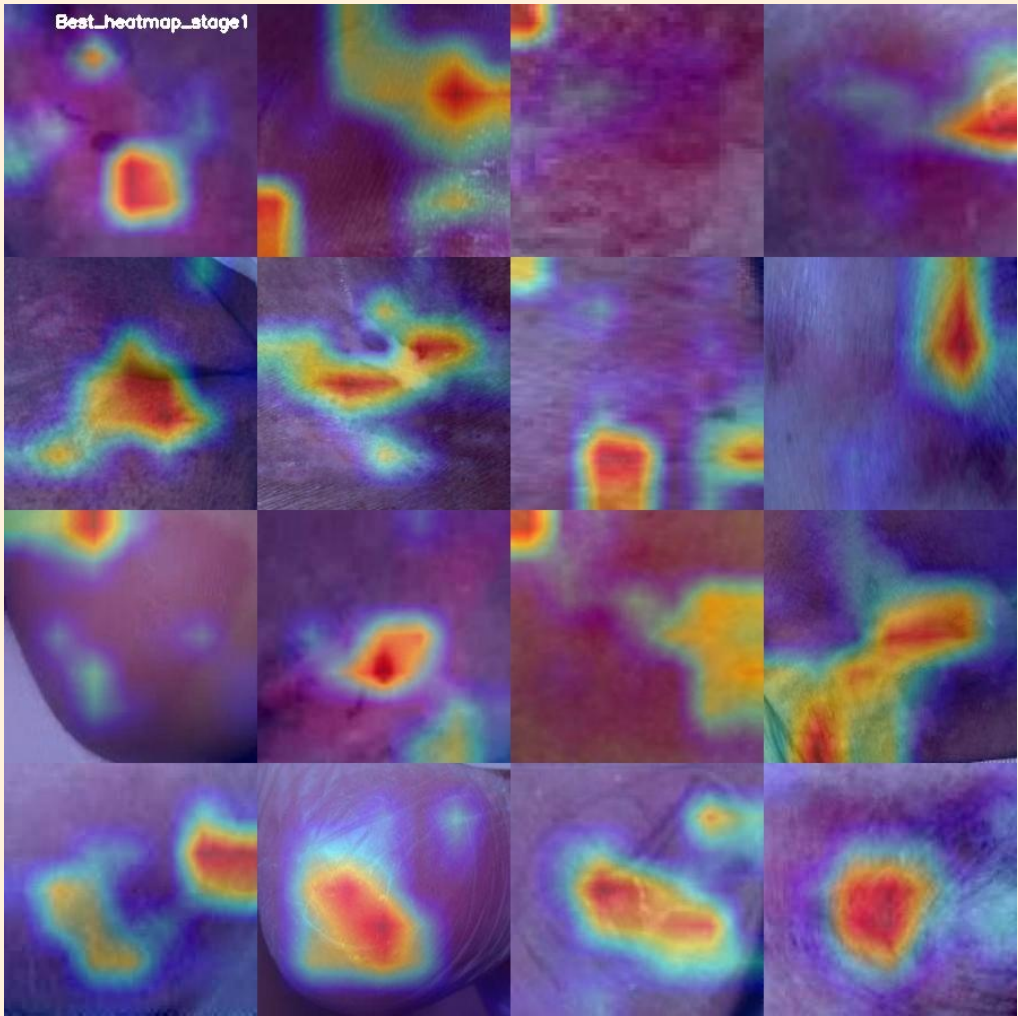


Stage4

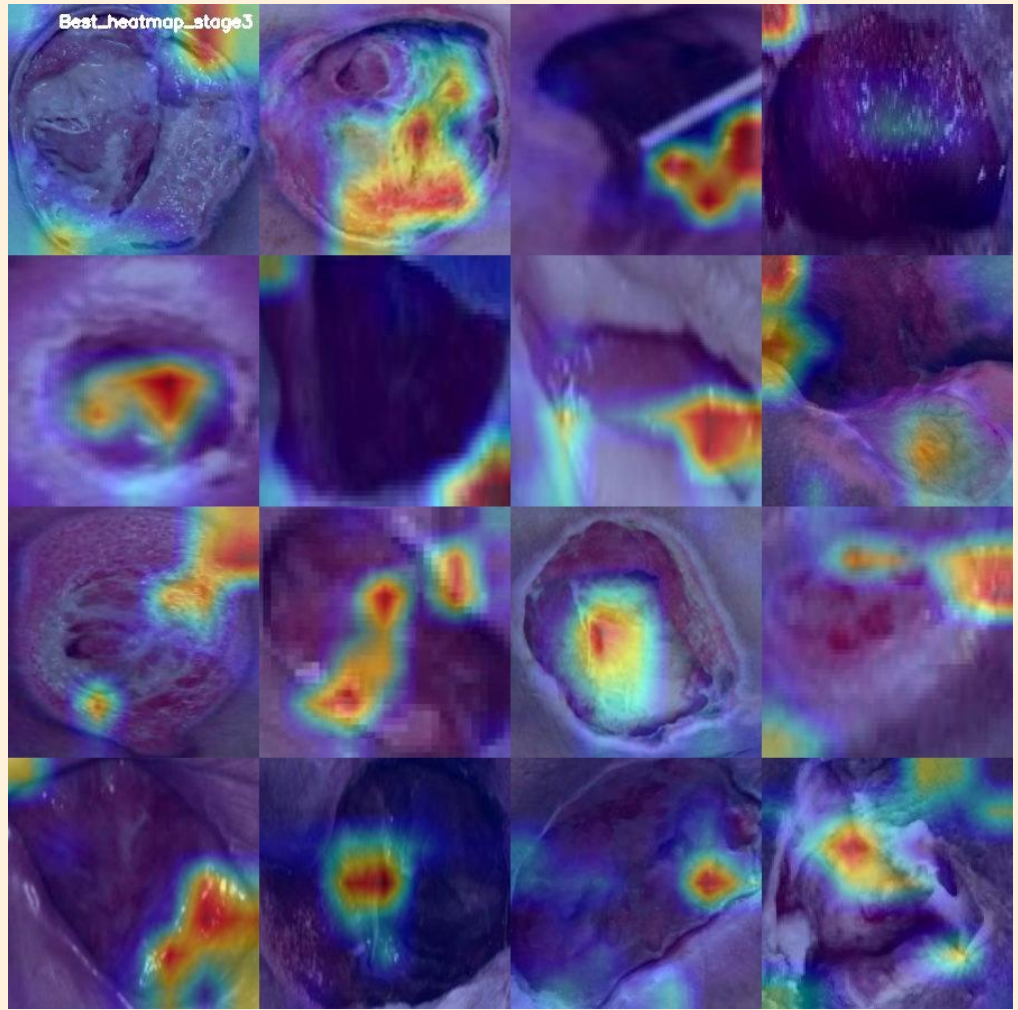


1&2級合併Best

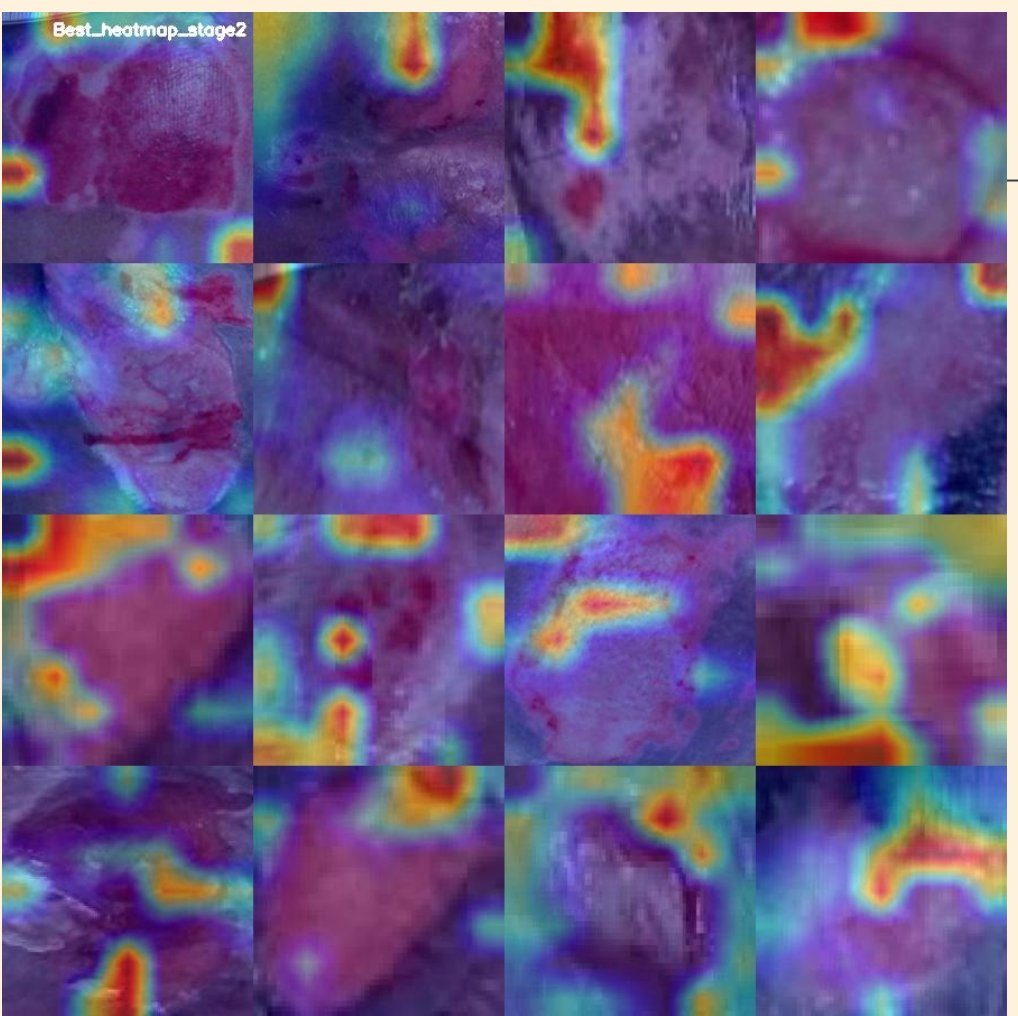
Stage1



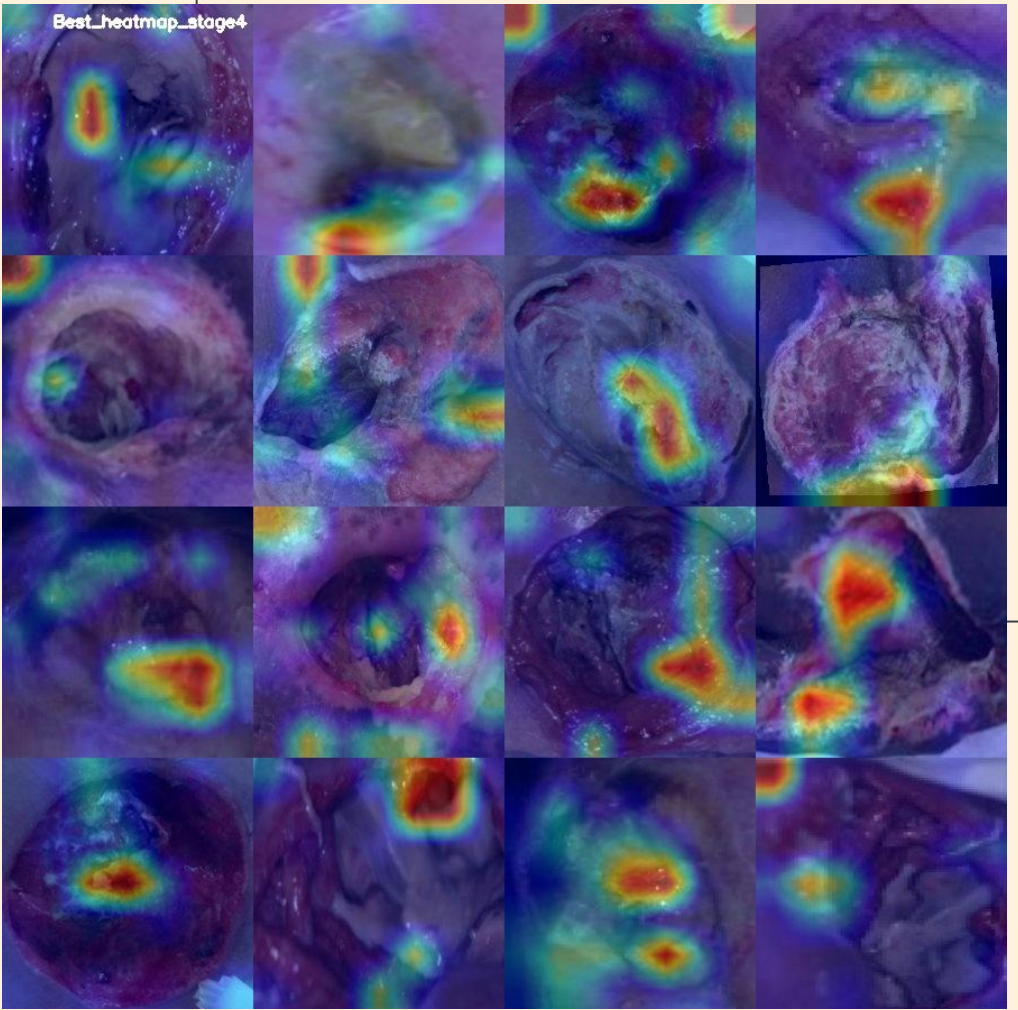
Stage3



Stage2



Stage4




```

target_layer=model0.backbone[0][-1]
hook_handle = target_layer.register_forward_hook(forward_hook)
output = model0(input1)
output = model1(output)
loss = criterions[0](output,labels1[2])
model0.zero_grad()
model1.zero_grad()
loss.backward()
hook_handle.remove()
fe=activation_grad.detach().cpu().numpy()
fe=np.mean(fe,axis=(0,2,3))
ac=activation.detach().cpu().numpy().copy()

```

```

for j in range(ac.shape[1]):
    ac[:, j, :, :] *= fe[j]
heatmap = np.sum(ac, axis=1).squeeze()
# print(heatmap.shape)
heatmap = np.maximum(heatmap, 0)
if np.max(heatmap) != 0:
    heatmap /= np.max(heatmap)

```

```

in1=input1.detach().cpu().numpy().reshape(16,-1)
max= np.max(in1,axis=1).reshape(16,1)
min= np.min(in1,axis=1).reshape(16,1)
im=((in1-min)/(max-min))*255).astype(np.uint8).reshape(16,3,224,224).transpose(0,2,3,1).reshape(4,4,224,224,3).transpose(0,2,1,3,4).reshape(224*4,224*4,3)
h=[]
for i in range(16):
    temp=cv2.resize(heatmap[i], (224,224))
    maxh=np.max(temp)
    minh=np.min(temp)
    eps = 1e-8
    temp=((temp-minh)/(maxh-minh+ eps))*255).astype(np.uint8)
    temp=cv2.cvtColor(cv2.applyColorMap(temp, cv2.COLORMAP_JET), cv2.COLOR_BGR2RGB)
    h.append(temp)
h=np.array(h)
h=h.reshape(4,4,224,224,3).transpose(0,2,1,3,4).reshape(4*224,4*224,3)
# plt.imshow(h)
img=np.zeros_like(im)
img[:, :, 0]=im[:, :, 0]*0.6+h[:, :, 0]*0.4
img[:, :, 1]=im[:, :, 1]*0.6+h[:, :, 1]*0.4
img[:, :, 2]=im[:, :, 2]*0.6+h[:, :, 2]*0.4
img=cv2.putText(img, f'Best_heatmap_stage1', (45,20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,255,255), 2)
img=cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
cv2.imwrite('./gifs/Best_heatmap_stage1.jpg',img)

```

forward:抓取卷核之輸出值(featuremap)
backward:取得最後分類輸出之loss倒傳遞計算號對特徵卷積輸出的lgrad

將經過1408個特徵卷積和卷積而得的中間特徵輸出圖，去與loss計算而得到他們之間的grad，並且，對1408張特徵圖中的每個像素之grad取平均，等於將7*7取平均而得1408，目的是為了將其與特徵輸出圖中的值，做相乘而得該特徵圖中的值對分類輸出的相關重要度，最後將所有圖加總的到特定位置之特徵總資訊對分類輸出的相關重要度

將7*7的特徵圖利用雙線性差值縮放到與原圖同樣大小，再對其依照像素值大小(=在分類上的重要度)給予連續顏色(jet對應)，以利於可視化上的直觀性(像素值越大越接近藍->紅中紅色那側顏色，代表越重要)

用以做出cam中的下圖功能

